

**Санкт-Петербургский государственный электротехнический
университет «ЛЭТИ» им. В.И. Ульянова (Ленина)**

Архитектура информационных систем

Санкт-Петербург
2026

Содержание

Глоссарий	5
1. Архитектурные основы	7
1.1. Лекция 1: Введение. Принципы построения приложения	7
1.1.1. Компоненты	7
1.1.1.1. Компонент как класс	7
1.1.2. Слои и зависимости	8
1.1.3. Принципы SOLID	9
1.1.4. Внедрение зависимостей	10
1.2. Лекция 2: Архитектурные стили и оценка продуктивности слоёв	12
1.2.1. Чистая архитектура (Clean Architecture)	12
1.2.2. Продуктивность слоя	12
1.2.3. Альтернативные архитектурные паттерны	14
1.2.3.1. Архитектура на основе фреймворка (Rails, Django)	14
1.2.3.2. Модель акторов (Erlang/Elixir/Akka)	14
1.2.3.3. Проектирование, ориентированное на данные (Data-oriented design)	14
1.3. Лекция 3: Протоколы взаимодействия: HTTP, REST, gRPC, GraphQL, WebSocket	15
1.3.1. Зачем нужны сетевые протоколы взаимодействия	15
1.3.1.1. От физического канала к HTTP	15
1.3.2. Прикладные протоколы взаимодействия	16
1.3.3. REST	16
1.3.4. gRPC	16
1.3.5. GraphQL	16
1.3.6. WebSocket	17
1.4. Лекция 4: Синхронные и асинхронные модели взаимодействия сервисов	18
1.4.1. Цена синхронного взаимодействия и альтернатива	18
1.4.1.1. Повторные попытки (Retries)	18
1.4.1.2. Идемпотентность (Idempotency)	18
1.4.1.3. Автоматическое размыкание цепи (Circuit Breaker)	19
1.4.1.4. Передача дедлайна (Deadline Propagation)	19
1.4.2. Зачем асинхронность	19
1.4.3. Брокеры сообщений и очереди	19
1.4.4. Обмен файлами как асинхронный контракт	19
1.4.5. Надёжность и согласованность	19
1.4.6. Постскриптум: критерий архитектурного выбора	19
2. Фундаментальные структуры хранилищ данных	21
2.1. Лекция 5: Классификация нагрузки	21
2.1.1. Типы операций	21
2.1.2. Метрики нагрузки	21
2.1.3. Характеристики нагрузки	22
2.1.4. Связь метрик: от DAU к QPS	22
2.1.5. Иерархия памяти	22
2.1.6. Оценка производительности: Little's Law	23
2.1.7. Фундаментальные компромиссы	23
2.2. Лекция 6: LSM Дерево	24
2.2.1. Общая идея	24
2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)	24
2.2.3. Memtable	24
2.2.4. Bloom Filter	25
2.2.5. Полная структура LSM-дерева	25
2.2.6. Практическое применение	26
2.3. Лекция 7: B-дерево: индексирование на диске	27

2.3.1. В-фактор и структура узла	27
2.3.2. Практическое применение	28
2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища	30
2.4.1. Транзакции	30
2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)	30
2.4.3. Транзакционные и аналитические хранилища	30
2.4.4. Пример 1: маркетплейс	31
2.4.5. Пример 2: строковое vs колоночное хранение	31
2.4.6. Сжатие колонок	32
2.5. Лекция 9: Хранилища в оперативной памяти	33
2.5.1. Аналитика: DuckDB и векторизация	33
2.5.2. Кэширование: Redis	33
3. Методы поиска похожих/близких элементов	35
3.1. Лекция 10: Специализированные индексы для поиска	35
3.1.1. Inverted Index (перевёрнутые индексы)	35
3.1.2. Trie (префиксные деревья)	36
3.1.3. Bitmap indexes	37
3.2. Лекция 11: Векторный поиск и embedding	38
3.2.1. Основы векторных представлений	38
3.2.2. Нормализация векторов и выбор метрики	39
3.2.3. Применение в проектировании систем	41
3.3. Лекция 12: Приближённый поиск ближайших соседей (ANN)	43
3.3.1. HNSW (Hierarchical Navigable Small World)	43
3.3.2. LSH (Locality-Sensitive Hashing)	47
3.3.3. Векторные БД в продакшене	50
3.4. Лекция 13: Геопространственные структуры	51
3.4.1. R-Tree	51
3.4.2. QuadTree	54
3.4.3. KD-Tree	56
4. Операционные компоненты	60
4.1. Лекция 14: Прокси и Rate Limiting	60
4.1.1. Forward Proxy vs Reverse Proxy	60
4.1.2. Алгоритмы распределения нагрузки	61
4.1.3. Ограничение частоты запросов (Rate Limiting)	63
4.2. Лекция 15: Безопасность и управление доступом	66
4.2.1. Аутентификация vs Авторизация	66
4.2.2. RBAC (Role-Based Access Control)	66
4.2.3. ABAC (Attribute-Based Access Control)	67
4.2.4. JWT и OAuth 2.0	69
4.2.5. Single Sign-On (SSO)	70
4.3. Лекция 16: Observability: логирование, мониторинг и планирование задач	72
4.3.1. Три столпа Observability	72
4.3.2. Структурированное логирование	72
4.3.3. Prometheus: сбор метрик	73
4.3.4. Grafana: визуализация и дашборды	75
4.3.5. Distributed Tracing (Jaeger, OpenTelemetry)	75
4.3.6. Планировщики задач	76
5. Практические задания	79
5.1.1. Варианты	79
5.1.2. Практика 1: Проектирование API и взаимодействия (Лекции 1-4: компоненты, слои, SOLID, протоколы)	80
5.1.3. Предисловие	80
5.1.4. Задание	80

5.1.5. Практика 2: Реализация спроектированного приложения (Лекции 1-4: реализация, тесты, отчёт)	84
5.1.6. Предисловие	84
5.1.7. Задание	84
5.1.8. Практика 3: OLTP-система и расчёт нагрузки (Лекции 5-9)	85
5.1.9. Введение	85
5.1.10. Задачи практики	85
5.1.11. Пример выполнения	86
5.1.12. Задача 1. Модель данных	86
5.1.13. Задача 2. Описание endpoint'a	87
5.1.14. Задача 3. Baseline производительности	87
5.1.15. Задача 4. Расчёт max RPS	87
5.1.16. Задача 5. Расчёт DAU	88
5.1.17. Проверка на здравый смысл	88
5.1.18. Справочник	88
5.1.19. Эталонная нагрузка: pgbench TPC-B	88
5.1.20. Инструменты для других СУБД	89
5.1.21. Формула CPU-time	89
5.1.22. Формула DAU	89
5.1.23. Практика 4: Специализированные индексы и поиск (Лекции 10-13: Inverted Index, векторный поиск, геопоиск)	90
5.1.24. Предисловие	90
5.1.25. Задание	90
5.1.26. Пример	90
5.1.27. Практика 5: Операционные компоненты + OLAP (Лекции 14-16: балансировка, авторизация, мониторинг, ETL)	94
5.1.28. Предисловие	94
5.1.29. Задание	94
5.1.30. Пример	95

Глоссарий

Ad-hoc анализ — Разовые, незапланированные аналитические запросы для исследования данных или проверки гипотез. В отличие от регулярных отчётов, выполняются по требованию без предварительной настройки ETL-пайплайнов. Требуется интерактивной производительности и гибкого SQL.

API — Application Programming Interface — программная абстракция, определяющая методы и протоколы взаимодействия между программными компонентами. Обеспечивает унифицированный доступ к функциональности системы посредством стандартизированных вызовов, скрывая детали внутренней реализации.

BI — Business Intelligence — технологии и практики сбора, интеграции, анализа и представления бизнес-информации. Включает инструменты для создания отчётов, дашбордов, визуализации данных и поддержки принятия управленческих решений на основе данных.

CDN — Content Delivery Network — географически распределённая сеть серверов для кэширования и доставки статического контента пользователям с минимальной задержкой.

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

CPU-time — Среднее процессорное время на транзакцию; оценивается через число ядер и TPS (мс на транзакцию).

CRM — Customer Relationship Management — система управления взаимоотношениями с клиентами. Программное обеспечение для автоматизации стратегий взаимодействия с клиентами, включая продажи, маркетинг, техподдержку и аналитику клиентских данных.

DAU — Daily Active Users — количество уникальных пользователей, взаимодействовавших с системой за сутки.

СУБД — Система управления базами данных — комплекс программных средств для создания, модификации и управления данными в базе данных. Обеспечивает структурированное хранение, эффективный доступ, целостность данных и управление конкурентным доступом.

DML — Data Manipulation Language — операции над данными в БД: SELECT/INSERT/UPDATE/DELETE.

downstream — В контексте конкретного вызова — вызываемая сторона (зависимость), которая обрабатывает запрос, полученный от upstream-сервиса.

Эмбединг — Векторное представление объекта (текста, изображения, пользователя, товара), полученное моделью машинного обучения для измерения семантической близости через расстояния в векторном пространстве.

Индекс — Вспомогательная структура данных в системах управления базами данных, предназначенная для оптимизации производительности операций поиска и извлечения информации. Организует упорядоченное представление данных для существенного сокращения времени доступа к записям.

Ввод-вывод — Input/Output — совокупность механизмов и протоколов обмена данными между вычислительной системой и внешними устройствами или другими системами. Включает операции получения данных из источников и передачи результатов обработки, представляя критически важный компонент архитектуры компьютерных систем.

IOPS — I/O Operations Per Second — количество операций ввода-вывода (чтения/записи) в секунду на уровне диска.

IoT — Internet of Things — сетевое взаимодействие физических устройств с датчиками и контроллерами. Характеризуется высокой частотой передачи небольших порций данных (телеметрия, метрики).

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

LLM — Large Language Model — большая языковая модель на основе нейронных сетей, обученная на огромных объёмах текстовых данных. Способна генерировать код, текст, отвечать на вопросы и выполнять задачи программирования на основе естественно-языковых промптов.

LOC — Lines of Code — метрика объёма программного кода, измеряемая количеством строк. Используется для оценки размера кодовой базы, сложности проекта и трудозатрат на разработку. Не включает пустые

строки и комментарии при подсчёте физических строк (PLOC), либо учитывает только исполняемые инструкции при подсчёте логических строк (LLOC).

MAU — Monthly Active Users — количество уникальных пользователей за месяц.

Метаданные — Структурированная информация, описывающая свойства других данных: время создания, размер, тип, владелец, права доступа. Используется для индексирования и маршрутизации без чтения основного содержимого.

Морселизация — Техника параллельной обработки данных путём разбиения на небольшие фрагменты (morsels) фиксированного размера (обычно ~1024 строки). Каждый morsel обрабатывается независимо векторизованным движком, что обеспечивает эффективное использование CPU-кэшей и параллелизацию на уровне потоков.

OLAP — Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

OLTP — Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

QPS — Queries Per Second — количество запросов к базе данных в секунду. Один HTTP-запрос может породить несколько запросов к БД.

RPS — Requests Per Second — количество пользовательских запросов (например HTTP) в секунду.

Сессия — Логический контекст взаимодействия клиента с системой между множественными запросами. Хранит состояние пользователя: аутентификацию, корзину, настройки. Реализуется через cookies + серверное хранилище (Redis) или JWT-токены.

SLA — Service Level Agreement — соглашение об уровне обслуживания с количественными метриками: uptime (99.9% = 8.76 ч простоя/год), задержка ($p_{99} < 200$ мс).

Support — Служба поддержки и обработка обращений пользователей: тикеты, инциденты, запросы на возврат и консультации.

TPS — Transactions Per Second — количество транзакций БД в секунду.

Транзакция — Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

upstream — В контексте конкретного вызова — вызывающая сторона (клиент), которая инициирует запрос к другому сервису и ожидает результат.

1. Архитектурные основы

Курс начинается с фундаментальных принципов построения приложений. Прежде чем говорить о конкретных технологиях (базах данных, очередях, кешах), нужно понять **как вообще организовать код**, чтобы система была понятной, расширяемой и тестируемой.

Модуль отвечает на вопросы:

- **Что такое компонент?** — базовая единица организации кода (класс, модуль, сервис)
- **Как разделять ответственность?** — архитектурные паттерны (слоистая, гексагональная, event-driven архитектуры)
- **Как компоненты общаются?** — протоколы взаимодействия (REST, gRPC, GraphQL, WebSocket)

1.1. Лекция 1: Введение. Принципы построения приложения

Первая лекция закладывает фундамент: **что такое компонент и как компоненты зависят друг от друга?**

Компонент — это базовая единица вычисления или хранения данных с чётким контрактом (интерфейсом) и набором зависимостей. Это может быть класс, модуль, библиотека или микросервис. Главное — у него есть чёткая граница: что он принимает на вход, что возвращает, от чего зависит.

Управление зависимостями — ключевая задача архитектуры. Если компонент A зависит от компонента B, то изменения в B могут сломать A. Когда в проекте сотни компонентов, неконтролируемые зависимости превращают код в «клубок спагетти», где невозможно понять, что от чего зависит.

Лекция вводит понятия **слоя** (множество компонентов с общей ответственностью) и **правил межслойных зависимостей** (например, «слой бизнес-логики не должен зависеть от слоя UI»). Эти правила помогают избежать циклических зависимостей и упрощают тестирование.

Компонент — Единица вычисления либо хранения данных, имеющая явно определённый контракт взаимодействия (интерфейс) и набор зависимостей.

1.1.1. Компоненты

Рассмотрим несколько примеров:

1.1.1.1. Компонент как класс

```
#include <iostream>

class NumberPrinter {
public:
    void print(int x) {
        std::cout << x << std::endl;
    }

    void printRange(int from, int to) {
        for (int i = from; i <= to; ++i) {
            std::cout << i << std::endl;
        }
    }
};
```

Единица вычисления: форматирование и вывод чисел

Интерфейс: сигнатуры публичных методов

Зависимость: консоль ввода-вывода

Форма реализации: класс (ООП-модель)

- Компонент как функция (Elixir)

```
def send_message(host, port, message) do
  {:ok, socket} = :gen_tcp.connect(host, port, [:binary, active: false])
  :ok = :gen_tcp.send(socket, message)
```

```

:gen_tcp.close(socket)
end

```

Единица вычисления: передача сообщения

Интерфейс: сигнатура функции

Зависимость: сеть (TCP stack)

Форма реализации: функция (функциональная модель)

- Компонент как функция (Go)

```

func LoadUserNames(db *sql.DB) ([]string, error) {
    rows, err := db.Query("SELECT name FROM users")
    if err != nil {
        return nil, err
    }
    defer rows.Close()

    var result []string
    for rows.Next() {
        var name string
        if err := rows.Scan(&name); err != nil {
            return nil, err
        }
        result = append(result, name)
    }
    return result, nil
}

```

Единица вычисления: извлечение данных

Интерфейс: сигнатура функции

Зависимость: БД (SQL, соединение)

Форма реализации: функция (процедурная / Go-модель)

1.1.2. Слои и зависимости

Слой — Множество компонентов, объединённых общей ответственностью и уровнем абстракции, и участвующих в системе согласно установленным правилам межслойных зависимостей. Говорят что слой А зависит от слоя Б если хотя бы один компонент слоя А зависит от компонента слоя Б.

Рассмотрим пример слоя персистентности, который сохраняет различные модели в CSV:

```

// domain/models.go - слой моделей данных
package domain

type User struct{ ID int; Name, Email string }
type Item struct{ ID int; Title string; Price float64 }
type Storage struct{ ID int; Location string; Capacity int }

// persistence/csv_persistence.go - слой персистентности
package persistence

import (
    "fmt"
    "os"
    "strings"

    "example/domain" // Явная зависимость от слоя моделей
)

func SaveUsersToCSV(users []domain.User, filename string) error {

```



```

lines := make([]string, 0, len(users))
for _, u := range users {
    lines = append(lines, fmt.Sprintf("%d,%s,%s", u.ID, u.Name, u.Email))
}
return write(filename, lines)
}

func SaveItemsToCSV(items []domain.Item, filename string) error {
    lines := make([]string, 0, len(items))
    for _, i := range items {
        lines = append(lines, fmt.Sprintf("%d,%s,%v", i.ID, i.Title, i.Price))
    }
    return write(filename, lines)
}

func SaveStoragesToCSV(storages []domain.Storage, filename string) error {
    lines := make([]string, 0, len(storages))
    for _, s := range storages {
        lines = append(lines, fmt.Sprintf("%d,%s,%d", s.ID, s.Location, s.Capacity))
    }
    return write(filename, lines)
}

func write(filename string, lines []string) error {
    return os.WriteFile(filename, []byte(strings.Join(lines, "\n")), 0644)
}

```

Анализ слоёв:

Слой	Компонент	Контракт (интерфейс)	Зависимости
domain	User	struct{ ID int; Name, Email string }	—
	Item	struct{ ID int; Title string; Price float64 }	—
	Storage	struct{ ID int; Location string; Capacity int }	—
persistence	SaveUsersToCSV	([]domain.User, string) error	domain.User, ФС
	SaveItemsToCSV	([]domain.Item, string) error	domain.Item, ФС
	SaveStoragesToCSV	([]domain.Storage, string) error	domain.Storage, ФС
	write	(string, []string) error	ФС

Заметим:

- **Слой моделей** (domain): три компонента-структуры без зависимостей
- **Слой персистентности** (persistence): четыре компонента-функции, три из которых явно зависят от типов слоя моделей
- Слой персистентности **зависит** от слоя моделей, но не наоборот

1.1.3. Принципы SOLID

SOLID — набор принципов проектирования, уменьшающих связанность кода и повышающих локальность изменений.

- **Принцип единственной ответственности (SRP):** у компонента должна быть одна причина для изменения.
- **Принцип открытости/закрытости (ОСР):** поведение расширяется через новые реализации, а не через модификацию стабильного кода.

- **Принцип подстановки Лисков (LSP):** подтип должен корректно заменять базовый тип без изменения ожидаемого поведения.
- **Принцип разделения интерфейсов (ISP):** лучше несколько узких интерфейсов, чем один «толстый» универсальный.
- **Принцип инверсии зависимостей (DIP):** бизнес-логика зависит от абстракций, а не от конкретных деталей инфраструктуры.

В контексте SOLID интерфейс удобно понимать как **контракт взаимодействия**: он фиксирует, **что** компонент обязан предоставить, но не навязывает, **как** это реализовано. Форма интерфейса зависит от языка:

- **Go:** отдельный тип `interface` с набором методов;
- **Java/C#** — явная декларация `interface` и классы-реализации;
- **Python:** обычно протокол поведения через абстрактные базовые классы (abc) или структурные протоколы (Protocol).

Практический эффект SOLID: изменение формата хранения данных или транспортного протокола затрагивает ограниченное число компонентов, а не всю систему.

1.1.4. Внедрение зависимостей

После того как в системе появляется много компонентов, ключевым становится вопрос управления зависимостями между ними. Внедрение зависимостей (Dependency Injection, DI) — это способ передачи компоненту его зависимостей извне, а не создания их внутри компонента.

DI-контейнер — инфраструктурный механизм, который:

- регистрирует реализации компонентов;
- строит и разрешает граф зависимостей;
- создаёт экземпляры в правильном порядке (с учётом времени жизни объектов).

По способу разрешения зависимостей обычно различают два подхода:

- **На этапе компиляции (compile-time DI):** граф зависимостей формируется заранее, ошибки связей обнаруживаются до запуска приложения.
- **Во время выполнения (run-time DI):** контейнер разрешает зависимости при запуске, что даёт больше гибкости конфигурации, но переносит часть ошибок на момент старта/работы системы.

Без DI компонент жёстко связан с конкретными реализациями (например, конкретной СУБД или конкретным HTTP-клиентом), что усложняет тестирование и замену инфраструктурных модулей. С DI компонент работает через интерфейсы, а конкретные реализации подключаются в точке композиции приложения (composition root).

Простейший пример конструкторной инъекции:

```
type UserRepo interface {
    FindByID(id int) (User, error)
}

type UserService struct {
    repo UserRepo
}

func NewUserService(repo UserRepo) *UserService {
    return &UserService{repo: repo}
}

func BuildApp() *UserService {
    db := NewPostgresDB()
    repo := NewPostgresUserRepo(db)
    return NewUserService(repo) // сборка графа зависимостей
}
```

На небольших проектах такой граф зависимостей часто собирают вручную. По мере роста системы это становится трудоёмко: увеличивается число компонентов, ветвлений конфигурации и рисков ошибиться в порядке инициализации. Поэтому на практике используют готовые библиотеки:

- **Go:** uber-go/dig, google/wire;
- **Java/Kotlin:** Spring Container, Guice, Dagger;
- **.NET:** встроенный контейнер `Microsoft.Extensions.DependencyInjection`, Autofac;
- **Python:** `dependency-injector`, `Dishka`;
- **Node.js/TypeScript:** `InversifyJS`, `NestJS DI`.

Отдельный класс нюансов связан с временем жизни объектов. Обычно различают:

- **Singleton** (один экземпляр на приложение),
- **Scoped** (один экземпляр на запрос/операцию),
- **Transient** (новый экземпляр при каждом запросе зависимости).

Точные правила объявления и разрешения времени жизни зависят от конкретной библиотеки контейнера.

Практические правила:

- создавать зависимости на границе приложения, а не внутри бизнес-методов;
- передавать зависимости через конструктор или параметры инициализации;
- не превращать DI-контейнер в «глобальный сервис-локатор».

Результат: проще модульные тесты, дешевле замена инфраструктуры, прозрачнее граф зависимостей.

1.2. Лекция 2: Архитектурные стили и оценка продуктивности слоёв

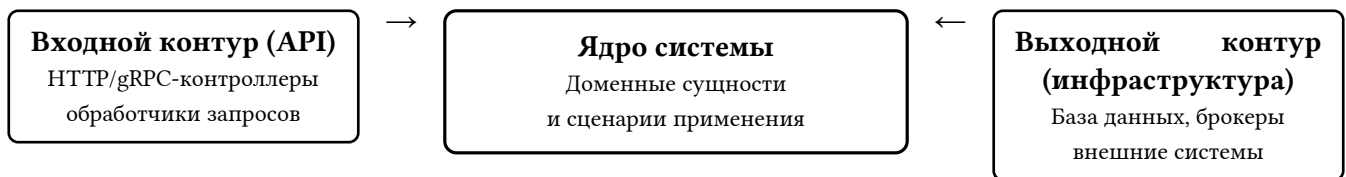
В предыдущем разделе были зафиксированы локальные правила проектирования компонентов и связей между ними. Теперь рассмотрим архитектуру на уровне всей системы: направление зависимостей, оценку продуктивности слоёв и альтернативные архитектурные подходы.

1.2.1. Чистая архитектура (Clean Architecture)

Чистая архитектура задаёт правило направленности зависимостей: внешние слои зависят от внутренних, но не наоборот.

Типичная структура:

- **Entities / Domain:** бизнес-сущности и инварианты;
- **Use Cases / Application:** сценарии применения и оркестрация бизнес-операций;
- **Interface Adapters:** контроллеры, presenters, репозитории-адаптеры;
- **Frameworks & Drivers:** веб-фреймворк, база данных, брокеры сообщений, внешние API.



Зависимости направлены к центру: внешний код зависит от бизнес-ядра.

Схема 1: Домен в центре, внешние контуры по сторонам

Ключевая идея: бизнес-правила должны оставаться работоспособными при смене базы данных, очереди или веб-фреймворка. Это не исключает использование фреймворков; это требует, чтобы фреймворк был деталью, а не центром архитектуры.

Связка с предыдущим материалом:

- SOLID снижает связанность на уровне отдельных компонентов;
- DI управляет связями между компонентами;
- Clean Architecture фиксирует направление зависимостей на уровне всей системы.

Вместе они формируют практический каркас, позволяющий масштабировать кодовую базу без неконтролируемого роста архитектурного долга.

1.2.2. Продуктивность слоя

После фиксации принципов проектирования можно количественно оценить качество декомпозиции слоёв.

Прокси-компонент — компонент, не реализующий собственной вычислительной ответственности, а лишь делегирующий вызовы компонентам других слоёв.

Продуктивность слоя — количественная мера архитектурной самостоятельности слоя, определяемая долей компонентов, реализующих собственную вычислительную ответственность.

Формальное задание меры

Пусть:

- N — общее количество компонентов в слое
- P — количество прокси-компонентов в слое

Тогда коэффициент проксирования слоя определяется как:

$$K_{\text{proxy}} = \frac{P}{N}$$

а продуктивность слоя — как дополняющая величина:

$$K_{\text{prod}} = 1 - K_{\text{proxy}} = 1 - \frac{P}{N}$$

В стандартной литературе (например, Clean Architecture) слой считается продуктивным, если $K_{\text{prod}} \geq 0.8$, то есть не менее 80% компонентов реализуют собственную вычислительную ответственность.

Пример непродуктивного проксирующего слоя:

Рассмотрим излишний слой сервисов, большинство компонентов которого только делегируют вызовы к слою персистентности:

```
// service/user_service.go - непродуктивный слой сервисов
package service

import (
    "example/domain"
    "example/persistence"
    "strings"
)

func GetUser(id int) (*domain.User, error) {
    return persistence.LoadUser(id)
}

func SaveUser(user *domain.User) error {
    return persistence.SaveUser(user)
}

func DeleteUser(id int) error {
    return persistence.DeleteUser(id)
}

func ListUsers() ([]*domain.User, error) {
    return persistence.ListUsers()
}

func SearchUsersByName(query string) ([]*domain.User, error) {
    users, err := persistence.ListUsers()
    if err != nil {
        return nil, err
    }

    var result []*domain.User
    for _, u := range users {
        if strings.Contains(strings.ToLower(u.Name), strings.ToLower(query)) {
            result = append(result, u)
        }
    }
    return result, nil
}
```

Анализ слоя сервисов:

Компонент	Контракт	Собственная ответственность
GetUser	(int) (*User, error)	Нет — прямая делегация
SaveUser	(*User) error	Нет — прямая делегация
DeleteUser	(int) error	Нет — прямая делегация
ListUsers	() ([]*User, error)	Нет — прямая делегация
SearchUsersByName	(string) ([]*User, error)	Да — фильтрация и поиск

Продуктивность слоя: $K_{\text{prod}} = 1 - \frac{4}{5} = 0.2$ (20% — непродуктивный слой)

Такой слой не добавляет архитектурной ценности и лишь усложняет кодовую базу, увеличивая количество уровней косвенности без предоставления дополнительной функциональности.

1.2.3. Альтернативные архитектурные паттерны

После изучения классических подходов (слоистая архитектура, Clean Architecture, SOLID) важно понять, что **нет единственно правильного способа** организации кода. Разные задачи требуют разных подходов.

Раздел показывает альтернативные парадигмы:

- **Framework-based архитектура** (Rails, Django) — фреймворк диктует структуру, вы заполняете пробелы. Быстро для типовых задач, но ограничивает гибкость.
- **Actor Model** (Erlang, Akka) — вместо объектов и функций — изолированные процессы, обменивающиеся сообщениями. Идеально для высоконагруженных телекоммуникационных систем.
- **Data-Oriented Design** — оптимизация под архитектуру процессора (кеш-линии, SIMD). Критично для игровых движков и симуляций, где производительность — главное.

Выбор архитектуры зависит от контекста: стартапу нужна скорость разработки (Rails), телеком-системе — отказоустойчивость (Erlang), игровому движку — производительность (DOD). Нет «лучшей» архитектуры — есть подходящая для конкретной задачи.

1.2.3.1. Архитектура на основе фреймворка (Rails, Django)

Фреймворк предполагает стандартную структуру папок, именование файлов и шаблоны проектирования (зачастую, MVC), что уменьшает количество принимаемых решений и ускоряет разработку.

- “Мнения” фреймворка: Rails/Django имеют предустановленные “мнения” по работе с БД (ORM), маршрутизацией, шаблонизацией и аутентификацией.

1.2.3.2. Модель акторов (Erlang/Elixir/Akka)

- Революционная технология, созданная в Ericsson еще в 1986 году для телекоммуникационных систем с требованием “пяти девяток” доступности.
- Она на десятилетия опередила современные подходы: предоставляет
 - встроенную распределенность (кластеры из коробки),
 - философия “Let it Crash!” с автоматическим самовосстановлением
 - горячее обновление кода без остановки системы
 - невероятную параллельность (миллионы изолированных процессов)
- WhatsApp: Всего 50 инженеров обрабатывали 2 миллиарда сообщений в день на одном сервере!
- Discord

1.2.3.3. Проектирование, ориентированное на данные (Data-oriented design)

Подход, фокусирующийся на эффективной организации данных в памяти для оптимального использования кэша процессора. Используется где производительность упирается в скорость доступа к памяти.

- игровые движки
- обработки физики
- AI
- симуляции

1.3. Лекция 3: Протоколы взаимодействия: HTTP, REST, gRPC, GraphQL, Web-Socket

1.3.1. Зачем нужны сетевые протоколы взаимодействия

В предыдущих лекциях мы рассмотрели декомпозицию приложения на компоненты и выбор архитектурного стиля. Следующий шаг — определить, как передавать прикладную ценность конечному пользователю.

В простейшем случае взаимодействие происходит в пределах одного компьютера: ввод -> приложение -> вывод.

Примеры локального взаимодействия (без сети):

- **CLI-утилита** получает аргументы и stdin, выполняет обработку и выводит результат в stdout.
- **Настольное приложение** принимает события клавиатуры и мыши через API ОС и отображает интерфейс через графический стек.
- **Офлайн-игра** принимает ввод с контроллера и выводит кадры через API видеокарты.

Примеры распределённого взаимодействия (через сеть):

- **Мобильное банковское приложение** отображает баланс после запроса к серверу компании.
- **Интернет-магазин** подтверждает заказ через цепочку сервисов оплаты, склада и доставки.
- **IoT-датчик** отправляет телеметрию в облачную инфраструктуру для хранения и аналитики.

Во всех распределённых сценариях между клиентом и приложением присутствует сеть. Следовательно:

- данные должны быть упакованы в формат передачи;
- сообщение должно быть доставлено по сетевому каналу;
- принимающая сторона должна корректно распаковать и интерпретировать сообщение.

Этот процесс задаётся сетевыми протоколами. В упрощённом виде цепочка выглядит так: «соединение двух компьютеров -> передача битов -> доставка пакета целевому процессу -> согласование формата запроса и ответа».

1.3.1.1. От физического канала к HTTP

На нижних уровнях сеть решает задачу **доставки**: как сигнал проходит по среде, как кадры и пакеты достигают нужного узла, как поток данных попадает в конкретный процесс. На верхних уровнях решается задача **смысла**: в каком формате передаются данные, как описывается операция, как интерпретировать успешный и ошибочный результат.

Для архитектора здесь важен практический вывод: протокол нельзя выбирать в отрыве от сети.

- нестабильный канал напрямую влияет на таймауты и стратегию повторов;
- свойства транспорта влияют на модель обмена;
- выбор прикладного протокола влияет на контракт API и его эволюцию.

Для практики достаточно обзорного понимания уровней OSI:

- **L1 Физический** — среда передачи сигнала: кабель, Wi-Fi, оптика, радио.
- **L2 Канальный** — кадры в локальной сети, MAC-адресация, Ethernet.
- **L3 Сетевой** — IP-адресация и маршрутизация между сетями.
- **L4 Транспортный** — TCP/UDP, порты, доставка потока данных прикладному процессу.
- **L5-L6 Сеансовый и представления** — управление сессией, TLS-шифрование, кодировки и сериализация.
- **L7 Прикладной** — протоколы уровня приложения, например HTTP, где фиксируются правила API и семантика сообщений.

Важно различать транспортный протокол и прикладной контракт. HTTP задаёт правила обмена (метод, заголовки, статус, тело сообщения), но сам по себе не определяет бизнес-семантику полезной нагрузки (payload). Семантика и формат данных определяются прикладным протоколом и договорённостями API.

Поэтому поверх одного и того же HTTP можно реализовать разные модели взаимодействия и разные форматы полезной нагрузки:

- REST использует HTTP напрямую и обычно передаёт ресурсы в JSON (иногда XML);
- GraphQL использует HTTP как транспорт для запросов к схеме (payload содержит текст запроса и переменные);
- WebSocket устанавливается через механизм обновления протокола HTTP (Upgrade) и затем работает как постоянный двусторонний канал сообщений;
- gRPC работает поверх HTTP/2 и передаёт бинарные сообщения Protocol Buffers;
- загрузка файлов обычно передаётся как multipart/form-data или как бинарный поток, в зависимости от контракта API.

Когда компоненты расположены в разных процессах или на разных машинах, их взаимодействие нельзя трактовать как обычный вызов функции. Требуется явно определить **контракт сетевого взаимодействия**: формат сообщений, правила обработки ошибок, таймауты, ретраи и семантику ответов.

1.3.2. Прикладные протоколы взаимодействия

В рамках лекции рассматриваются четыре прикладных протокола взаимодействия: REST, gRPC, GraphQL и WebSocket. Они решают разные классы задач: классический обмен «запрос-ответ», высокопроизводительные межсервисные удалённые вызовы процедур (RPC), гибкие выборки данных и двустороннюю коммуникацию в реальном времени. Выбор протокола определяется требованиями к производительности, удобству интеграции, гибкости модели данных и характеру пользовательского сценария.

1.3.3. REST

- **Модель взаимодействия:** запрос-ответ на основе HTTP-методов (GET, POST, PUT, DELETE) и кодов статуса.
- **Формат и транспорт:** обычно HTTP + JSON; ресурсы адресуются URI (например, /api/users/123).
- **Преимущества:** простота, широкая совместимость, удобная диагностика и развитая экосистема (Swagger/OpenAPI).
- **Ограничения:** избыточный объём данных и необходимость множества конечных точек API для разных сценариев представления.
- **Нюансы внедрения:** важно строго соблюдать семантику HTTP-методов, корректно использовать коды ответов, а также проектировать версионирование API (например, /v1) и политику обратной совместимости.

1.3.4. gRPC

- **Модель взаимодействия:** удалённые вызовы процедур (RPC), включая одиночные и потоковые вызовы.
- **Формат и транспорт:** Protocol Buffers как схема данных, HTTP/2 как транспорт.
- **Преимущества:** высокая производительность, компактная бинарная сериализация, строгая типизация контракта.
- **Ограничения:** более сложная отладка, обязательная кодогенерация и ограничения прямого использования в браузере (обычно через gRPC-Web).
- **Нюансы внедрения:** необходима дисциплина эволюции .proto-контрактов (зарезервированные поля и совместимость схем), настройка дедлайнов и метаданных, а также понимание инфраструктурных ограничений (балансировщики, прокси, наблюдаемость HTTP/2-трафика).

1.3.5. GraphQL

- **Модель взаимодействия:** клиент формулирует запрос как дерево полей и получает только требуемые данные.
- **Формат и транспорт:** чаще всего HTTP с единой точкой входа (/graphql); ответы обычно в JSON.
- **Преимущества:** уменьшение избыточной и недостаточной выборки данных (over-fetching/under-fetching) и удобная поддержка разных экранов и клиентских сценариев.
- **Ограничения:** более сложная эксплуатация (контроль сложности запросов, кэширование, авторизация на уровне полей).
- **Нюансы внедрения:** критично ограничивать глубину и вычислительную сложность запросов, предотвращать проблему N+1 через пакетирование запросов (batching/DataLoader) и явно проектировать границы ответственности слоя резолверов.

1.3.6. WebSocket

- **Модель взаимодействия:** постоянное двустороннее соединение между клиентом и сервером.
- **Формат и транспорт:** устанавливается через HTTP Upgrade, далее обмен сообщениями идёт по WebSocket-каналу.
- **Преимущества:** низкая задержка и серверные push-обновления без постоянного опроса.
- **Ограничения:** более сложное управление соединениями (повторное подключение, контроль активности соединения, регулирование входящего потока) и хранение состояния соединений на сервере.
- **Нюансы внедрения:** требуются протокол прикладных сообщений (типы событий, версии), механизмы восстановления соединения, защита от «медленных» клиентов и план масштабирования соединений с состоянием (маршрутизация с привязкой к узлу или внешний канал публикации/подписки).

1.4. Лекция 4: Синхронные и асинхронные модели взаимодействия сервисов

В синхронной модели взаимодействия клиент отправляет запрос и ожидает ответ в рамках того же пользовательского действия. Для REST, gRPC и GraphQL это базовый режим работы; WebSocket обычно используется для постоянного двустороннего обмена, но также требует управления таймингами и отказами. Синхронная модель удобна, когда нужен мгновенный результат, но именно здесь появляются основные эксплуатационные риски распределённых систем.

1.4.1. Цена синхронного взаимодействия и альтернатива

Когда сервис делает **синхронный** вызов по схеме «запрос-ответ» и ждёт результат, он напрямую зависит от доступности и скорости downstream-сервиса. Практически это означает три базовых класса отказов:

- **Тайм-аут:** зависимость отвечает слишком долго, и вызывающая сторона прерывает ожидание.
- **Неопределённый результат операции:** вызывающая сторона не получила ответ и не знает, была ли операция выполнена на стороне зависимости.
- **Каскадная деградация:** медленный или недоступный downstream удерживает ресурсы upstream-сервисов (пулы соединений, воркеры, потоки), снижая общую пропускную способность системы.

В синхронной модели именно вызывающая сторона (клиент запроса) обычно задаёт timeout и стратегию повторов (retry), потому что только она знает допустимое время ожидания в рамках пользовательского сценария и может принять решение: ждать дальше, повторить вызов или вернуть ошибку наверх. Если эти параметры не заданы, зависший downstream начинает удерживать поток/воркер, и при росте нагрузки это быстро приводит к исчерпанию ресурсов.

Упрощённо это выглядит так:

```
// Клиент управляет временем и повторами вызова.
ctx, cancel := context.WithTimeout(parentCtx, 300*time.Millisecond)
defer cancel()

for attempt := 1; attempt <= 3; attempt++ {
    resp, err := paymentsClient.Charge(ctx, req) // синхронный RPC/HTTP-вызов
    if err == nil {
        return resp, nil
    }
    if !isTemporary(err) {
        return nil, err // логическую ошибку не ретраим
    }
    sleep(backoffWithJitter(attempt))
}
return nil, ErrUpstreamUnavailable
```

Такая логика необходима, но сама по себе создаёт дополнительные риски, которые нужно проектировать заранее.

1.4.1.1. Повторные попытки (Retries)

Проблема. Повторы без ограничений и без случайного разброса задержки (jitter) во время инцидента создают «шторм повторов». **Следствие.** Зависимость получает ещё больше трафика в момент деградации, а время восстановления увеличивается. **Что делать.** Повторять только временные ошибки, ограничивать число попыток, применять экспоненциальную задержку (backoff) и jitter, а также общий бюджет времени на операцию.

1.4.1.2. Идемпотентность (Idempotency)

Проблема. Повтор неидемпотентного запроса может выполнить бизнес-действие повторно. **Следствие.** Появляются дубликаты операций (например, двойное списание или двойная отправка заказа). **Что делать.** Использовать ключ идемпотентности (idempotency key) для операций с побочным эффектом и возвращать результат первой успешной обработки при повторе.

1.4.1.3. Автоматическое размыкание цепи (Circuit Breaker)

Проблема. Даже при наличии таймаутов трафик продолжает идти в недоступную зависимость.

Следствие. Возникает каскадная деградация: upstream теряет ресурсы на заведомо безуспешные вызовы.

Что делать. При превышении порога ошибок размыкать цепь, временно отклонять вызовы или переключаться на резервный сценарий (fallback), затем выполнять ограниченные пробные запросы в промежуточном состоянии восстановления.

1.4.1.4. Передача дедлайна (Deadline Propagation)

Проблема. В цепочке сервисов внутренние таймауты часто не согласованы с исходным SLA запроса.

Следствие. Внутренний вызов может ждать дольше, чем разрешено клиентом, и результат становится бесполезным. **Что делать.** Передавать единый дедлайн по всей цепочке, рассчитывать таймауты от оставшегося бюджета и логировать остаток времени в трассировке.

Практически это означает, что timeout, retry, idempotency, circuit breaker и deadline propagation следует рассматривать как единый контур устойчивости, а не как независимые опции.

Альтернатива — **асинхронное взаимодействие**: сервис не блокируется в ожидании ответа, а публикует событие или задачу в брокер/очередь. Это снижает связность по времени и лучше переживает пики нагрузки, но добавляет модель согласования состояния со временем (eventual consistency), необходимость дедупликации и контроль отставания потребителей очереди.

Во многих практических сценариях (обработка видео, генерация отчётов, интеграции с внешними системами, массовые рассылки) мгновенный ответ не нужен, поэтому асинхронная модель оказывается устойчивее и дешевле в эксплуатации.

1.4.2. Зачем асинхронность

- Развязка сервисов: отправитель не ждёт, пока получатель завершит обработку
- Устойчивость к пикам нагрузки: очередь буферизует всплески
- Повышение надёжности: повторная доставка, управляемые повторы и очередь проблемных сообщений (DLQ)
- Улучшение UX: пользователь получает быстрый 202 Accepted, тяжёлая работа уходит в фон

1.4.3. Брокеры сообщений и очереди

- **Kafka** — журнал событий (event log), высокая пропускная способность, партиции, группы потребителей
- **RabbitMQ** — классические очереди задач, гибкая маршрутизация, подтверждения доставки
- **Pub/Sub** — одно событие читают несколько подписчиков (аналитика, уведомления, антифрод)
- **Task Queue** — одно задание исполняет один воркер (рендер PDF, отправка email)

1.4.4. Обмен файлами как асинхронный контракт

- Подходит для batch-интеграций между системами и организациями
- Примеры: S3/shared folder/SFTP, форматы CSV/Parquet/JSONL
- Важные практики: версионирование схемы, checksum, идемпотентная загрузка, дедупликация
- Риски: задержки, частичная загрузка, конфликт версий; нужны валидаторы и правила ретраев

1.4.5. Надёжность и согласованность

- Гарантии доставки: не более одного раза (at-most-once), не менее одного раза (at-least-once), практически однократно (effectively-once, через идемпотентность)
- Согласование состояния со временем (eventual consistency): состояние сходится не мгновенно, а через поток событий
- Подход Saga: длинные бизнес-процессы с компенсирующими действиями

1.4.6. Постскриптум: критерий архитектурного выбора

Итог первого модуля: большинство инструментов «работают» и действительно способны решить задачу. REST, gRPC, GraphQL, WebSocket, очереди и файловые интеграции — каждый подход может дать рабочий результат в правильном контексте.

Поэтому хороший архитектор не выбирает «любимый» инструмент заранее. Он выбирает инструмент под задачу, учитывая ограничения:

- требования к задержке и пропускной способности;
- характер пользовательского сценария (синхронный ответ или фоновая обработка);
- стоимость сопровождения и наблюдаемости;
- требования к надёжности, безопасности и эволюции контракта.

Рассмотренные в модуле принципы нужны именно для этого: принимать инженерные решения не по привычке, а по критериям конкретной системы.

2. Фундаментальные структуры хранилищ данных

Мы познакомились с архитектурными паттернами построения приложений и протоколами обмена данными между компонентами. Теперь возникает следующий вопрос: где и как хранить данные?

Задача выбора хранилища нетривиальна. На рынке сотни решений: реляционные базы, документные хранилища, key-value stores, колоночные базы, графовые базы, временные ряды. Каждое решение оптимизировано под определённый паттерн нагрузки, и неправильный выбор приводит к проблемам производительности, которые сложно исправить без миграции.

Чтобы осознанно выбирать между PostgreSQL и ClickHouse, между Redis и MongoDB, нужно понимать внутреннее устройство этих систем. В этом модуле мы разберём фундаментальные структуры данных, лежащие в основе современных хранилищ.

2.1. Лекция 5: Классификация нагрузки

Прежде чем выбирать базу данных или структуру хранения, нужно понять **какая нагрузка** будет на систему. База данных для интернет-магазина (миллионы мелких транзакций) и база данных для аналитики (сканирование миллиардов строк) требуют совершенно разных подходов.

Эта лекция вводит классификацию нагрузок:

- **OLTP** (Online Transaction Processing) — много коротких транзакций, чтение/запись отдельных строк (интернет-магазин, банковские переводы, бронирование билетов)
- **OLAP** (Online Analytical Processing) — редкие, но тяжелые запросы, сканирование миллионов строк для агрегации (отчёты, бизнес-аналитика, data science)

Для OLTP критично время отклика одной транзакции (миллисекунды), для OLAP — пропускная способность (миллионы строк в секунду). Отсюда разные структуры хранения: OLTP использует строковое хранение (B-tree), OLAP — колоночное (Parquet, ClickHouse).

Лекция показывает, как оценить нагрузку через метрики (RPS, размер данных, паттерны доступа) и на основе этого выбрать подходящее хранилище.

2.1.1. Типы операций

Зачастую взаимодействие с хранилищем сводится к ограниченному набору операций:

Тип	Операция	Пример
Чтение	Point lookup	SELECT * FROM users WHERE id = 123
	Multi-get	SELECT * FROM users WHERE id IN (1, 2, 3)
	Range scan	SELECT * FROM orders WHERE date BETWEEN '...' AND '...'
	Prefix scan	SELECT * FROM logs WHERE key LIKE 'user:123:%'
	Full scan	SELECT AVG(price) FROM products
Запись	Insert	вставка новой записи (ошибка при дубликате)
	Update	изменение существующей записи
	Upsert	INSERT ... ON CONFLICT DO UPDATE
	Delete	удаление записи
	Batch write	запись нескольких записей
Комб.	Read-modify-write	UPDATE accounts SET balance = balance + 100
	Increment	INCR views:page:123 (атомарный счётчик)

2.1.2. Метрики нагрузки

Метрика	Расшифровка	Описание
DAU	Daily Active Users	уникальные пользователи за сутки
MAU	Monthly Active Users	уникальные пользователи за месяц

RPS	Requests Per Second	пользовательских запросов (например HTTP) в секунду
QPS	Queries Per Second	запросов к БД в секунду
TPS	Transactions Per Second	транзакций БД в секунду
IOPS	I/O Operations Per Second	операций ввода-вывода в секунду

2.1.3. Характеристики нагрузки

Read/Write ratio:

- **90/10** — преобладание чтения: кэширование, CDN, статический контент
- **50/50** — сбалансированная: социальные сети, e-commerce
- **10/90** — преобладание записи: логирование, IoT, метрики

Размер записи:

- **< 1 KB** — Метаданные, Сессия, счётчики
- **1-100 KB** — JSON-документы, профили пользователей
- **100 KB - 1 MB** — статьи, посты с изображениями
- **> 1 MB** — медиафайлы, бинарные данные

Паттерн доступа:

- **random** — point lookup по ключу, случайные запросы
- **sequential** — range scan, time-series, логи

Требования к латентности:

- **p50 < 50 мс** — медиана, типичный пользовательский опыт
- **p99 < 200 мс** — 1% худших запросов, важно для SLA
- **p999 < 1 с** — 0.1% худших, критично для финтеха

Почему p99 важнее среднего: если p99 = 500 мс при 10000 QPS, то 100 запросов в секунду будут медленными — это 8.6 млн медленных запросов в сутки.

2.1.4. Связь метрик: от DAU к QPS

Ключевая задача — перевести бизнес-метрики (DAU) в технические (QPS, IOPS).

Полезные константы: секунд в сутках $\approx 10^5$ (точно: 86400), в месяце $\approx 2.5 \times 10^6$, в году $\approx 3 \times 10^7$.

Формула DAU $\rightarrow$ RPS:

$$RPS_{\text{средний}} = \frac{DAU \times \text{действий на пользователя}}{10^5}$$

Пример: 100K DAU, 20 действий/день $\rightarrow 100000 \times \frac{20}{100000} = 20$ RPS (средний).

Учёт пиков: нагрузка распределена неравномерно. Активные часы — 12 из 24, пиковые — 4 часа с нагрузкой в 3× от среднего.

$$RPS_{\text{пиковый}} \approx RPS_{\text{средний}} \times 3$$

RPS $\rightarrow$ QPS: один HTTP-запрос может порождать несколько запросов к БД.

- GET /products/:id $\rightarrow$ 1 SELECT (QPS = RPS)
- POST /orders $\rightarrow$ BEGIN + 3 INSERT + 2 UPDATE + COMMIT (QPS $\approx 6 \times$ RPS)

2.1.5. Иерархия памяти

Уровень	Латентность	Пропускная способность
L1 cache	1 нс	1 ТБ/с
L2 cache	4 нс	500 ГБ/с
L3 cache	12 нс	200 ГБ/с
RAM	100 нс	50 ГБ/с
SSD (random)	100 мкс	500 МБ/с

SSD (seq)	50 мкс	3 ГБ/с
HDD (random)	10 мс	1 МБ/с
HDD (seq)	5 мс	200 МБ/с

Случайный доступ на HDD в 200 раз дороже последовательного. На SSD разница меньше (5-10х), но всё ещё значительна.

Источник: таблица основана на данных Jeff Dean (Google) и Peter Norvig, актуализированных Colin Scott. См. Latency Numbers Every Programmer Should Know и Interactive Latency.

2.1.6. Оценка производительности: Little's Law

Как связать время запроса с пропускной способностью? **Little's Law** — фундаментальный закон теории очередей:

$$L = \lambda \times W$$

Где L — среднее число запросов в системе (concurrency), λ — throughput (RPS), W — среднее время обработки.

Применение для capacity planning:

- Знаем W (время запроса) и L (число ядер) $\rightarrow$ находим λ (max RPS)
- Знаем λ (целевой RPS) и $L \rightarrow$ находим требуемое W

Пример: сервер с 28 ядрами, время запроса 1 мс:

$$\lambda = \frac{L}{W} = \frac{28}{0.001} = 28000 \text{ RPS (теоретический максимум)}$$

Реальность: накладные расходы (context switching, lock contention, GC) снижают эффективность. Коэффициент полезной работы — 0.3–0.5:

$$\lambda_{\text{реальный}} = 28000 \times 0.4 \approx 11000 \text{ RPS}$$

Декомпозиция запроса — ключ к оценке W . Разбиваем HTTP-запрос на операции:

- CPU (parsing, сериализация): 10–100 мкс
- B-tree read: глубина $\times$ время чтения страницы (см. следующие лекции)
- B-tree write: аналогично + WAL
- COMMIT (fsync): 100–500 мкс на SSD

Sanity check: результат сравниваем с бенчмарками. PostgreSQL pgbench: 2000–3000 TPS (tuned). Если ваш расчёт даёт 100K+ RPS — пересмотрите допущения.

2.1.7. Фундаментальные компромиссы

Read vs Write: оптимизация под чтение замедляет запись и наоборот.

Space vs Time:

- **Компрессия** — тратим CPU за уменьшение места на диске
- **Денормализация** — тратим место за скорость чтения
- **Индексы** — тратим место и замедляем запись за ускорение чтения

2.2. Лекция 6: LSM Дерево

После анализа паттернов нагрузки встает вопрос: **как физически организовать данные на диске?** Первая структура, которую мы изучаем — **LSM-дерево** (Log-Structured Merge Tree).

LSM оптимизировано под **высокую скорость записи**. Это критично для систем, где запись доминирует над чтением: логи, метрики, IoT-телеметрия, временные ряды. Традиционные B-деревья требуют случайной записи на диск (дорого), а LSM превращает случайные записи в последовательные (дешево).

Принцип работы:

1. Записи сначала попадают в память (MemTable)
2. При заполнении MemTable сбрасывается на диск как отсортированный файл (SSTable)
3. Периодически файлы сливаются (compaction), чтобы избежать фрагментации

LSM используется в Cassandra, RocksDB, LevelDB, HBase. Лекция показывает компромиссы: быстрая запись, но чтение требует проверки нескольких файлов (read amplification).

2.2.1. Общая идея

LSM-дерево решает проблему эффективной записи путём отложенной сортировки и слияния. Вместо немедленной записи на диск в отсортированную структуру, данные буферизуются в памяти и периодически сбрасываются отсортированными порциями.

2.2.2. Отсортированная таблица строк (Sorted String Table, SSTable)

SSTable — имутабельный файл с парами ключ-значение, отсортированными по ключу.

Идея: данные ищутся крупными отсортированными блоками, что делает последовательное чтение быстрым и удобным для диапазонных запросов.

Что решает:

- Дешёвая запись на диск большими порциями
- Быстрые диапазонные чтения благодаря сортировке
- Предсказуемые Ввод-вывод за счёт последовательного доступа

Что внутри:

- Набор отсортированных блоков данных
- Лёгкий индекс на блоки, чтобы не читать весь файл

Go-псевдоинтерфейс и оценки:

```
// SSTable: имутабельное чтение по ключу и диапазону.
type SSTable interface {
    // Get: бинарный поиск по индексу + чтение блока.
    //  $O(\log b + k)$ ,  $b$  — число блоков,  $k$  — размер блока.
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: последовательное чтение по диапазону.
    //  $O(\log b + r)$ ,  $r$  — число записей в диапазоне.
    RangeScan(start, end []byte, fn func(k, v []byte))
}
```

2.2.3. Memtable

Memtable — изменяемая структура в памяти для накопления записей перед сбросом в SSTable.

Смысл для пользователя: быстрые записи (в памяти), а на диск данные уходят порциями, уже отсортированными.

Что решает:

- Низкая задержка записи
- Сглаживание нагрузки на диск

Что внутри:

- Упорядоченная структура в памяти (обычно Skip List или RB-дерево)

- Быстро поддерживает вставки и поиск по ключу

Go-псевдоинтерфейс и оценки:

```
// Memtable: изменяемые записи, сортировка в памяти.
type Memtable interface {
    // Put: вставка или обновление.
    // O(log m), m — число ключей.
    Put(key, value []byte)

    // Get: поиск по ключу.
    // O(log m).
    Get(key []byte) (value []byte, ok bool)

    // RangeScan: диапазонный проход.
    // O(log m + r).
    RangeScan(start, end []byte, fn func(k, v []byte))

    // Flush: упорядоченный сброс в новый SSTable.
    // O(m) на последовательную запись.
    Flush(builder SSTableBuilder)
}
```

2.2.4. Bloom Filter

Bloom Filter — вероятностная структура для быстрой проверки отсутствия ключа в SSTable.

Зачем он нужен: чтобы не лезть на диск, когда ключа точно нет. Если фильтр говорит “нет”, чтение файлов можно пропустить. Если говорит “возможно”, тогда делаем обычное чтение.

Практический эффект:

- Сильно сокращает Ввод-вывод при поиске несуществующих ключей
- Используется для каждого SSTable и держится в памяти
- Позволяет экономить дисковые обращения, особенно при большой базе

Что внутри:

- Битовый массив фиксированного размера
- Несколько хеш-функций, выставляющих биты
- Нет ложных отрицаний, но возможны ложные положительные ответы

Go-псевдоинтерфейс и оценки:

```
// BloomFilter: вероятностная проверка наличия.
type BloomFilter interface {
    // Add: устанавливает k битов.
    // O(k), k — число хешей.
    Add(key []byte)

    // MightContain: проверка k битов.
    // O(k).
    MightContain(key []byte) bool
}
```

2.2.5. Полная структура LSM-дерева

LSM-дерево — это конвейер для больших объёмов данных, где запись идёт быстро, а порядок поддерживается фоновыми слияниями.

Основные части:

1. WAL — журнал для надёжности: каждый запрос записи сначала фиксируется последовательно.
2. Memtable — быстрая запись и сортировка в памяти.
3. Immutable Memtable — замороженная память, ожидающая сброса.
4. Набор SSTable на диске — отсортированные файлы, каждый с Bloom Filter.

Как работает запись:

- Клиент пишет ключ-значение.
- Запись попадает в WAL и Memtable (быстро, в памяти).
- При переполнении Memtable сбрасывается в новый SSTable (последовательно).

Как работает чтение:

- Поиск идёт от самых свежих структур к старым: Memtable → Immutable → SSTable.
- Перед чтением SSTable проверяется Bloom Filter, чтобы не читать лишнее с диска.
- Результат берётся из первой найденной версии (самой свежей).

Как поддерживается порядок и место:

- На диске постепенно накапливаются SSTable с пересекающимися ключами.
- Фоновая компакция сливает их в более крупные и упорядоченные файлы.
- При слиянии старые версии и записи удаления выкидываются, освобождая место.

Что это даёт пользователю:

- Высокую скорость записи (почти всегда последовательная Ввод-вывод)
- Предсказуемые чтения при больших объёмах данных
- Поддержку диапазонных запросов благодаря сортировке

2.2.6. Практическое применение

Ключевое преимущество LSM: очень быстрые записи и высокая пропускная способность на больших объёмах данных за счёт последовательной Ввод-вывод и фоновых слияний.

Хорошо работает для операций:

- Частые вставки и обновления по ключу (write-heavy нагрузки)
- Пакетные записи и стриминг событий
- Диапазонные запросы по ключу (key range scan)
- Чтение свежих данных, когда важна скорость записи и допустима амортизированная задержка

Плохо подходит для:

- Случайных чтений по множеству ключей без кэша (много SSTable)
- Низкой и стабильной задержки чтения в p99 (компакция и пересечения)
- Частых операций удаления, если критично немедленно освободить место

Где в реальной жизни встречается:

- Журналы событий, аналитические события и трекинг поведения
- Метрики, тайм-серии, логи (высокий поток записи)
- Key-Value хранилища (конфиги, сессии, кеш с долговременной персистентностью)
- Большие базы, где критична скорость записи и последовательное чтение

Типичные системы на этой идее: LevelDB/RocksDB, Cassandra, HBase, Bigtable.

2.3. Лекция 7: В-дерево: индексирование на диске

LSM-дерево оптимизировано под запись, но что если нужна **быстрая точечная выборка** (чтение одной записи по ключу)? Для этого существует **В-дерево** — классическая структура индексирования, используемая в большинстве реляционных баз данных.

В-дерево оптимизировано под **случайное чтение**. Ключевая идея: минимизировать количество обращений к диску, группируя данные в узлы размером с дисковый блок (обычно 4-16 КБ). Один узел содержит много ключей (сотни), что дает высокий branching factor и малую глубину дерева.

Для поиска в базе на миллиард записей достаточно 3-4 чтений с диска ($\log_{1000}(1,000,000,000) \approx 3$). Это на порядки быстрее LSM, где нужно проверять несколько SSTable-файлов.

В-дерево используется в PostgreSQL, MySQL, Oracle, SQLite для индексов. Лекция показывает компромиссы: быстрое чтение, но медленная запись (требует in-place модификации узлов на диске).

В-дерево решает проблему эффективного использования кэша диска и минимизации случайных обращений путём группировки данных в узлы фиксированного размера, соответствующие блокам диска.

2.3.1. В-фактор и структура узла

В-фактор — это максимальное количество указателей на детей в одном узле (или эквивалентно, максимальное количество ключей + 1).

Внутренняя структура узла порядка В:

- Ключи: $[k_1, k_2, \dots, k_{B-1}]$ в отсортированном порядке (максимум $B - 1$ ключей)
- Указатели на детей: $[p_0, p_1, \dots, p_B]$ где p_i указывает на поддерево с ключами в диапазоне $[k_i, k_{i+1})$
- Листовой узел содержит значения вместо указателей на поддерева
- Каждый узел занимает ровно одно дисковое обращение — это ключевая идея

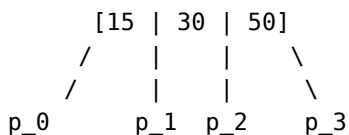
Выбор В-фактора:

- В выбирается так, чтобы узел точно поместился в один блок диска
- Если блок диска = 4 КБ, а каждая пара (ключ, указатель) занимает 10 байт, то $B \approx 400 - 500$
- Маленький В (например, В=4): много уровней дерева, много обращений к диску
- Большой В (например, В=500): глубина дерева $\mathcal{O}(\log_B n)$ очень мала (для 1 млн записей при В=500 глубина всего 3-4 уровня), но поиск внутри узла медленнее

Инвариант В-дерева:

- Все листья находятся на одной глубине h (в отличие от AVL, где глубина может отличаться на 1)
- Каждый внутренний узел содержит от $\lceil \frac{B}{2} \rceil$ до $B - 1$ ключей
- Корень содержит минимум 1 ключ

Пример узла В-дерева при В=4:



p_0: ключи < 15
p_1: ключи в [15, 30)
p_2: ключи в [30, 50)
p_3: ключи >= 50

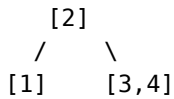
Пример роста В-дерева порядка В=3 при вставке 1,2,3,4,5:

Вставляем 1: [1]

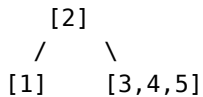
Вставляем 2: [1,2]

Вставляем 3: [1,2,3]

Вставляем 4: узел переполнен! Разделяем:



Вставляем 5:



При вставке нового элемента в переполненный узел:

- Узел разделяется на две половины
- Медиана поднимается в родителя
- Если родитель тоже переполнен, рекурсивно разделяем его
- Это может привести к вырастанию корня и увеличению высоты дерева на 1
- Create – $\mathcal{O}(\log_B n)$
 - Поиск листового узла $\mathcal{O}(\log_B n)$ дисковых обращений
 - Вставка ключа-значения в лист $\mathcal{O}(1)$
 - При переполнении узла: разделение на две половины и поднятие медианы $\mathcal{O}(\log_B n)$ обращений вверх по дереву
- Read – $\mathcal{O}(\log_B n)$
 - Бинарный поиск по ключам текущего узла $\mathcal{O}(\log B)$ — в памяти, быстро
 - Переход к дочернему узлу через одно дисковое обращение
 - Повторение до листа: максимум $\mathcal{O}(\log_B n)$ обращений
- Update – $\mathcal{O}(\log_B n)$
 - Поиск записи $\mathcal{O}(\log_B n)$
 - Обновление значения in-place (B-дерево не версионировано, в отличие от LSM)
- Delete – $\mathcal{O}(\log_B n)$
 - Поиск и удаление ключа $\mathcal{O}(\log_B n)$
 - При недостаточном количестве ключей: слияние с соседним узлом или перемещение ключей из соседа
- Range Query – $\mathcal{O}(\log_B n + \frac{R}{B})$, R — размер результата
 - Поиск левой границы $\mathcal{O}(\log_B n)$
 - Последовательное сканирование листьев (один узел = один читаемый блок диска)
 - Эффективно благодаря локальности: листья часто находятся рядом на диске

Преимущества относительно LSM:

- Нет компакций, стабильная задержка
- Меньше I/O амортизировано: B больше, чем типичный размер блока

2.3.2. Практическое применение

Ключевое преимущество B-дерева: предсказуемая и низкая задержка чтения/обновления за счёт прямого доступа к узлам на диске.

Хорошо работает для операций:

- Случайные чтения по ключу с жёсткими требованиями к задержке
- Чтение и обновление отдельных записей in-place
- Смешанные нагрузки (примерно равный read/write)
- Транзакционные OLTP базы данных

Плохо подходит для:

- Очень высоких скоростей записи (вставка вызывает дробление узлов)
- Сценариев с большим числом последовательных вставок, где важна чистая пропускная способность

- Стриминга событий, где запись идёт непрерывным потоком

Где в реальной жизни встречается:

- Реляционные БД с индексами: PostgreSQL, MySQL (InnoDB)
- Файловые системы и файловые индексы
- OLTP-системы с частыми точечными запросами и обновлениями

Недостатки:

- Запись требует read-modify-write: чтение узла → изменение → запись обратно
- Сложность балансировки при вставке и удалении
- Для частых записей LSM может затратить сильно меньше Ввод-вывод

2.4. Лекция 8: ACID, уровни изоляции, аналитические и транзакционные хранилища

После изучения структур хранения (LSM, B-tree) возникает вопрос: **как гарантировать корректность данных при одновременной работе множества клиентов?**

Представьте банковский перевод: деньги списываются со счёта А и зачисляются на счёт Б. Если между двумя операциями произойдет сбой, деньги могут исчезнуть (списались, но не зачислились) или удвоиться. Нужен механизм, гарантирующий: **либо обе операции выполнены, либо ни одна.**

Эта лекция вводит концепцию **транзакций** и свойства **ACID**:

- **Atomicity** — всё или ничего (commit или rollback)
- **Consistency** — данные остаются корректными (не нарушаются ограничения целостности)
- **Isolation** — параллельные транзакции не мешают друг другу
- **Durability** — после commit данные сохранены даже при сбое

Особенно сложен вопрос **изоляции**: если две транзакции читают и меняют одни и те же строки, какой результат они увидят? Лекция разбирает уровни изоляции (Read Uncommitted, Read Committed, Repeatable Read, Serializable) и аномалии (dirty read, phantom read).

2.4.1. Транзакции

Транзакция — Последовательность операций чтения и записи в базе данных, рассматриваемая как единая логическая единица работы. Гарантирует атомарность выполнения: либо все операции завершаются успешно и фиксируются (COMMIT), либо при ошибке все изменения откатываются (ROLLBACK).

BEGIN;

```
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
```

```
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
```

COMMIT; -- или ROLLBACK для отката

2.4.2. ACID (Atomicity, Consistency, Isolation, Durability)

- **Atomicity** — транзакция выполняется полностью или не выполняется вовсе; реализуется через WAL
- **Consistency** — поддерживаются ограничения целостности (foreign keys, constraints)
- **Isolation** — параллельные транзакции не влияют друг на друга; регулируется уровнями изоляции
- **Durability** — изменения сохраняются навсегда после COMMIT; обеспечивается fsync

2.4.3. Транзакционные и аналитические хранилища

OLTP — Online Transaction Processing — класс систем обработки транзакций в реальном времени. Характеризуется большим количеством коротких транзакций (чтение, вставка, обновление, удаление), строковым хранением данных и требованиями к консистентности.

- Паттерн: много мелких операций чтения/записи отдельных записей
- Примеры запросов:
 - SELECT * FROM users WHERE id = 123
 - UPDATE orders SET status = 'shipped' WHERE id = 456
 - INSERT INTO transactions (user_id, amount) VALUES (789, 100.50)
- Строковое хранение (row-oriented): вся строка читается за один блок диска
- СУБД: PostgreSQL, MySQL, Oracle
- Применение: интернет-магазины, банковские системы, CRM

OLAP — Online Analytical Processing — класс систем для аналитической обработки данных. Ориентирован на выполнение сложных агрегирующих запросов по большим объёмам данных, использует колоночное хранение для оптимизации производительности аналитики.

- Паттерн: большие агрегирующие запросы по миллионам строк
- Примеры запросов:
 - SELECT country, SUM(revenue) FROM sales GROUP BY country
 - SELECT date, COUNT(*) FROM events WHERE timestamp > '2024-01-01' GROUP BY date
 - Анализ трендов, построение отчётов, data science
- Колоночное хранение (column-oriented): колонки хранятся последовательно

- Читаем только нужные колонки (не всю строку)
- Лучше сжатие: однотипные данные в одном блоке
- Векторизованные вычисления на CPU
- СУБД: ClickHouse, Apache Druid, Vertica, Snowflake
- Применение: аналитика метрик, BI-дашборды, Data Warehouses
- Недостатки: медленные вставки и UPDATE/DELETE отдельных записей

2.4.4. Пример 1: маркетплейс

Вы заходите на маркетплейс. Какие данные куда попадают?

OLTP (транзакционные операции):

- Добавление товара в корзину → INSERT INTO cart (user_id, product_id)
- Подписка на продавца → UPDATE users SET subscriptions = subscriptions || 123
- Оформление заказа → транзакция с несколькими INSERT/UPDATE
- Проверка наличия товара → SELECT stock FROM products WHERE id = 456

OLAP (аналитические события):

- Клики по экранам/категориям → поток событий (user_id, screen, timestamp)
- Время на странице → (user_id, page_id, duration_sec)
- Информация о девайсе → (user_id, device_type, os, browser)
- Просмотры товаров без покупки → (user_id, product_id, timestamp)

Эти события собираются батчами и загружаются в OLAP-хранилище для анализа:

```
-- Сколько пользователей iOS заходило на категорию "Электроника" за неделю?
SELECT COUNT(DISTINCT user_id)
FROM events
WHERE screen = 'category_electronics' AND os = 'iOS'
      AND timestamp > NOW() - INTERVAL '7 days'
```

2.4.5. Пример 2: строковое vs колоночное хранение

Таблица sales (5 строк):

id	date	product	country	revenue
1	2024-01-10	Laptop	USA	1200
2	2024-01-10	Phone	Germany	800
3	2024-01-11	Laptop	USA	1200
4	2024-01-11	Tablet	France	600
5	2024-01-12	Phone	Germany	800

Строковое хранение (OLTP):

Блок 1: [1, 2024-01-10, Laptop, USA, 1200]

Блок 2: [2, 2024-01-10, Phone, Germany, 800]

Блок 3: [3, 2024-01-11, Laptop, USA, 1200]

...

Запрос SELECT * FROM sales WHERE id = 3 → читаем 1 блок (быстро). Запрос SELECT SUM(revenue) FROM sales → читаем все 5 блоков, парсим все колонки (медленно).

Колоночное хранение (OLAP):

id: [1, 2, 3, 4, 5]

date: [2024-01-10, 2024-01-10, 2024-01-11, 2024-01-11, 2024-01-12]

product: [Laptop, Phone, Laptop, Tablet, Phone]

country: [USA, Germany, USA, France, Germany]

revenue: [1200, 800, 1200, 600, 800]

Запрос SELECT SUM(revenue) FROM sales → читаем только колонку revenue (быстро). Запрос SELECT * FROM sales WHERE id = 3 → читаем все 5 колонок и собираем строку (медленно).

Сжатие на колоночном хранении:

- revenue: [1200, 800, 1200, 600, 800]
 - Dictionary Encoding: словарь {1200→0, 800→1, 600→2}, массив [0,1,0,2,1]
 - Было: 5 × 4 байта = 20 байт, стало: 3 × 4 + 5 × 2 бита = 13 байт
- country: [USA, Germany, USA, France, Germany]
 - Dictionary: {USA→0, Germany→1, France→2}, массив [0,1,0,2,1]
 - Было: 35 байт (строки), стало: 15 байт (словарь + индексы)
- product: [Laptop, Phone, Laptop, Tablet, Phone]
 - Dictionary + RLE: [(Laptop,1), (Phone,1), (Laptop,1), (Tablet,1), (Phone,1)]

2.4.6. Сжатие колонок

Run-Length Encoding (RLE) — кодирование серий повторяющихся значений:

Исходные данные: [5, 5, 5, 5, 3, 3, 7]

После RLE: [(5, 4), (3, 2), (7, 1)] // (значение, количество)

Эффективно для колонок с длинными последовательностями одинаковых значений (статусы, флаги, отсортированные данные).

Dictionary Encoding — замена повторяющихся значений индексами в словаре:

Исходные данные: [USA, Germany, USA, France, Germany, USA]

Словарь: {USA→0, Germany→1, France→2}

После кодирования: [0, 1, 0, 2, 1, 0]

Было: 6 строк × ~7 байт = 42 байта

Стало: 3 строки словаря (21 байт) + 6 индексов × 2 бита = ~23 байта

Эффективно для строковых колонок с низкой кардинальностью (страны, категории, статусы).

Bit Packing — использование минимального числа битов для хранения чисел:

Данные: [5, 12, 3, 15, 7] // максимум = 15 → нужно 4 бита

Обычное хранение: 5 × 32 бита = 160 бит

Bit packing: 5 × 4 бита = 20 бит (сжатие в 8 раз)

Эффективно для чисел с ограниченным диапазоном (возраст, рейтинги, маленькие счётчики).

Delta Encoding — хранение разностей между последовательными значениями:

Timestamps: [1704067200, 1704067260, 1704067320, 1704067380]
(2024-01-01 00:00:00, +60s, +60s, +60s)

После delta: [1704067200, 60, 60, 60]

Было: 4 × 32 бита = 128 бит

Стало: 32 бита (база) + 3 × 7 бит (дельты) = ~53 бита

Эффективно для монотонных последовательностей (timestamp, auto-increment id, версии).

Frame of Reference (FOR) — вычитание минимума + bit packing:

Данные: [1005, 1012, 1003, 1015, 1007]

Min = 1003, вычитаем: [2, 9, 0, 12, 4]

Максимум = 12 → нужно 4 бита

Было: 5 × 32 бита = 160 бит

Стало: 32 бита (min) + 5 × 4 бита = 52 бита (сжатие в 3 раза)

Эффективно для чисел в узком диапазоне (цены в одной категории, ID в пределах батча).

2.5. Лекция 9: Хранилища в оперативной памяти

Все предыдущие лекции были о дисковых структурах (LSM, B-tree). Но что если **вообще не использовать диск** и хранить все данные в RAM?

RAM в 100 000 раз быстрее диска для случайного доступа. Это радикально меняет дизайн: не нужно минимизировать дисковые обращения, можно использовать более простые структуры (хеш-таблицы вместо B-tree), не нужна сложная компрессия.

Лекция охватывает два класса in-memory систем:

- **OLTP** (Redis, Memcached) — key-value хранилища для кеша, сессий, rate limiting. Простые структуры данных (hash, list, set), миллионы операций в секунду.
- **OLAP** (DuckDB) — аналитическая база данных прямо в процессе приложения. Векторизованное выполнение (SIMD), колоночный формат в памяти, встроенная в Python/R/Node.js.

In-memory базы критичны для сценариев с низкой латентностью: real-time биддинг в рекламе (решение за менее 10 мс), финансовый трейдинг, игровые лидерборды. Также DuckDB революционизирует data science: вместо Pandas (медленный на больших данных) можно делать SQL-запросы прямо к Parquet-файлам с производительностью, близкой к ClickHouse.

2.5.1. Аналитика: DuckDB и векторизация

Векторизованное выполнение — обработка данных батчами (обычно 1–10 тыс. строк) вместо «строка-за-строкой»:

- Модель operator-at-a-time: один оператор работает сразу по вектору значений
- SIMD/AVX инструкции CPU и меньшие ветвления
- Лучшая локальность: кэш-линии и предвыборка памяти
- Меньше интерпретационного overhead и виртуальных вызовов

Почему это важно для аналитики:

- Колоночное хранение держит данные одного типа подряд, удобно для векторов
- Сжатие (RLE/Dictionary/Delta, описанные выше) распаковывается пачками и дает быстрые сканы
- Поздняя материализация: фильтрация по колонкам до сборки строк

DuckDB — встраиваемая аналитическая СУБД с векторизованным движком:

```
duckdb.sql("SELECT * FROM 'data.parquet' WHERE price > 100")
```

Архитектура:

- Встраивается в процесс (как SQLite), без сервера
- Zero-copy чтение из Parquet/CSV/JSON напрямую
- Морселизация: данные обрабатываются кусками 1024 строки
- Adaptive query execution: план корректируется по статистике во время выполнения

Когда DuckDB vs альтернативы:

- DuckDB: локальная аналитика на ноутбуке, 10 ГБ–1 ТБ данных, нужен SQL
- ClickHouse: серверная OLAP, петабайты, распределённые запросы
- Pandas: императивный код, < 10 ГБ, Python-экосистема

Где используют на практике:

- BI-дашборды и Ad-hoc анализ (быстрые сканы больших таблиц)
- ETL/ELT и проверка качества данных
- Feature engineering в ML и аналитика экспериментов
- Встраиваемая аналитика в приложениях и data-products
- Интерактивный анализ логов/метрик на ноутбуках и edge-узлах

2.5.2. Кэширование: Redis

Хранилище в оперативной памяти (in-memory key-value store).

Архитектура:

- Однопоточная модель + event-driven I/O (epoll/kqueue)

- Нет overhead на синхронизацию

Структуры данных:

- **Skip List** (sorted sets) — уже рассмотрен в LSM; ZADD leaderboard 100 "player1"
- **Incremental Rehashing** — постепенное перехеширование без блокировки
- **HyperLogLog** — вероятностный подсчёт уникальных; 12 КБ для миллиардов элементов
- Strings, Lists, Sets, Hashes, Bitmaps, Streams

Применение: кэширование, Сессия, очереди, real-time аналитика

3. Методы поиска похожих/близких элементов

Предыдущий модуль был про **точный поиск**: найти запись по ключу (B-tree), выполнить транзакцию (ACID), посчитать агрегацию (OLAP). Но многие задачи требуют **поиска похожих элементов**, где нет точного совпадения.

Примеры:

- **Текстовый поиск** — найти документы, содержащие слова “machine learning” (не точное совпадение строки, а поиск по словам)
- **Рекомендации** — найти товары, похожие на те, что купил пользователь (семантическая близость)
- **Поиск изображений** — найти фотографии, похожие на загруженную (визуальная близость)
- **Геопоиск** — найти рестораны в радиусе 1 км (пространственная близость)

Для этих задач не подходят классические индексы (B-tree, hash). Нужны специализированные структуры:

- **Inverted Index** — для полнотекстового поиска (Google, Elasticsearch)
- **Vector Search** — для семантического поиска (эмбединги текста, изображений, пользователей)
- **Geospatial Indexes** — для поиска по координатам (карты, доставка, такси)

Модуль показывает, как эти структуры работают и когда их использовать.

3.1. Лекция 10: Специализированные индексы для поиска

Первая задача — **полнотекстовый поиск**: найти все документы, содержащие определённые слова. Нельзя просто сканировать все тексты (слишком медленно), нужна индексная структура.

Inverted Index (инвертированный индекс) — основа всех поисковых систем (Google, Elasticsearch, Sphinx). Идея проста: вместо хранения “документ → слова” храним “слово → список документов”. Тогда поиск слова “machine” сводится к одному обращению к индексу.

Лекция показывает:

- Построение inverted index с токенизацией и стеммингом
- Операции AND/OR для комбинации терминов
- Ранжирование по TF-IDF (частота слова в документе vs частота в корпусе)
- Bloom Filter — вероятностная структура для быстрой проверки “слово точно НЕ в документе” (экономия дисковых чтений)

Эти структуры используются не только в поиске, но и в логах (быстрая проверка “содержит ли лог-файл ошибку X”) и метаданных (Cassandra использует Bloom Filter для оптимизации чтения SSTable).

3.1.1. Inverted Index (перевёрнутые индексы)

В прошлом модуле мы изучили прямой индекс, который ищет данные по ключу (см. B-дерево). Основная идея обратного индекса: хранить для каждого термина список документов, где он встречается. Это позволяет быстро находить документы по словам запроса.

Go-псевдоинтерфейс и оценки:

```
// InvertedIndex: термин -> список документов.
type InvertedIndex interface {
    // Add: добавить документ в индекс.
    // O(t), t — число терминов в документе.
    Add(docID int, terms []string)

    // QueryAND: пересечение posting lists.
    // O(sum L_i), L_i — длины списков терминов.
    QueryAND(terms []string) []int

    // QueryOR: объединение posting lists.
    // O(sum L_i).
    QueryOR(terms []string) []int
}
```

Внутреннее устройство:

- Словарь терминов: term -> смещение posting list на диске
- Posting list: отсортированный список docID с частотами и позициями

Пример индекса (термины выделены цветом):

```
cat -> [1, 3]
sat -> [1, 2]
dog -> [2]
ran -> [3]
```

Добавление документа Doc4: "cat sat sat":

```
cat -> [1, 3, 4]
sat -> [1, 2, 4]    // tf=2, positions=[2,3] для Doc4
```

Удаление Doc2 (логическое):

```
dog -> [2]    // docID=2 помечен удалённым
sat -> [1, 2] // docID=2 остается до пересборки/слияния
```

Фразовый запрос "cat sat":

- 1) пересечение списков cat и sat -> [1]
- 2) проверка позиций: cat@1, sat@2 -> фраза совпала

Практическое применение:

- Полнотекстовый поиск: слова, фразы
- Быстрые запросы по большим корпусам документов
- Поиск по логам/документации/каталогам
- Системы: Elasticsearch, Lucene, Solr

3.1.2. Trie (префиксные деревья)

Основная идея: хранить ключи посимвольно, чтобы быстро отвечать на запросы по префиксу.

Go-псевдоинтерфейс и оценки:

```
// Trie: поиск по префиксу.
type Trie interface {
    // Insert: добавить слово.
    // O(L), L — длина слова.
    Insert(word string)

    // Contains: точный поиск.
    // O(L).
    Contains(word string) bool

    // StartsWith: получить все слова по префиксу.
    // O(P + r), P — длина префикса, r — число результатов.
    StartsWith(prefix string) []string
}
```

Внутреннее устройство:

- Узлы по символам, переходы по алфавиту
- Маркер окончания слова
- Компрессия (radix trie) снижает память

Пример построения:

Вставляем "car":

```
root -> c -> a -> r (*)
```

Вставляем "cat":

```
root -> c -> a -> r (*)
        \-> t (*)
```

Поиск префикса “ca”:

- 1) идём root -> c -> a
- 2) обходим поддерево и получаем: car, cat

Удаление “car”:

- 1) снимаем маркер окончания с r
- 2) если у r нет детей, удаляем r и поднимаемся вверх

Практическое применение:

- Автодополнение, подсказки, поиск по префиксу
- Поиск по словарям, маршрутизация по строковым ключам

3.1.3. Bitmap indexes

Основная идея: для каждого значения категории хранить битовый массив, где 1 означает, что запись содержит это значение.

Go-псевдоинтерфейс и оценки:

```
// BitmapIndex: значение -> битовый вектор.
type BitmapIndex interface {
    // Set: установить бит для записи.
    // 0(1).
    Set(value string, rowID int)

    // And/Or: булевы операции над выборками.
    // 0(n/word), n — число строк.
    And(values []string) []int
    Or(values []string) []int
}
```

Внутреннее устройство:

- Для каждого значения хранится битсет длиной = числу строк
- Быстрые битовые операции AND/OR/NOT по словам машинного слова
- Сжатие битсетов (Roaring, WAH) для экономии памяти

Практическое применение:

- Фильтры по категориальным полям (status, country, type)
- Аналитические запросы и агрегации
- Колоночные хранилища и OLAP системы

3.2. Лекция 11: Векторный поиск и embedding

Inverted Index хорош для точного поиска слов, но не понимает **смысл**. Запрос “купить ноутбук” не найдет документ со словом “laptop”, хотя это синонимы. Нужен **семантический поиск** — поиск по смыслу, а не по точным словам.

Решение — **векторные представления (embeddings)**. Модель машинного обучения переводит текст, изображение или товар в вектор чисел (обычно 128-1536 размерность). Близкие по смыслу объекты получают близкие векторы в многомерном пространстве.

Примеры:

- Текст: “машинное обучение” и “machine learning” → похожие векторы (даже на разных языках!)
- Изображения: две фотографии кошек → близкие векторы
- Рекомендации: пользователи с похожими покупками → близкие векторы

Лекция показывает:

- Как создаются эмбединги (BERT для текста, ResNet для изображений)
- Метрики близости: cosine similarity (угол между векторами), euclidean distance, dot product
- Применения: семантический поиск (Google, ChatGPT), рекомендации (Spotify, Netflix), детекция дубликатов

Ключевая задача: как **быстро найти ближайших соседей** среди миллионов векторов? Это подводит к следующей лекции про ANN (Approximate Nearest Neighbors).

3.2.1. Основы векторных представлений

Идея: объект (текст, изображение, товар) переводится в вектор чисел так, чтобы близкие по смыслу объекты были близки по расстоянию.

Go-псевдоинтерфейс и оценки:

```
// VectorStore: хранение и поиск по близости.
type VectorStore interface {
    // Add: добавить вектор в индекс.
    // O(d), d — размерность.
    Add(id int, vec []float32)

    // Search: найти k ближайших и вернуть id + метрику близости.
    // O(k * log n) амортизировано для индекса, O(n*d) для полного скана.
    Search(query []float32, k int) []Neighbor
}

type Neighbor struct {
    ID      int
    Score   float32 // расстояние или similarity
}
```

Внутренняя идея:

- Векторы фиксированной размерности (например, 384/768)
- Индекс нужен, чтобы не сравнивать запрос со всеми векторами

Метрики близости:

Косинусное сходство (Cosine similarity)

$$\cos(q, v) = \frac{q \cdot v}{\|q\| \cdot \|v\|}$$

- Смысл: сравнивает направление, не зависит от длины вектора
- Когда использовать: семантический поиск, где длина текста не должна влиять на близость
- Пример: запрос “как вернуть товар” должен найти и короткую карточку FAQ, и длинную статью поддержки про возвраты — обе про один смысл, хотя объем текста разный
- Пример: поиск похожих запросов в Support-тикетах: короткие и длинные формулировки про один дефект должны совпадать

Евклидово расстояние L2 (L2 distance)

$$\|q - v\|_2 = \sqrt{\sum_i (q_i - v_i)^2}$$

- Смысл: абсолютное расстояние между точками в пространстве
- Когда использовать: когда масштаб признаков имеет значение и “дальше” должно быть строго хуже
- Пример: поиск похожих товаров по числовым характеристикам (вес, размеры, объём батареи). Большие отличия по одному из параметров должны увеличивать расстояние
- Пример: поиск похожих изображений по Эмбедингу из модели, где длина вектора несет информацию о интенсивности/яркости и ее нельзя игнорировать

Скалярное произведение (Dot product)

$$q \cdot v = \sum_i q_i v_i$$

- Смысл: близость растет, если векторы “смотрят” в одну сторону и имеют большую длину
- Когда использовать: когда длина вектора отражает силу сигнала (уверенность, популярность)
- Пример: рекомендательная система — вектор пользователя и вектор товара. Большой модуль товара (популярный) усиливает сходство
- Пример: при ранжировании контента важен не только смысл, но и вес/сила признака (например, частота взаимодействий)

Практическое применение:

- Семантический поиск по текстам и документам
- Поиск похожих товаров/изображений/песен
- Рекомендации и кластеризация

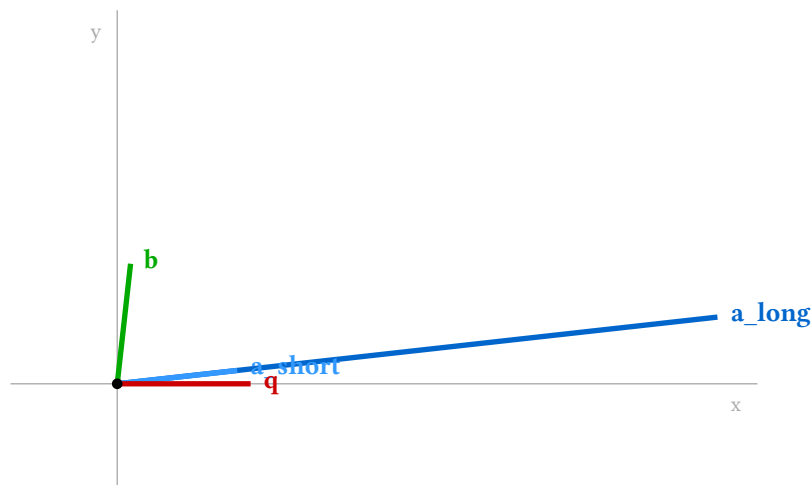
3.2.2. Нормализация векторов и выбор метрики

Нормализация вектора — приведение длины к 1: $\hat{v} = \frac{v}{\|v\|}$. После нормализации все векторы лежат на единичной сфере.

Пример вычисления метрик с нормализацией и без:

Векторы:

- $q = [1.0, 0.0]$ (запрос)
- $a_short = [0.9, 0.1]$ (близкое направление, короткий)
- $a_long = [4.5, 0.5]$ (то же направление что a_short , длинный в 5×)
- $b = [0.1, 0.9]$ (перпендикулярное направление, короткий)



Метрика	$f(q, a_s)$	$f(q, a_l)$	$f(q, b)$	$f(q_n, a_s_n)$	$f(q_n, a_l_n)$	$f(q_n, b_n)$
Косинус	0.994	0.994	0.11	0.994	0.994	0.11
L2	0.141	3.536	1.273	0.111	0.111	1.334

Dot product	0.9	4.5	0.1	0.994	0.994	0.11
-------------	-----	-----	-----	-------	-------	------

Наблюдения:

- Косинус** не зависит от длины:
 - $\cos(q, a_{\text{short}}) = \cos(q, a_{\text{long}}) \approx 0.994$ (одно направление, разная длина)
 - $\cos(q, b) \approx 0.11$ (другое направление)
 - После нормализации значения те же $\rightarrow$ косинус инвариантен к масштабу
- Dot product** сильно зависит от длины:
 - Без нормализации: $\text{dot}(q, a_{\text{short}}) = 0.9$, $\text{dot}(q, a_{\text{long}}) = 4.5$, $\text{dot}(q, b) = 0.1 \rightarrow a_{\text{long}}$ “выигрывает” у a_{short} в $5\times$ раз, хотя направление то же!
 - С нормализацией: $\text{dot}(q_n, a_{\text{short}_n}) = \text{dot}(q_n, a_{\text{long}_n}) \approx 0.994 \rightarrow$ одинаковы
 - Вывод:** без нормализации dot product “награждает” длинные векторы
- L2 расстояние:**
 - Без нормализации: $L2(q, a_{\text{short}}) = 0.14$, $L2(q, a_{\text{long}}) = 3.54$, $L2(q, b) = 1.27 \rightarrow a_{\text{short}}$ ближе, чем a_{long} (короче), хотя направление то же
 - С нормализацией: $L2(q_n, a_{\text{short}_n}) = L2(q_n, a_{\text{long}_n}) \approx 0.111 \rightarrow$ одинаковы $\rightarrow L2(q_n, b_n) \approx 1.334 \rightarrow$ дальше (другое направление)
 - Вывод:** после нормализации L2 игнорирует длину, смотрит только на направление
- Для нормализованных: косинус = dot product (например, $0.994 = 0.994$)
- $L2^2$ для норм. $\approx 2 - 2 \times \text{косинус}$: $0.012 \approx 0.012$

Практический вывод:

После нормализации все три метрики становятся эквивалентны по порядку ранжирования:

$$\cos(u, v) = u \text{ dot } v = 1 - \frac{\|u-v\|^2}{2}$$

Доказательство:

Пусть u, v — нормализованные векторы ($\|u\| = \|v\| = 1$).

$$1. \text{ Косинус} = \text{скалярное произведение: } \cos(u, v) = \frac{u \text{ dot } v}{\|u\| \text{ dot } \|v\|} = \frac{u \text{ dot } v}{1 \text{ dot } 1} = u \text{ dot } v$$

$$2. \text{ Связь с L2: } \|u - v\|^2 = (u - v) \text{ dot } (u - v) = u \text{ dot } u - 2(u \text{ dot } v) + v \text{ dot } v$$

Для нормализованных: $u \text{ dot } u = \|u\|^2 = 1$, аналогично $v \text{ dot } v = 1$:

$$\|u - v\|^2 = 1 - 2(u \text{ dot } v) + 1 = 2 - 2(u \text{ dot } v)$$

$$\text{Отсюда: } u \text{ dot } v = 1 - \frac{\|u-v\|^2}{2}$$

$$3. \text{ Итоговая цепочка: } \cos(u, v) = u \text{ dot } v = 1 - \frac{\|u-v\|^2}{2}$$

Проверка на наших данных:

- $\cos(q_n, a_{\text{short}_n}) = 0.9939$
- $\text{dot}(q_n, a_{\text{short}_n}) = 0.9939$
- $1 - L2^2/2 = 1 - 0.0122/2 = 0.9939$

Все три значения совпадают ✓

Следствия для векторного поиска:

- Поиск k максимальных по косинусу = поиск k максимальных по dot product = поиск k минимальных по L2
- Индексы для L2 (например, HNSW) работают для косинусного поиска после нормализации
- Вычисление dot product быстрее косинуса (не нужны нормы и деление)

Когда нормализация недопустима:

Если длина вектора несёт смысл, нормализация теряет информацию:

- Dot product как мера силы сигнала (уверенность модели, популярность товара)
- Эмбединги изображений, где модуль кодирует интенсивность/яркость
- Рекомендации, где длина = частота взаимодействий

В этих случаях косинус и L2 по нормализованным векторам искажают результат.

3.2.3. Применение в проектировании систем

Понимание связи метрик и нормализации критично при выборе векторного индекса:

Архитектурные решения:

1. Семантический поиск (поиск по смыслу, не по ключевым словам):
 - Нормализуем векторы при индексации и поиске
 - Поиск ближайших соседей по L2 → эквивалентен косинусному поиску
 - Экономим на вычислениях: dot product вместо полной формулы косинуса
2. Рекомендательные системы с учётом популярности:
 - НЕ нормализуем: длина вектора товара = популярность
 - Ищем соседей по dot product (максимальное скалярное произведение)
 - Компромисс: гибридный подход (фильтрация по популярности + поиск по нормализованным)
3. Выбор метрики в зависимости от задачи:
 - Косинус (семантика без учёта длины) → нормализация + поиск по L2
 - L2 (абсолютное расстояние важно) → поиск по L2 без нормализации
 - Dot product (длина несёт смысл) → поиск максимального скалярного произведения

Практические сценарии выбора метрики и индекса:

1. Поиск похожих товаров в маркетплейсе:

- Задача: найти похожие товары по описанию для рекомендаций “похожие товары”
- Решение: длина вектора не важна (важен только смысл описания) → нормализуем векторы, ищем по L2

```
embeddings = model.encode(product_descriptions)
embeddings /= np.linalg.norm(embeddings, axis=1, keepdims=True)
# Поиск k ближайших соседей по L2 = косинусный поиск
```

2. Поиск ответов в документации для support-тикетов:

- Задача: по запросу “как вернуть товар” найти релевантные статьи базы знаний
- Решение: семантическая близость важнее длины текста → нормализация + поиск по L2

```
query_emb = model.encode(user_query)
query_emb /= np.linalg.norm(query_emb)
# Ищем ближайшие статьи по L2 расстоянию
```

3. Рекомендации товаров с учётом популярности:

- Задача: рекомендовать товары пользователю, популярные товары должны ранжироваться выше
- Решение: длина вектора товара = популярность → НЕ нормализуем, используем dot product

```
# Векторы товаров: ||v|| больше для популярных
item_vectors = embeddings * popularity_weights[:, None]
# Ищем максимальное скалярное произведение
scores = user_vector @ item_vectors.T
top_items = np.argsort(scores)[-1:]
```

4. Дедупликация документов (поиск дубликатов):

- Задача: найти почти идентичные документы (например, копии статей)
- Решение: важна семантическая идентичность → нормализация + косинусный поиск (через L2)

```
normalized_emb = embeddings / np.linalg.norm(embeddings, axis=1, keepdims=True)
# Для каждого документа ищем ближайшие по L2
# Малое L2 расстояние = высокое косинусное сходство
```

Во всех этих сценариях возникает общая проблема: как эффективно найти k ближайших соседей среди миллионов векторов? Наивный перебор (brute-force) требует $O(n \times d)$ операций — для базы из 10М векторов размерности 768 это 7.7 миллиардов операций на запрос.

Следующая лекция посвящена индексным структурам, которые решают эту проблему через приближённый поиск (Approximate Nearest Neighbors, ANN) — жертвуем небольшой точностью ради многократного ускорения.

3.3. Лекция 12: Приближённый поиск ближайших соседей (ANN)

Точный поиск k ближайших соседей (k -NN) требует сравнения запроса со всеми векторами в базе. Для больших коллекций (миллионы-миллиарды векторов) это неприемлемо медленно. Приближённый поиск (ANN) строит специальные индексы, которые находят “почти ближайших” соседей за $O(\log n)$ вместо $O(n)$.

Компромисс ANN:

- **Recall** (полнота): доля истинно ближайших соседей среди найденных (обычно 90-99%)
- **Latency** (задержка): время поиска (миллисекунды вместо секунд)
- **Memory** (память): индекс требует дополнительной памяти сверх векторов

Основные подходы к ANN:

- **Графовые индексы** (HNSW, NSW) — навигация по графу близости
- **Хеш-индексы** (LSH) — похожие векторы попадают в одни корзины
- **Квантизация** (IVF, PQ) — разбиение пространства на кластеры

3.3.1. HNSW (Hierarchical Navigable Small World)

Основная идея:

Проблема: найти k ближайших векторов к запросу. Brute-force перебирает все n векторов в базе, вычисляя расстояние до каждого ($O(n \times d)$, где d — размерность). Для миллионов векторов это слишком медленно.

Решение: строим многослойный граф:

- Верхние слои — «магистральные» длинные рёбра для быстрого перехода к нужной области
- Нижние слои — локальные короткие рёбра для точного поиска

Алгоритм поиска: спускаемся сверху вниз, на каждом слое жадно переходим к ближайшему соседу. Сложность **sublinear** — эмпирически близко к $O(\log n)$ при правильном выборе параметров M и ef (вместо $O(n)$ у brute-force).

Поддерживаемые метрики:

- **Евклидово расстояние (L2)** — обычное расстояние между точками в пространстве
- **Косинусное сходство** — мера угла между векторами (чем меньше угол, тем ближе)
 - Важно: векторы должны быть нормализованы (длина = 1)

Ограничение: HNSW показывает лучшие результаты на метрических расстояниях (L2 или косинус через нормализацию). Скалярное произведение (inner product) без нормализации не является метрикой (не выполняется неравенство треугольника), что может снизить recall.

Варианты работы с inner product:

- Нормализовать векторы → сводится к косинусному расстоянию
- Использовать специальные трансформации (например, MIPS→L2)
- Применить другие индексы (например, IVF с inner product)

Go-псевдоинтерфейс и оценки:

```
// HNSW: графовый индекс ближайших соседей.
type HNSW interface {
    // Add: вставка с поиском входной точки.
    // O(M * log n) амортизировано.
    Add(id int, vec []float32)

    // Search: greedy поиск по слоям.
    // O(M * log n) амортизировано.
    Search(query []float32, k int) []int
}
```

Структура данных:

Многослойный граф:

- Вершины = векторы
- Рёбра = связи между близкими векторами (не более M соседей на вершину)

- Слои L0, L1, L2, ...:
 - L0 (нижний) содержит все векторы
 - L1 содержит случайное подмножество (50% от L0)
 - L2 содержит случайное подмножество L1 (25% от L0)
 - И так далее до единственной вершины на верхнем слое (entry point)

Каждая вершина хранит:

```
type Node struct {
    id      int
    vector  []float32
    level   int           // максимальный слой, где есть эта вершина
    layers  [][]int        // соседи на каждом слое: layers[l] = [id1, id2, ...]
}
```

Параметр M (макс. число рёбер): обычно 16-32

- Больше M → точнее поиск, но медленнее вставка и больше памяти
- Меньше M → быстрее, но хуже recall

Добавление вектора:

1. Генерируем случайный уровень l для новой вершины v
(экспоненциальное распределение: $p(l) = (1/2)^l$)
2. Начинаем с entry_point на верхнем слое
Для каждого слоя сверху вниз до l :
 - Жадный поиск: идём к ближайшему соседу, пока улучшается расстояние
 - Запоминаем найденную вершину как точку входа для следующего слоя
3. На слоях от l до 0 (где v присутствует):
 - Ищем efConstruction ближайших соседей (efConstruction ~ 100-200)
 - Выбираем из них M лучших (эвристика: разнообразие + близость)
 - Добавляем рёбра $v \rightarrow$ соседи
 - Для каждого соседа: добавляем ребро сосед $\rightarrow v$
Если у соседа стало $> M$ рёбер, обрезаем до M (оставляем лучшие)
4. Если l выше текущего entry_point, обновляем entry_point = v

Поиск к ближайших:

1. Начинаем с entry_point на верхнем слое
curr = entry_point
2. Для каждого слоя сверху вниз до L0:
 - Жадный спуск: пока есть улучшение
 - * Смотрим всех соседей curr на этом слое
 - * Переходим к ближайшему к queue
 - * Повторяем, пока curr не перестанет улучшаться
 - Переходим на слой ниже с текущим curr
3. На L0 (нижний слой):
 - Расширяем поиск: поддерживаем priority queue из ef кандидатов (ef ~ 50-500)
 - Для каждого кандидата смотрим его соседей
 - Добавляем в queue, если улучшают расстояние
 - Останавливаемся, когда нет новых улучшений
 - Возвращаем топ-k из queue

Пример 1: Структура многослойного графа

9 точек в 2D-пространстве с координатами:

```
A(0,0)  B(1,0)  C(2,0)  D(3,0)  E(4,0)
         F(2,1)  G(3,1)  H(4,1)  I(5,1)
```

Граф построен с параметром $M=3$ (макс. 3 соседа на вершину на слое):

Слой L2 (entry point):

A: [D] // только одно дальнее ребро-магистраль
D: [A]

Слой L1:

A: [C, D]
C: [A, D, F]
D: [A, C, G]
G: [D]

Слой L0 (все векторы):

A: [B, C]
B: [A, C]
C: [A, B, D, F]
D: [C, E, G]
E: [D, H]
F: [C, G]
G: [D, F, H]
H: [E, G, I]
I: [H]

Ключевое свойство: **одна вершина может быть на нескольких слоях, но списки соседей разные.**
Например, D есть на L2, L1 и L0 с разными ребрами.

Пример 2: Поиск ближайшего к query(5.2, 1.1)

Ожидаемый результат: I(5,1) — ближайшая точка.

Фаза 1: Жадный спуск по слоям

L2: Старт с entry=A(0,0)
- dist(query, A) = 5.3
- Соседи A на L2: [D]
- dist(query, D) = 2.2 → D ближе, переходим
- Соседи D на L2: [A] → нет улучшений
→ Спускаемся на L1 с curr=D

L1: curr=D(3,0)
- Соседи D на L1: [A, C, G]
- dist(query, A) = 5.3
- dist(query, C) = 3.3
- dist(query, G) = 2.2 → G ближе, переходим
- Соседи G на L1: [D] → нет улучшений
→ Спускаемся на L0 с curr=G

L0: curr=G(3,1)
- Соседи G на L0: [D, F, H]
- dist(query, D) = 2.3
- dist(query, F) = 3.2
- dist(query, H) = 1.2 → H ближе, переходим
- Соседи H на L0: [E, G, I]
- dist(query, I) = 0.2 → I ближе, переходим
- Соседи I на L0: [H] → нет улучшений
→ Жадный поиск остановился на I

Фаза 2: Расширенный поиск с ef (на L0)

Поддерживаем очередь кандидатов размера ef=10
Стартуем с I, добавляем соседей I → [H]
Из H берем соседей → [E, G, I]
Из E → [D, H], из G → [D, F, H]

Собрали кандидатов: {I, H, E, G, D, F}

Вычисляем точные расстояния:

$\text{dist}(\text{query}, I) = 0.20$

$\text{dist}(\text{query}, H) = 1.20$

$\text{dist}(\text{query}, E) = 1.56$

...

Топ-3 результата: [I, H, E]

Вывод: Жадный спуск быстро приводит в нужную область (за $O(\log n)$ переходов), а расширенный поиск с ef подбирает соседей для точности.

Пример 3: Вставка новой точки $v(3.5, 0.8)$

1. Генерируем уровень v : $\text{level}(v) = 1$ (случайно)
→ v будет присутствовать на $L1$ и $L0$
2. Поиск точки входа (как в примере 2):
 $L2: A \rightarrow D$ ($\text{curr}=D$)
 $L1: D \rightarrow G$ ($\text{curr}=G$)
 $L0: G$ (уже близко к v)
3. Вставка на $L1$:
 - Ищем $\text{efConstruction}=5$ ближайших к v на $L1$
 - Кандидаты: $\{D(\text{dist}=0.95), G(\text{dist}=0.54), C(\text{dist}=1.73), A(\text{dist}=3.62)\}$
 - Выбираем $M=3$ лучших: $[G, D, C]$
 - Добавляем рёбра: $v \rightarrow [G, D, C]$
 - Обратные рёбра: $G.\text{neighbors_L1} += [v]$, $D.\text{neighbors_L1} += [v]$, $C.\text{neighbors_L1} += [v]$
 - Проверяем: у D теперь $[A, C, G, v] = 4$ соседа $> M$
Обрезаем до $M=3$ лучших: $D.\text{neighbors_L1} = [C, G, v]$ (убрали дальнего A)
4. Вставка на $L0$:
 - Аналогично: $v \rightarrow [G, D, F]$
 - Обновляем обратные рёбра

Итоговый граф:

$L2$: без изменений (v не попал на $L2$)

$L1$:

$A: [C]$ // изменено: убрали D
 $C: [A, D, F, v]$ // добавили v
 $D: [C, G, v]$ // изменено: убрали A , добавили v
 $G: [D, v]$ // добавили v
 $v: [G, D, C]$ // новая вершина

$L0$:

$D: [C, E, G, v]$ // добавили v
 $F: [C, G, v]$ // добавили v
 $G: [D, F, H, v]$ // добавили v (обрезали до $M=3$, если нужно)
 $v: [G, D, F]$ // новая вершина
(остальные без изменений)

Ключевой момент: вставка может изменить граф (обрезка рёбер при превышении M), поэтому качество графа зависит от порядка вставок. Практика: случайный порядок или периодическая переиндексация.

Практическое применение:

- Семантический поиск с большими коллекциями (миллионы векторов)
- Поиск ближайших соседей в рекомендациях
- Быстрый приближённый поиск (ANN, Approximate Nearest Neighbors) с управляемой точностью (recall 95% за 1-5 мс)

3.3.2. LSH (Locality-Sensitive Hashing)

Основная идея:

Проблема: как быстро найти похожие векторы без построения сложных графов (как в HNSW)?

Решение: хешируем векторы специальным образом — **похожие векторы с высокой вероятностью попадают в один бакет хеш-таблицы**. Вместо сравнения со всеми векторами в базе, сравниваем только с теми, что попали в те же бакеты.

Ключевое свойство locality-sensitive hash-функции:

Если $\text{distance}(v_1, v_2)$ мала $\rightarrow P(\text{hash}(v_1) == \text{hash}(v_2))$ велика

Если $\text{distance}(v_1, v_2)$ велика $\rightarrow P(\text{hash}(v_1) == \text{hash}(v_2))$ мала

Go-псевдоинтерфейс и оценки:

```
// LSH: вероятностный поиск ближайших.
type LSH interface {
    // Add: добавить вектор в бакеты.
    // O(L * k * d), L — число таблиц, k — число хешей.
    Add(id int, vec []float32)

    // Search: искать в бакетах запроса.
    // O(L * k * d + c * d), c — кандидаты.
    Search(query []float32, k int) []int
}
```

Структура данных:

LSH использует **L независимых хеш-таблиц**, каждая со своей хеш-функцией:

Таблица 1: $\text{hash}_1(v) \rightarrow$ бакет в таблице 1

Таблица 2: $\text{hash}_2(v) \rightarrow$ бакет в таблице 2

...

Таблица L: $\text{hash}_L(v) \rightarrow$ бакет в таблице L

Параметры:

- **L** (число таблиц) — обычно 10-50. Больше L $\rightarrow$ выше recall, но медленнее
- **k** (число хешей на таблицу) — обычно 3-10. Больше k $\rightarrow$ меньше коллизий, но жёстче фильтр

Random Projection LSH (для косинусного расстояния):

Самая популярная схема для векторных поисков. Используем **случайные гиперплоскости**.

Одна хеш-функция:

1. Генерируем случайный вектор r (нормальное распределение)
2. $\text{hash_bit}(v) = \text{sign}(v \cdot r)$ // скалярное произведение
= 1, если $v \cdot r \geq 0$
= 0, если $v \cdot r < 0$

Гиперплоскость r делит пространство на 2 полупространства. Векторы с одной стороны $\rightarrow \text{hash}=1$, с другой $\rightarrow \text{hash}=0$.

Для одной таблицы используем **k случайных гиперплоскостей** $r_1, r_2, \dots, r_k$:

```
hash(v) = [hash_bit(v, r1), hash_bit(v, r2), ..., hash_bit(v, rk)]
          = битовая строка длины k
          = номер бакета (от 0 до  $2^k - 1$ )
```

Пример: 6 векторов в 2D

Векторы (нормализованные):

A(1.0, 0.2) B(0.9, 0.3) C(0.3, 0.9)
D(0.2, 1.0) E(-0.7, 0.7) F(-1.0, -0.1)

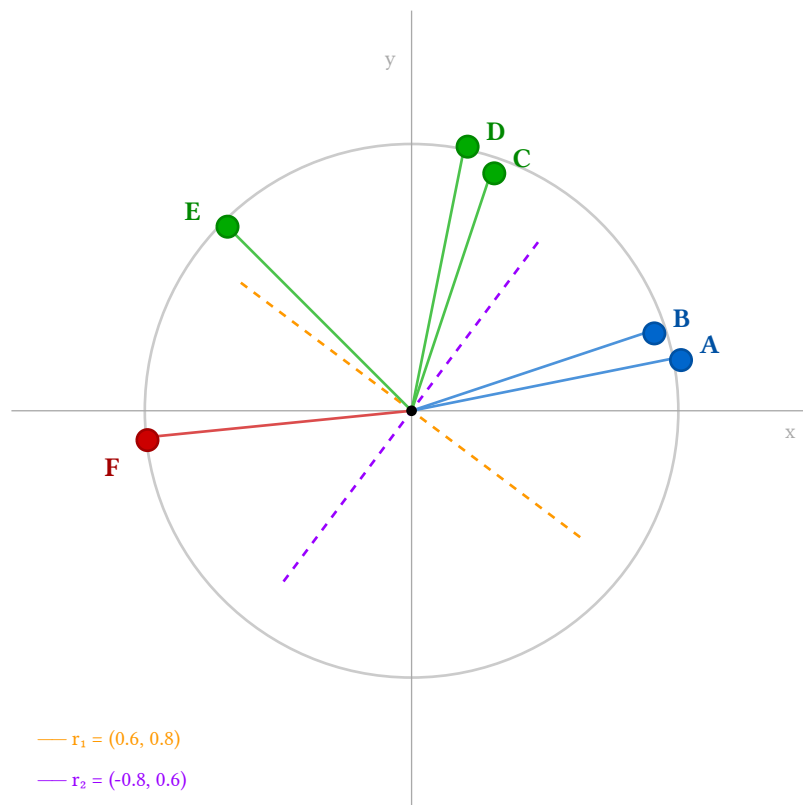


Figure 1: Векторы на единичной окружности с гиперплоскостями. Цвета: синий = бакет 2 (A,B), зелёный = бакет 3 (C,D,E), красный = бакет 1 (F)

Строим LSH с $L=2$ таблицами, $k=2$ хешей на таблицу.

Таблица 1:

Случайные векторы:

$r_1 = (0.6, 0.8)$ // гиперплоскость наклонена вправо-вверх
 $r_2 = (-0.8, 0.6)$ // гиперплоскость наклонена влево-вверх

Вычисляем хеши:

A(1.0, 0.2): $A \cdot r_1 = 0.76 > 0 \rightarrow \text{bit}_1=1$, $A \cdot r_2 = -0.68 < 0 \rightarrow \text{bit}_2=0$
hash(A) = [1,0] = бакет 2

B(0.9, 0.3): $B \cdot r_1 = 0.78 > 0 \rightarrow \text{bit}_1=1$, $B \cdot r_2 = -0.54 < 0 \rightarrow \text{bit}_2=0$
hash(B) = [1,0] = бакет 2

C(0.3, 0.9): $C \cdot r_1 = 0.90 > 0 \rightarrow \text{bit}_1=1$, $C \cdot r_2 = 0.30 > 0 \rightarrow \text{bit}_2=1$
hash(C) = [1,1] = бакет 3

D(0.2, 1.0): $D \cdot r_1 = 0.92 > 0 \rightarrow \text{bit}_1=1$, $D \cdot r_2 = 0.44 > 0 \rightarrow \text{bit}_2=1$
hash(D) = [1,1] = бакет 3

E(-0.7, 0.7): $E \cdot r_1 = 0.14 > 0 \rightarrow \text{bit}_1=1$, $E \cdot r_2 = 0.98 > 0 \rightarrow \text{bit}_2=1$
hash(E) = [1,1] = бакет 3

F(-1.0, -0.1): $F \cdot r_1 = -0.68 < 0 \rightarrow \text{bit}_1=0$, $F \cdot r_2 = 0.74 > 0 \rightarrow \text{bit}_2=1$

$$\text{hash}(F) = [0, 1] = \text{бакет } 1$$

Бакеты таблицы 1:

Бакет 1: [F]

Бакет 2: [A, B] ← A и B близки → попали в один бакет ✓

Бакет 3: [C, D, E] ← C, D близки → попали в один бакет ✓

Таблица 2 (другие случайные r_3, r_4):

Аналогично строим вторую таблицу с другими гиперплоскостями.

Из-за случайности могут быть другие группировки.

Поиск query(0.85, 0.4):

1. Вычисляем $\text{hash}(\text{query})$ для каждой таблицы:

Таблица 1: $\text{hash}(\text{query}) = [1, 0] = \text{бакет } 2$

Таблица 2: $\text{hash}(\text{query}) = [1, 1] = \text{бакет } 3$ (допустим)

2. Собираем кандидатов из соответствующих бакетов:

Из таблицы 1, бакет 2: {A, B}

Из таблицы 2, бакет 3: {C, D, E} (допустим)

Объединение: {A, B, C, D, E}

3. Точно вычисляем расстояния только для кандидатов:

$\text{dist}(\text{query}, A) = 0.15$

$\text{dist}(\text{query}, B) = 0.10$ ← ближайший

$\text{dist}(\text{query}, C) = 0.58$

$\text{dist}(\text{query}, D) = 0.72$

$\text{dist}(\text{query}, E) = 1.52$

4. Возвращаем топ-k: [B, A]

Не проверяли F → сэкономили вычисления!

Почему несколько таблиц ($L > 1$)?

С одной таблицей можем пропустить близкие векторы (низкий recall):

- Если query и близкий вектор v случайно попали в разные бакеты → v потеряли

С L таблицами:

- Достаточно попасть в один бакет **хотя бы в одной таблице**
- Вероятность пропустить близкий вектор снижается экспоненциально

Компромисс:

- Больше L → выше recall, но больше кандидатов → медленнее
- Больше k → меньше ложных срабатываний, но можем пропустить близкие векторы

Практическое применение:

- Дедупликация больших коллекций (поиск почти-дубликатов документов)
- Первичная фильтрация в поисковых системах (миллиарды векторов)
- Сценарии, где допустим trade-off recall/скорость (recall 70-80%)
- Предфильтрация перед точным ранжированием (LSH → сузили до 1000 кандидатов → точный rerank)

Сравнение LSH и HNSW:

Характеристика	LSH	HNSW
Recall (точность)	70-85% — вероятностный	90-99% — детерминированный
Скорость поиска	Субмиллисекунды — только хеширование	1-5 мс — обход графа
Память на вектор	Компактно: несколько битов на таблицу	Граф: $M \times L$ связей (обычно 100-500 байт)

Характеристика	LSH	HNSW
Скорость вставки	$O(L \times k)$ — очень быстро, только хеширование	$O(M \log n)$ — медленнее, поиск + обновление графа
Параметры настройки	L (таблицы), k (хеши на таблицу)	M (макс. связей), ef (размер очереди)
Когда использовать	- Огромные базы (миллиарды векторов) - Допустим низкий recall - Нужна максимальная скорость - Предфильтрация кандидатов	- Средние/большие базы (миллионы) - Нужен высокий recall (>95%) - Можно потратить 1-10 мс - Продакшен-поиск с гарантиями

Когда выбирать комбинацию:

LSH (грубый фильтр) → HNSW (точный rerank)
 ↓ ↓
 1 млрд векторов → 10K кандидатов → топ-100 результатов

3.3.3. Векторные БД в продакшене

Специализированные векторные базы данных:

Weaviate (open-source):

- Индекс: HNSW по умолчанию
- Фичи: GraphQL API, гибридный поиск (вектор + фильтры), multi-tenancy
- Когда: нужен self-hosted с богатым API и фильтрацией
- Пример use case: семантический поиск по документам с фильтрами по метаданным (дата, автор, теги)

Pinecone (managed SaaS):

- Индекс: проприетарная оптимизация поверх HNSW
- Фичи: автоматическое шардирование, репликация, serverless масштабирование
- Когда: не хочется управлять инфраструктурой, нужна отказоустойчивость out-of-the-box
- Пример use case: recommendation engine с миллионами пользователей

Qdrant (open-source, Rust):

- Индекс: HNSW с оптимизациями
- Фичи: payload-фильтры, sparse vectors, quantization для экономии памяти
- Когда: нужен производительный self-hosted с низким overhead
- Пример use case: поиск похожих изображений в e-commerce с фильтрацией по категориям

Где встречается в реальных продуктах:

RAG (Retrieval-Augmented Generation):

Вопрос → векторный поиск по базе знаний → контекст для LLM → ответ

- Чат-боты техподдержки (поиск по тикетам)
- Корпоративные ассистенты (поиск по внутренним документам)
- Научные поисковики (поиск релевантных статей)

E-commerce и контент:

- Поиск похожих товаров по изображению (Zalando, ASOS)
- Рекомендации контента (YouTube, Spotify embeddings)
- Дедупликация (поиск дубликатов товаров у разных продавцов)

Мониторинг и безопасность:

- Поиск аномалий в логах через embeddings (Elastic)
- Детекция фрода по паттернам транзакций
- Поиск похожих инцидентов в системах мониторинга

3.4. Лекция 13: Геопространственные структуры

Последний тип поиска похожих элементов — **геопространственный поиск**: найти все объекты в определённой области или ближайшие к точке.

Задачи:

- **Карты и навигация** — найти все рестораны в радиусе 1 км, POI на видимой части карты
- **Доставка и такси** — найти ближайшего водителя, рассчитать зоны доставки
- **IoT и логистика** — отслеживание транспорта, геофенсинг (оповещения при входе в зону)
- **Игры** — поиск ближайших игроков, рендеринг объектов на карте

Нельзя использовать обычные индексы (B-tree работает с одномерными ключами, а координаты — двумерные). Нужны специализированные структуры для 2D/3D пространства.

Лекция разбирает три классических подхода:

- **R-Tree** — иерархия ограничивающих прямоугольников (MBR). Оптимален для произвольных областей (здания, районы). Используется в PostGIS, SQLite.
- **QuadTree** — рекурсивное деление пространства на 4 квадранта. Прост, хорош для равномерных данных (игры, рендеринг карт).
- **KD-Tree** — деление по медиане попеременно по осям X и Y. Оптимален для kNN-поиска точек (ближайший ресторан, ближайший водитель).

Каждая структура оптимизирована под свой паттерн запросов. Лекция показывает, когда использовать какую.

3.4.1. R-Tree

Основная идея:

Проблема: найти все объекты (точки, прямоугольники) в заданной области на карте. Перебор всех объектов $O(n)$ слишком медленный для миллионов точек.

Решение: **группируем объекты по минимальным ограничивающим прямоугольникам (MBR, Minimum Bounding Rectangle)**. Строим иерархическое дерево прямоугольников — если запрос не пересекает MBR узла, пропускаем все его дочерние объекты.

Go-псевдоинтерфейс и оценки:

```
// RTree: пространственный индекс для прямоугольников и точек.
type RTree interface {
    // Insert: вставка объекта с его MBR.
    //  $O(\log n)$  в среднем,  $O(n)$  в худшем.
    Insert(id int, rect Rect)

    // Search: поиск всех объектов, пересекающих область.
    //  $O(\log n + r)$ , где  $r$  — число найденных объектов.
    Search(area Rect) []int

    // Nearest: поиск k ближайших объектов к точке.
    //  $O(\log n + k)$  в среднем.
    Nearest(point Point, k int) []int
}
```

Структура данных:

R-Tree — это **сбалансированное дерево**, похожее на B-Tree, но для пространственных данных:

- **Листовые узлы** содержат MBR реальных объектов (рестораны, дома, границы районов)
- **Внутренние узлы** содержат MBR, покрывающие все дочерние узлы
- **Параметр M** — максимальное число детей в узле (обычно 4-16)
- **Минимальное заполнение** — обычно $M/2$ (как в B-Tree)

Ключевое свойство: MBR могут **перекрываться** (в отличие от B-Tree, где ключи разделены четко).

Пример: построение R-Tree для 8 ресторанов

Объекты (точки на карте):

A: Кафе "Север" (1, 8) E: Пиццерия "Запад" (2, 3)
B: Бургерная "Восток" (8, 9) F: Суши "Центр" (5, 4)
C: Паста "Юг" (9, 2) G: Стейкхаус "Парк" (7, 5)
D: Вок "Площадь" (3, 1) H: Кофейня "Набережная" (6, 7)

Построим R-Tree с M=4 (макс. 4 объекта в узле):

Шаг 1: Группировка объектов по близости

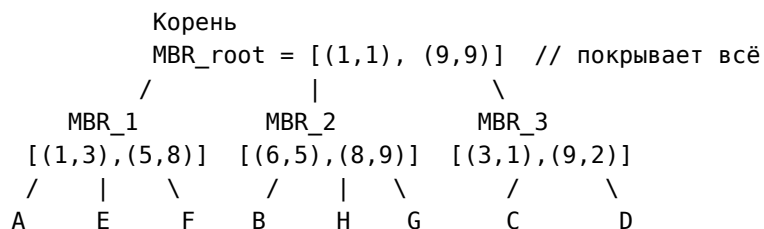
Алгоритм вставки пытается минимизировать увеличение площади MBR. Группируем:

Группа 1 (северо-запад): A(1,8), E(2,3), F(5,4)
MBR_1 = [(1,3), (5,8)] // левый нижний и правый верхний углы

Группа 2 (северо-восток): B(8,9), H(6,7), G(7,5)
MBR_2 = [(6,5), (8,9)]

Группа 3 (юг): C(9,2), D(3,1)
MBR_3 = [(3,1), (9,2)]

Шаг 2: Дерево



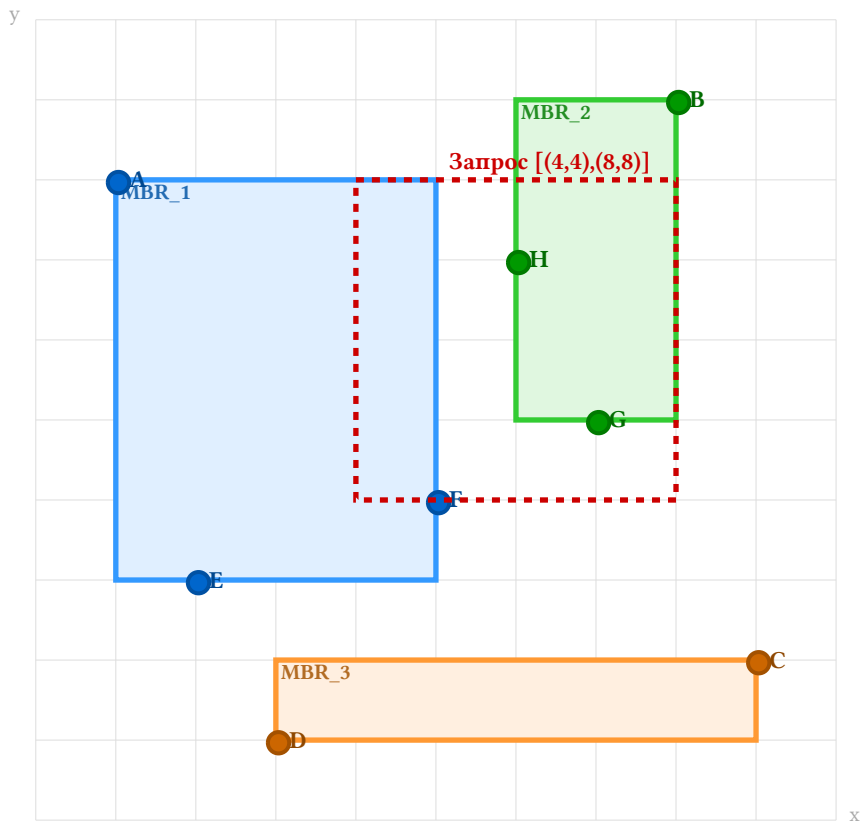


Figure 2: R-Tree: 8 ресторанов сгруппированы в 3 MBR. Красный пунктир — область запроса. MBR_3 не пересекается → объекты C, D не проверяются

Пример поиска: найти все рестораны в области $[(4,4), (8,8)]$

Шаг 1: Проверяем корень

MBR_root = $[(1,1), (9,9)]$ пересекает $[(4,4), (8,8)]$ ✓
→ Идём вглубь

Шаг 2: Проверяем детей корня

MBR_1 = $[(1,3), (5,8)]$:
Пересечение: $x: [4,8] \cap [1,5] = [4,5]$ ✓
 $y: [4,8] \cap [3,8] = [4,8]$ ✓
→ Пересекает, идём в MBR_1

MBR_2 = $[(6,5), (8,9)]$:
Пересечение: $x: [4,8] \cap [6,8] = [6,8]$ ✓
 $y: [4,8] \cap [5,9] = [5,8]$ ✓
→ Пересекает, идём в MBR_2

MBR_3 = $[(3,1), (9,2)]$:
Пересечение: $x: [4,8] \cap [3,9] = [4,8]$ ✓
 $y: [4,8] \cap [1,2] = \emptyset$ ✗
→ НЕ пересекает, пропускаем C и D!

Шаг 3: Проверяем объекты в MBR_1

A(1,8): $1 \notin [4,8]$ ✗
E(2,3): $2 \notin [4,8]$ ✗
F(5,4): $5 \in [4,8], 4 \in [4,8]$ ✓ → Найден!

Шаг 4: Проверяем объекты в MBR_2

B(8,9): $8 \in [4,8], 9 \notin [4,8]$ ✗

H(6,7): $6 \in [4,8]$, $7 \in [4,8]$ ✓ → Найден!

G(7,5): $7 \in [4,8]$, $5 \in [4,8]$ ✓ → Найден!

Результат: [F, H, G] — проверили только 6 из 8 объектов

Ключевой момент: благодаря MBR_3 мы **не проверяли** объекты C и D, сэкономив вычисления!

Вставка нового объекта:

Добавим ресторан I: Гриль “Мост” (4, 6)

1. Начинаем с корня, ищем узел с минимальным увеличением площади

MBR_1 до: [(1,3), (5,8)], площадь = $4 \times 5 = 20$

MBR_1 после вставки I(4,6): [(1,3), (5,8)] — не меняется!

Увеличение площади: 0

MBR_2 до: [(6,5), (8,9)], площадь = $2 \times 4 = 8$

MBR_2 после вставки I(4,6): [(4,5), (8,9)], площадь = $4 \times 4 = 16$

Увеличение площади: 8

→ Выбираем MBR_1 (меньшее увеличение)

2. Вставляем I в MBR_1:

MBR_1 теперь содержит: A, E, F, I (4 объекта — лимит M=4)

Если бы при вставке узел переполнился (>M объектов), происходит **split** — узел разбивается на два, как в B-Tree.

Практическое применение:

Геопоиск:

- Найти все рестораны в радиусе 1 км от пользователя (Яндекс.Карты, Google Maps)
- Поиск недвижимости в районе (Циан, Avito)
- Логистика: какие заказы можно забрать курьеру в области

ГИС (геоинформационные системы):

- Поиск пересечений границ районов, дорог, зданий
- Анализ застройки (какие дома попадают в зону риска)

Базы данных:

- **PostGIS** (расширение PostgreSQL) — использует R-Tree для индексов GIST
- **SQLite R-Tree** — встроенный модуль для мобильных приложений
- **MongoDB** — 2dsphere индекс на основе вариаций R-Tree

Игры и физика:

- Детекция коллизий (какие объекты пересекаются на сцене)
- Видимость объектов (frustum culling в 3D-движках)

3.4.2. QuadTree

Основная идея:

Проблема: та же, что у R-Tree — быстрый геопоиск. Но QuadTree использует **регулярное разбиение пространства** вместо адаптивных MBR.

Решение: рекурсивно делим плоскость на 4 равных квадранта до тех пор, пока в каждом квадранте не останется мало объектов (обычно ≤ 4).

Структура:

- Каждый узел представляет квадрат на карте
- Если в квадрате >N объектов → делим на 4 подквадранта (NW, NE, SW, SE)
- Листья содержат до N объектов

Go-псевдоинтерфейс и оценки:

```
// QuadTree: регулярное разбиение 2D-пространства.  
type QuadTree interface {  
    Insert(id int, p Point)          // O(log n) для сбалансированного  
    RangeSearch(area Rect) []int    // O(log n + r)  
    Nearest(p Point, k int) []int    // O(log n + k)  
}
```

Пример: добавление 9 зданий на карту

Точки (здания):

A(1,7) B(2,8) C(3,6) D(6,8) E(7,7)
F(1,2) G(3,3) H(7,2) I(8,1)

Параметр: макс. 2 объекта на квадрант.

Шаг 1: Начальный квадрат [0,0]-[10,10]

Вставляем A, B → 2 объекта, OK

Вставляем C → 3 объекта > 2 → SPLIT!

Шаг 2: После split

```
      [0,0]-[10,10]  
    /   |   |   \  
  NW   NE  SW  SE  
[0,5] [5,5] [0,0] [5,0]  
[5,10] [10,10] [5,5] [10,5]
```

NW: A(1,7), B(2,8), C(3,6) → 3 объекта > 2 → SPLIT снова!

Шаг 3: Финальное дерево после всех вставок

```
      Root [0,0]-[10,10]  
    /   |   |   \  
  NW   NE  SW  SE  
[0,5,5,10] [0,0,5,5]  
 /  |  \  \  
NW NE SW SE      SW SE  
A  C  -  B      G  F
```

D, E в NE корня

H, I в SE корня

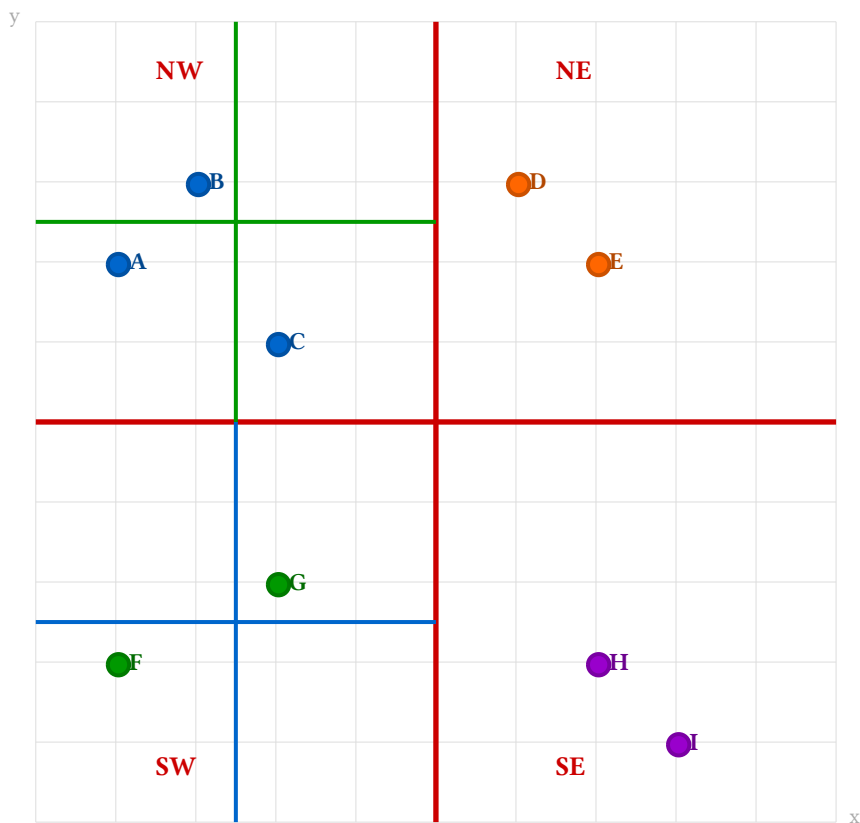


Figure 3: QuadTree: регулярное разбиение на квадранты. Красные линии — основное деление, синие/зелёные — подразбиения переполненных квадрантов NW и SW

Поиск в области $[(2,2), (7,7)]$:

1. Корень: пересекает $\rightarrow$ идём в детей
2. Проверяем квадранты корня:
 - NW $[0,5,5,10]$: пересекает $[(2,5), (5,7)]$ $\checkmark \rightarrow$ идём вглубь
 - NE $[5,5,10,10]$: пересекает $[(5,5), (7,7)]$ $\checkmark \rightarrow$ идём вглубь
 - SW $[0,0,5,5]$: пересекает $[(2,2), (5,5)]$ $\checkmark \rightarrow$ идём вглубь
 - SE $[5,0,10,5]$: пересекает $[(5,2), (7,5)]$ $\checkmark \rightarrow$ идём вглубь
3. В каждом квадранте проверяем объекты:
 - Найдены: C(3,6), G(3,3), E(7,7) — другие за пределами

Преимущество: простая логика разбиения, хорошо для равномерно распределённых данных.

Недостаток: если точки сгруппированы в одном углу, дерево несбалансированное (глубина $O(n)$).

Практическое применение:

- Игровые движки (пространственное разбиение сцены)
- Рендеринг карт (тайлы OpenStreetMap — это QuadTree)
- Симуляции частиц (поиск столкновений)

3.4.3. KD-Tree

Основная идея:

Проблема: QuadTree делит пространство на равные части — неэффективно для неравномерных данных. R-Tree адаптивный, но сложный.

Решение: делим пространство попеременно по осям (x, y, z, x, y, z...), выбирая медиану объектов. Получаем бинарное дерево с балансировкой.

Структура:

- Узел хранит точку и ось разбиения (x или y)
- Левое поддерево: объекты с меньшей координатой по этой оси
- Правое поддерево: объекты с большей координатой

Go-псевдоинтерфейс и оценки:

```
// KDTree: k-мерное бинарное дерево.  
type KDTree interface {  
    Insert(id int, p Point)          //  $O(\log n)$  для сбалансированного  
    RangeSearch(area Rect) []int     //  $O(\sqrt{n} + r)$  в среднем  
    Nearest(p Point, k int) []int    //  $O(\log n + k)$  в среднем  
}
```

Пример: построение KD-Tree для 7 точек

Точки:

A(2,3) B(5,4) C(9,6) D(4,7) E(8,1) F(7,2) G(3,9)

Строим 2D KD-Tree (чередуем оси $x \rightarrow y \rightarrow x \rightarrow y \dots$):

Уровень 0 (ось X):

Сортируем по X: A(2), G(3), D(4), B(5), F(7), E(8), C(9)

Медиана: B(5,4)

Левые ($x < 5$): A, G, D

Правые ($x \geq 5$): F, E, C

Уровень 1 (ось Y):

Левая ветка (сортируем по Y): A(3), D(7), G(9)

Медиана: D(4,7)

Левые ($y < 7$): A

Правые ($y \geq 7$): G

Правая ветка (сортируем по Y): E(1), F(2), C(6)

Медиана: F(7,2)

Левые ($y < 2$): E

Правые ($y \geq 2$): C

Итоговое дерево:

```
      B(5,4) [ось X]  
     /      \  
  D(4,7) [Y]  F(7,2) [Y]  
   /    \  
A(2,3) G(3,9) E(8,1) C(9,6)
```

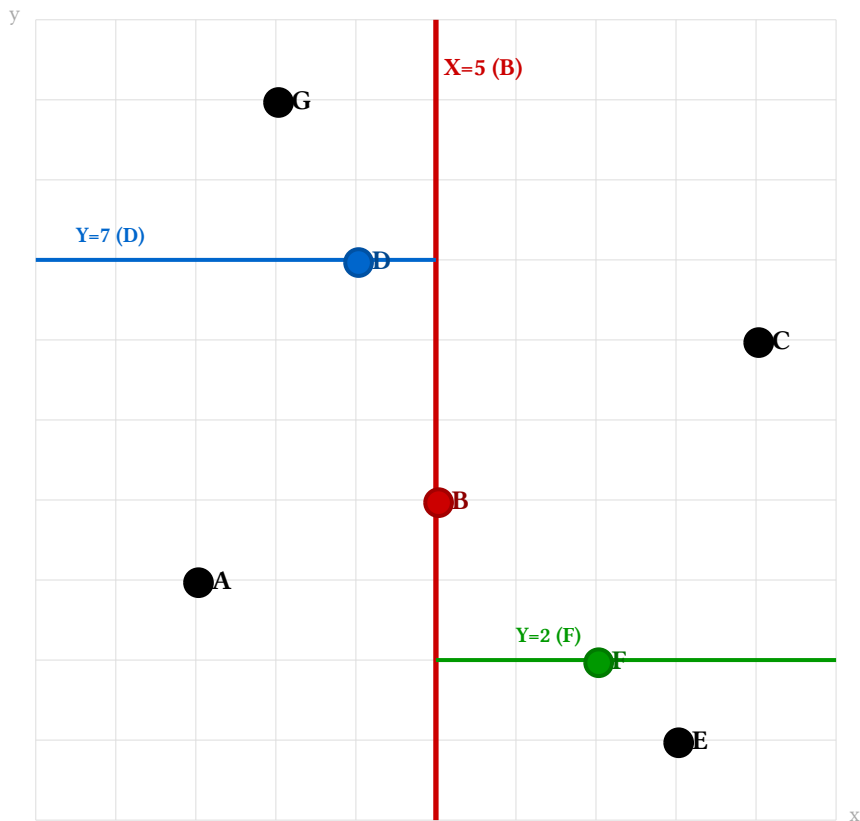


Figure 4: KD-Tree: разбиение по медианам с чередованием осей. Красная линия $X=5$ (корень B), синяя $Y=7$ (D для левой половины), зелёная $Y=2$ (F для правой половины)

Поиск ближайшего соседа к query(6,5):

1. Старт: $B(5,4)$, $\text{dist} = \sqrt{((6-5)^2 + (5-4)^2)} = 1.41$
best = B, best_dist = 1.41
2. B разбивает по X: $\text{query}.x=6 > B.x=5 \rightarrow$ идём вправо к F
3. $F(7,2)$, $\text{dist} = \sqrt{((6-7)^2 + (5-2)^2)} = 3.16 > 1.41$
best остаётся B
4. F разбивает по Y: $\text{query}.y=5 > F.y=2 \rightarrow$ идём вправо к C
5. $C(9,6)$, $\text{dist} = \sqrt{((6-9)^2 + (5-6)^2)} = 3.16 > 1.41$
best остаётся B
6. Откат: нужно ли проверить левую ветку F?
Расстояние до гиперплоскости $Y=2$: $|5-2|=3 > 1.41$
 $\rightarrow$ Нет, можем пропустить E
7. Откат к B: нужно ли проверить левую ветку B?
Расстояние до гиперплоскости $X=5$: $|6-5|=1 < 1.41$
 $\rightarrow$ ДА! Проверяем D
8. $D(4,7)$, $\text{dist} = \sqrt{((6-4)^2 + (5-7)^2)} = 2.83 > 1.41$
best остаётся B

Результат: B — ближайший сосед

Ключевой момент: откаты необходимы, но часто удаётся пропустить целые ветки (как ветку E).

Практическое применение:

- Поиск k ближайших соседей (kNN) в ML
- Поиск ближайших точек на карте (быстрее, чем R-Tree для малых k)
- Компьютерная графика (ray tracing, поиск пересечений)

Сравнение пространственных структур:

Характеристика	R-Tree	QuadTree	KD-Tree
Разбиение	Адаптивные MBR	Регулярная сетка (4 квадранта)	Медианы по осям
Баланс дерева	Гарантирован (как B-Tree)	Зависит от данных	Гарантирован при построении
Range search	$O(\log n + r)$ — отлично	$O(\log n + r)$ — хорошо	$O(\sqrt{n} + r)$ — хуже
Nearest neighbors	$O(\log n + k)$	$O(\log n + k)$	$O(\log n + k)$ — лучше всех
Память	Высокая (MBR на узел)	Средняя	Низкая (только точки)
Лучший use case	Прямоугольные объекты (здания, районы)	Равномерные данные (тайлы карт, игры)	kNN для точек (ML, рекомендации)

Когда что выбирать:

- **R-Tree**: геопоиск по областям, объекты с размерами (здания, парки)
- **QuadTree**: игры, рендеринг карт, равномерное разбиение
- **KD-Tree**: kNN-поиск, точки без размера, ML

4. Операционные компоненты

После того как мы разобрали структуры данных для хранения и поиска (векторный поиск, геопространственные индексы), переходим к **операционным компонентам** — инфраструктурным элементам, которые обеспечивают надёжность, безопасность и наблюдаемость систем в продакшене.

Эти компоненты находятся «вокруг» вашего приложения и решают практические задачи:

- **Прокси и балансировка нагрузки** — как распределять трафик между несколькими инстансами приложения, защищаться от перегрузок
- **Безопасность** — как контролировать доступ к ресурсам (аутентификация, авторизация, интеграция с внешними провайдерами)
- **Наблюдаемость** — как понять, что происходит в системе (логи, метрики, трассировка, планирование задач)

Без этих компонентов приложение может работать локально, но в продакшене оно будет уязвимо к перегрузкам, атакам и непредсказуемым сбоям. Модуль показывает, что инфраструктура — это не просто «поднять сервер», а выстроить экосистему вокруг кода.

4.1. Лекция 14: Прокси и Rate Limiting

Когда ваше приложение начинает обрабатывать реальный трафик, возникают две проблемы: **как распределять нагрузку между несколькими инстансами приложения?** и **как защититься от перегрузок?**

Если у вас один сервер, всё просто — клиенты подключаются напрямую. Но как только вы запускаете несколько копий приложения (для масштабирования или резервирования), нужен механизм распределения трафика. Прокси-серверы решают эту задачу, а алгоритмы балансировки определяют, на какой инстанс отправить запрос.

Rate Limiting (ограничение скорости запросов) защищает от перегрузок: злоумышленники могут отправлять тысячи запросов в секунду, случайный всплеск трафика может положить базу данных, или один пользователь может случайно запустить бесконечный цикл запросов. Алгоритмы rate limiting позволяют контролировать нагрузку на уровне прокси, не доводя запросы до приложения.

Эта лекция объясняет, как nginx, HAProxy и Envoy становятся «умными шлюзами» перед вашим приложением.

4.1.1. Forward Proxy vs Reverse Proxy

Forward Proxy (прокси для клиента):

Клиент → Forward Proxy → Интернет

Назначение: прокси действует от имени клиентов, скрывая их от внешних серверов.

Use cases:

- Корпоративные сети: контроль доступа сотрудников к интернету
- VPN и обход блокировок (клиент подключается через прокси в другой стране)
- Кеширование контента (прокси сохраняет часто запрашиваемые ресурсы)
- Анонимизация (сервер видит IP прокси, а не клиента)

Пример:

Сотрудник компании → Корпоративный прокси → youtube.com

↓
Логирует запросы
Блокирует запрещенные сайты
Кеширует статику

Reverse Proxy (прокси для сервера):

Клиенты → Reverse Proxy → Backend серверы

Назначение: прокси действует от имени серверов, скрывая внутреннюю инфраструктуру от клиентов.

Use cases:

- Load balancing (распределение запросов по серверам)
- SSL termination (расшифровка HTTPS на прокси, внутри — HTTP)
- Кеширование статики и API-ответов
- Защита от DDoS (rate limiting, фильтрация)
- Единая точка входа для микросервисов

Пример: nginx как reverse proxy

```
upstream backend {  
    server app1.internal:3000;  
    server app2.internal:3000;  
    server app3.internal:3000;  
}  
  
server {  
    listen 80;  
    location / {  
        proxy_pass http://backend;  
        proxy_set_header Host $host;  
    }  
}
```

Популярные решения:

- **nginx** — легковесный, быстрый, до 10K req/s на одном инстансе
- **HAProxy** — специализированный балансировщик, TCP+HTTP
- **Envoy** — modern cloud-native proxy (используется в Istio, service mesh)
- **Traefik** — автоматическая конфигурация из Docker/Kubernetes labels

4.1.2. Алгоритмы распределения нагрузки

1. Round Robin (по кругу)

Как работает: запросы распределяются поочередно по серверам.

Пример:

Серверы: S1, S2, S3

Запросы: R1→S1, R2→S2, R3→S3, R4→S1, R5→S2, R6→S3...

Преимущества:

- Простота реализации
- Равномерное распределение запросов

Недостатки:

- Не учитывает загрузку серверов (если S1 обрабатывает тяжелые запросы, он перегружен)
- Не учитывает мощность серверов (S1 может быть слабее S2)

Когда использовать: все серверы одинаковые, запросы примерно одинаковой сложности.

2. Weighted Round Robin (взвешенный)

Как работает: серверам присваиваются веса, более мощные получают больше запросов.

Пример:

Серверы: S1 (вес 1), S2 (вес 2), S3 (вес 1)

Запросы: R1→S2, R2→S2, R3→S1, R4→S3, R5→S2, R6→S2, R7→S1...

↑ S2 получает в 2 раза больше запросов

Когда использовать: серверы разной мощности (например, 2 мощных + 1 слабый для failover).

3. Least Connections (минимум соединений)

Как работает: новый запрос отправляется на сервер с наименьшим числом активных соединений.

Пример:

Состояние:

- S1: 5 активных соединений
- S2: 3 активных соединения
- S3: 7 активных соединений

Новый запрос → S2 (минимум соединений)

Преимущества:

- Учитывает реальную загрузку серверов
- Подходит для long-lived connections (WebSocket, gRPC streaming)

Недостатки:

- Прокси должен отслеживать состояние соединений (stateful)
- Чуть сложнее, чем Round Robin

Когда использовать: запросы разной длительности, WebSocket, streaming.

4. IP Hash (хеширование по IP)

Как работает: IP клиента хешируется, запросы от одного IP всегда идут на один сервер.

Пример:

$\text{hash}(\text{IP клиента}) \% \text{количество_серверов} = \text{номер сервера}$

Клиент 192.168.1.10 → hash → 2 → всегда S2

Клиент 192.168.1.20 → hash → 1 → всегда S1

Преимущества:

- Sticky sessions (сессия пользователя сохраняется на одном сервере)
- Не нужен внешний session store

Недостатки:

- Неравномерное распределение, если мало уникальных IP
- При добавлении/удалении сервера — перераспределение клиентов

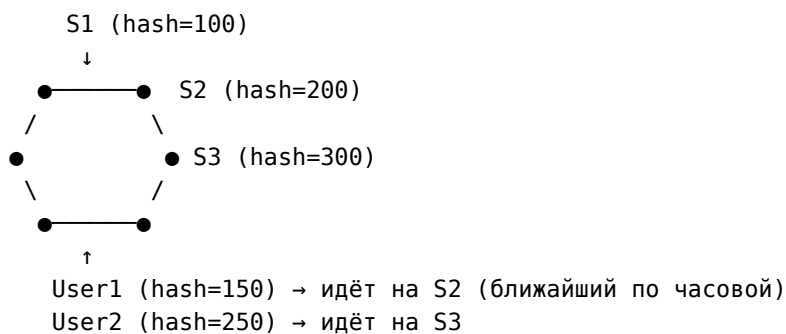
Когда использовать: нужны sticky sessions, нет централизованного session store.

5. Consistent Hashing (консистентное хеширование)

Как работает: серверы и ключи (например, user ID) размещаются на виртуальном кольце, ключ идет на ближайший сервер по часовой стрелке.

Пример:

Кольцо хеш-значений $[0 \dots 2^{32}-1]$:



Преимущества:

- Минимальное перераспределение при добавлении/удалении сервера
 - Только $1/N$ ключей перераспределяются (N — число серверов)
- Используется в распределенных кешах (Redis Cluster, Memcached)

Недостатки:

- Сложнее реализация
- Может быть неравномерное распределение (решается виртуальными узлами)

Когда использовать:

- Распределенные кеши (Redis, Memcached)
- Sharding баз данных
- CDN (routing к ближайшему серверу)

Сравнение алгоритмов балансировки:

Алгоритм	Stateful	Sticky Sessions	Масштабируемость	Когда использовать
Round Robin	Нет	Нет	Отлично (простое добавление серверов)	Stateless API, одинаковые серверы
Weighted RR	Нет	Нет	Отлично	Серверы разной мощности
Least Connections	Да (счётчики)	Нет	Хорошо	Long-lived connections, WebSocket
IP Hash	Нет	Да	Среднее (перераспределение при scaling)	Sticky sessions без внешнего store
Consistent Hashing	Нет	Да	Отлично (минимум перераспределения)	Распределенные кеши, sharding БД

4.1.3. Ограничение частоты запросов (Rate Limiting)

Зачем нужно:

- Защита от DDoS и злоупотреблений
- Контроль использования API (бесплатный план: 100 req/час, платный: 10K req/час)
- Предотвращение перегрузки backend

1. Token Bucket (ведро токенов)

Как работает:

- Ведро вмещает В токенов (burst capacity)
- Каждые 1/R секунд добавляется 1 токен (R — rate, токенов в секунду)
- Запрос тратит 1 токен; если токенов нет → отклоняется (429 Too Many Requests)

Пример: лимит 10 req/s, burst 20

Начальное состояние: 20 токенов (ведро полное)

t=0.0s: 15 запросов → токены: 20 - 15 = 5

t=0.5s: пополнение 5 токенов (10 req/s × 0.5s) → токены: 5 + 5 = 10

t=1.0s: 3 запроса → токены: 10 - 3 = 7

t=1.5s: пополнение 5 токенов → токены: 7 + 5 = 12

t=2.0s: 25 запросов → доступно только 12 → принято 12, отклонено 13 □

Преимущества:

- Разрешает кратковременные всплески (burst)
- Простая реализация

Недостатки:

- Если клиент долго не использовал API, накапливается полное ведро → может “выстрелить” всем лимитом сразу

Реализация (псевдокод):

```
type TokenBucket struct {  
    tokens float64 // текущее число токенов
```

```

    capacity int        // максимум токенов
    rate     float64    // токенов в секунду
    lastRefill time.Time
}

func (b *TokenBucket) Allow() bool {
    now := time.Now()
    elapsed := now.Sub(b.lastRefill).Seconds()

    // Пополняем токены
    b.tokens = min(b.capacity, b.tokens + elapsed * b.rate)
    b.lastRefill = now

    if b.tokens >= 1.0 {
        b.tokens -= 1.0
        return true // Разрешено
    }
    return false // Отклонено
}

```

2. Sliding Window Log (скользящее окно с логом)

Как работает:

- Храним timestamp каждого запроса в окне (например, последний час)
- Для нового запроса удаляем старые timestamps (>1 час назад)
- Если запросов в окне < лимита → разрешаем

Пример: лимит 5 req/час

Текущее время: 14:30

Лог запросов: [13:45, 13:50, 14:10, 14:25, 14:28]

Новый запрос в 14:30:

- Удаляем 13:45, 13:50 (старше часа)
- Остается: [14:10, 14:25, 14:28]
- $3 < 5$ → разрешаем, добавляем 14:30
- Лог: [14:10, 14:25, 14:28, 14:30]

Преимущества:

- Точный лимит (ровно N запросов в окне)

Недостатки:

- Требуется хранить все timestamps → $O(N)$ памяти
- Дорого для высокого лимита (100K req/час)

3. Sliding Window Counter (скользящее окно со счетчиками)

Как работает:

- Делим время на фиксированные окна (например, по 1 минуте)
- Для текущего момента вычисляем взвешенную сумму текущего и предыдущего окна

Пример: лимит 10 req/мин

Окна по 1 минуте:

14:29:00-14:30:00 (прошое): 8 запросов
 14:30:00-14:31:00 (текущее): 4 запроса

Запрос в 14:30:30 (50% в текущем окне):

Оценка = $8 \times (1 - 0.5) + 4 = 4 + 4 = 8$ запросов
 $8 < 10$ → разрешаем

Преимущества:

- Компактно: храним только 2 счетчика

- Приблизительно точный лимит

Недостатки:

- Небольшая погрешность на границах окон

Популярные реализации:

- **Redis** — INCR + EXPIRE для счетчиков
- **nginx** — limit_req_zone (Token Bucket)
- **Kong / API Gateway** — встроенные плагины rate limiting

4.2. Лекция 15: Безопасность и управление доступом

После решения проблемы распределения нагрузки переходим к **безопасности** — критическому аспекту любой продакшен-системы.

Когда ваше приложение выходит в интернет, возникают вопросы: **кто может получить доступ к данным? Как проверить, что пользователь — это действительно он? Как контролировать, что он может делать?** Без механизмов безопасности любой пользователь может удалить чужие данные, получить доступ к приватной информации или вообще подменить свою личность.

Эта лекция охватывает три уровня защиты:

1. **Аутентификация и авторизация** (RBAC, ABAC) — проверка личности и прав доступа
2. **Токены и федеративная аутентификация** (JWT, OAuth 2.0) — как передавать права между системами
3. **Single Sign-On (SSO)** — как пользователи логинятся один раз для доступа ко многим сервисам (SAML, OpenID Connect)

В больших организациях (Google, Microsoft) сотрудники не вводят пароль для каждого внутреннего сервиса — они логинятся через корпоративный SSO. Социальные сети используют OAuth, чтобы сторонние приложения могли получить доступ к вашим данным без передачи пароля. Эта лекция объясняет, как всё это работает.

4.2.1. Аутентификация vs Авторизация

Аутентификация (Authentication): Кто вы?

- Проверка личности пользователя
- Методы: пароль, OAuth, биометрия, 2FA

Авторизация (Authorization): Что вам разрешено делать?

- Проверка прав доступа
- Методы: RBAC, ABAC, ACL

Пример:

1. Аутентификация: пользователь `alice@company.com` вводит пароль → система проверяет → выдаёт токен
2. Авторизация: `alice` запрашивает `DELETE /api/users/123`
 - система проверяет: у `alice` есть роль `admin`?
 - если да → разрешить, иначе → `403 Forbidden`

4.2.2. RBAC (Role-Based Access Control)

Основная идея: права доступа назначаются **ролям**, пользователи получают роли.

Структура:

Пользователи → Роли → Права (Permissions)

Пример:

```
Alice → Role: Editor → Permissions: [read_posts, write_posts, delete_own_posts]
Bob   → Role: Admin  → Permissions: [read_posts, write_posts, delete_any_posts, manage_users]
Carol → Role: Viewer → Permissions: [read_posts]
```

Пример: система управления контентом

Роли и права:

```
type Permission string

const (
    ReadPosts      Permission = "posts:read"
    WritePosts     Permission = "posts:write"
    DeleteOwnPosts Permission = "posts:delete:own"
    DeleteAnyPosts Permission = "posts:delete:any"
    ManageUsers     Permission = "users:manage"
)
```

```

type Role struct {
    Name      string
    Permissions []Permission
}

var roles = map[string]Role{
    "viewer": {
        Name:      "Viewer",
        Permissions: []Permission{ReadPosts},
    },
    "editor": {
        Name:      "Editor",
        Permissions: []Permission{ReadPosts, WritePosts, DeleteOwnPosts},
    },
    "admin": {
        Name:      "Admin",
        Permissions: []Permission{ReadPosts, WritePosts, DeleteOwnPosts, DeleteAnyPosts,
ManageUsers},
    },
}

```

Проверка доступа:

```

func (u *User) HasPermission(perm Permission) bool {
    role := roles[u.Role]
    for _, p := range role.Permissions {
        if p == perm {
            return true
        }
    }
    return false
}

```

// Пример использования

```

if !user.HasPermission(DeleteAnyPosts) {
    return errors.New("403 Forbidden")
}

```

Преимущества:

- Простота управления (меняем права роли → изменения для всех пользователей)
- Легко понять структуру прав

Недостатки:

- Не поддерживает динамические правила (например, “редактировать только свои посты”)
- Взрыв ролей при сложных требованиях (нужна отдельная роль для каждой комбинации прав)

Где используется: большинство веб-приложений, CMS, корпоративные системы

4.2.3. ABAC (Attribute-Based Access Control)

Основная идея: доступ определяется **атрибутами** субъекта, ресурса и контекста.

Структура:

Субъект (User) + Ресурс (Resource) + Действие (Action) + Контекст (Context)
→ Политика (Policy) → Решение (Allow/Deny)

Пример: политики доступа

Атрибуты:

```

{
    "subject": {
        "id": "alice",
        "department": "engineering",

```

```
    "role": "senior",
    "clearance_level": 3
  },
  "resource": {
    "type": "document",
    "id": "doc-123",
    "classification": "confidential",
    "owner": "alice",
    "department": "engineering"
  },
  "action": "edit",
  "context": {
    "time": "2025-01-02T14:30:00Z",
    "ip": "192.168.1.10",
    "mfa_verified": true
  }
}
```

Политики (примеры):

Политика 1: Пользователь может редактировать свои документы

```
IF subject.id == resource.owner
AND action == "edit"
THEN Allow
```

Политика 2: Старшие сотрудники могут читать конфиденциальные документы своего департамента

```
IF subject.role == "senior"
AND subject.department == resource.department
AND resource.classification == "confidential"
AND action == "read"
THEN Allow
```

Политика 3: Доступ к секретным документам только в рабочее время с MFA

```
IF resource.classification == "secret"
AND subject.clearance_level >= 3
AND context.time BETWEEN 09:00 AND 18:00
AND context.mfa_verified == true
THEN Allow
ELSE Deny
```

Преимущества:

- Гибкость: можно выразить сложные правила
- Динамические проверки (время, IP, MFA)
- Меньше “взрыва” ролей

Недостатки:

- Сложнее реализация и отладка
- Производительность (нужно вычислять политики на каждый запрос)

Где используется:

- Банки и финансы (сложные правила доступа к данным)
- Государственные системы (классификация документов)
- Healthcare (HIPAA compliance, доступ по атрибутам пациента и врача)

Сравнение RBAC и ABAC:

Характеристика	RBAC	ABAC
Основа	Роли	Атрибуты (субъект, ресурс, контекст)
Сложность	Простая	Сложная
Гибкость	Низкая	Высокая

Характеристика	RBAC	ABAC
Динамические правила	Нет	Да (время, IP, MFA и т.д.)
Производительность	Быстро (проверка в памяти)	Медленнее (вычисление политик)
Когда использовать	Простые иерархии, CMS, админки	Сложные правила, финансы, healthcare

4.2.4. JWT и OAuth 2.0

JSON Web Token (JWT):

Что это: токен, содержащий закодированные claims (утверждения) о пользователе, подписанный сервером.

Структура:

Header.Payload.Signature

Пример:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpFSAwNlIiwicm9sZSI6ImVkaXRvciIsImhdCI6MTUxNjIzOTYyMn0.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Декодированный Payload:

```
{
  "sub": "1234567890",    // subject (user ID)
  "name": "Alice",
  "role": "editor",
  "iat": 1516239022,      // issued at (timestamp)
  "exp": 1516242622      // expiration (timestamp)
}
```

Как работает:

1. Клиент → POST /login (username, password)
2. Сервер проверяет credentials → генерирует JWT с claims
3. Сервер → возвращает JWT клиенту
4. Клиент сохраняет JWT (localStorage, cookie)
5. Для каждого запроса: клиент отправляет JWT в заголовке Authorization: Bearer <token>
6. Сервер проверяет подпись JWT → извлекает claims → проверяет права

Преимущества:

- Stateless (сервер не хранит сессии, все данные в токене)
- Масштабируемость (любой сервер может проверить JWT)

Недостатки:

- Нельзя отозвать токен до истечения exp (нужен blacklist)
- Размер токена больше, чем session ID

OAuth 2.0 Authorization Code Flow:

Сценарий: приложение “PhotoEditor” хочет получить доступ к фотографиям пользователя в Google Drive.

Flow:

1. Пользователь в PhotoEditor нажимает "Подключить Google Drive"
2. PhotoEditor перенаправляет на Google:
GET https://accounts.google.com/o/oauth2/v2/auth?
client_id=photeditor_client_id
&redirect_uri=https://photoeditor.com/callback
&scope=drive.readonly
&response_type=code
3. Пользователь видит экран согласия Google:

"PhotoEditor хочет получить доступ к вашим файлам на Drive"
→ Пользователь нажимает "Разрешить"

4. Google перенаправляет обратно на PhotoEditor с authorization code:
GET https://photoeditor.com/callback?code=AUTH_CODE_HERE

5. PhotoEditor (backend) обменивает code на access token:
POST <https://oauth2.googleapis.com/token>
{
 "code": "AUTH_CODE_HERE",
 "client_id": "...",
 "client_secret": "...", // секрет, известный только backend
 "redirect_uri": "...",
 "grant_type": "authorization_code"
}

6. Google возвращает access token:
{
 "access_token": "ya29.a0AfH6SMC...",
 "refresh_token": "1//0gX...",
 "expires_in": 3600,
 "token_type": "Bearer"
}

7. PhotoEditor использует access token для запросов к Google Drive API:
GET <https://www.googleapis.com/drive/v3/files>
Authorization: Bearer ya29.a0AfH6SMC...

Роли в OAuth 2.0:

- **Resource Owner** — пользователь (владелец фотографий)
- **Client** — приложение PhotoEditor
- **Authorization Server** — Google (выдаёт токены)
- **Resource Server** — Google Drive API (хранит фотографии)

4.2.5. Single Sign-On (SSO)

Что это: пользователь логинится один раз и получает доступ ко всем приложениям компании.

Пример:

Сотрудник входит в корпоративный портал (portal.company.com)

→ Автоматически получает доступ к:

- Email (mail.company.com)
- CRM (crm.company.com)
- Jira (jira.company.com)
- Confluence (wiki.company.com)

Протоколы SSO:

1. SAML 2.0 (Security Assertion Markup Language)

Enterprise-стандарт, работает через XML-сообщения.

Flow:

1. Пользователь → открывает CRM (Service Provider)
2. CRM видит, что пользователь не залогинен → перенаправляет на SSO (Identity Provider)
3. SSO → показывает форму логина
4. Пользователь вводит credentials → SSO проверяет
5. SSO → создаёт SAML Assertion (XML с данными пользователя) и подписывает
6. SSO → перенаправляет обратно на CRM с SAML Assertion
7. CRM → проверяет подпись, извлекает данные пользователя → создаёт сессию

Где используется: корпоративные системы, интеграции B2B (Okta, Azure AD, Google Workspace)

2. OAuth 2.0 / OpenID Connect (OIDC)

Более современный, работает с JSON Web Tokens.

OpenID Connect = OAuth 2.0 + id_token (JWT с данными пользователя)

Flow:

1. Пользователь → открывает App
2. App → перенаправляет на Auth0 (OIDC Provider):
GET /authorize?client_id=...&scope=openid profile email&response_type=code
3. Auth0 → показывает логин (или использует существующую сессию SSO)
4. Auth0 → перенаправляет обратно с authorization code
5. App → обменивает code на токены:
Получает: { access_token, id_token (JWT), refresh_token }
6. App → декодирует id_token:
{
 "sub": "user-123",
 "email": "alice@company.com",
 "name": "Alice",
 "picture": "https://...",
 "iss": "https://auth0.company.com"
}

Где используется: современные SaaS-приложения, мобильные приложения (Auth0, Okta, Keycloak)

Сравнение SAML vs OAuth/OIDC:

Характеристика	SAML 2.0	OAuth 2.0 / OIDC
Формат	XML	JSON (JWT)
Сложность	Высокая	Средняя
Мобильные приложения	Плохо поддерживается	Отлично
API	RESTful wrapper нужен	Нативно RESTful
Enterprise adoption	Высокая (legacy системы)	Растёт (новые системы)
Когда использовать	B2B интеграции, enterprise SSO	SaaS, мобильные приложения, API

4.3. Лекция 16: Observability: логирование, мониторинг и планирование задач

После того как система защищена и запущена в продакшене, возникает вопрос: **как понять, что происходит внутри?**

Когда приложение работает на вашем компьютере, вы видите вывод в консоль, можете поставить breakpoint и посмотреть переменные. Но в продакшене приложение работает на сервере, обрабатывает тысячи запросов в секунду, и вы не можете просто «остановить и посмотреть». Более того, если пользователь жалуется «у меня не работает», вам нужно понять, **что именно** пошло не так, **когда** это произошло и **почему**.

Observability (наблюдаемость) — это способность понять внутреннее состояние системы по её внешним выходным данным. Три столпа observability — это **логи** (что произошло), **метрики** (сколько и как быстро) и **трейсы** (путь запроса через систему).

Эта лекция также охватывает **планирование задач** (task scheduling) — механизмы запуска фоновых задач и построения data pipelines. В продакшене многие операции выполняются асинхронно: отправка email, обработка загруженных файлов, построение отчётов. Вместо того чтобы заставлять пользователя ждать, мы ставим задачу в очередь и обрабатываем её фоном. Celery, Airflow и Dagster решают эту проблему на разных уровнях абстракции.

4.3.1. Три столпа Observability

Логи (Logs): дискретные события (“user123 logged in at 14:30”)

Метрики (Metrics): агрегированные числовые данные (CPU usage: 65%, requests/sec: 1500)

Трейсы (Traces): путь запроса через микросервисы (request → API Gateway → Auth → Database)

4.3.2. Структурированное логирование

Проблема традиционных логов:

```
2025-01-02 14:30:15 ERROR Failed to process order: user not found
2025-01-02 14:30:16 INFO User alice@company.com logged in from 192.168.1.10
```

- Трудно парсить (нужны regex)
- Нет структуры для фильтрации
- Сложно агрегировать

Структурированные логи (JSON):

```
{
  "timestamp": "2025-01-02T14:30:15Z",
  "level": "ERROR",
  "message": "Failed to process order",
  "error": "user not found",
  "user_id": "user-123",
  "order_id": "order-456",
  "service": "order-service",
  "trace_id": "abc-def-ghi"
}

{
  "timestamp": "2025-01-02T14:30:16Z",
  "level": "INFO",
  "message": "User logged in",
  "user_email": "alice@company.com",
  "ip": "192.168.1.10",
  "service": "auth-service",
  "trace_id": "xyz-123-456"
}
```

Преимущества:

- Легко фильтровать: service="order-service" AND level="ERROR"

- Агрегация: сколько ошибок по сервисам?
- Корреляция: `trace_id` связывает логи одного запроса

Популярные библиотеки:

- **Go:** zap, logrus
- **Python:** structlog, python-json-logger
- **Node.js:** winston, pino

Централизованный сбор логов:

- **ELK Stack:** Elasticsearch (хранение) + Logstash (обработка) + Kibana (визуализация)
- **Loki** (от Grafana Labs): как Prometheus, но для логов
- **Splunk:** enterprise-решение

4.3.3. Prometheus: сбор метрик

Что это: time-series база данных для метрик. Scraping-based (Prometheus сам забирает метрики из приложений).

Типы метрик:

1. **Counter** — монотонно растущий счётчик (сбрасывается при рестарте)

Пример: `http_requests_total` (общее число запросов)
Использование: вычисление `rate` (запросов в секунду)

2. **Gauge** — текущее значение (может расти и падать)

Пример: `memory_usage_bytes`, `active_connections`

3. **Histogram** — распределение значений по бакетам

Пример: `http_request_duration_seconds` (сколько запросов заняло `<0.1s`, `<0.5s`, `<1s`, `<5s`)
Использование: вычисление `percentiles` (`p50`, `p95`, `p99`)

4. **Summary** — похож на Histogram, но `percentiles` вычисляются на клиенте

Пример: `request_duration_summary{quantile="0.95"}`

Пример инструментирования (Go):

```
import "github.com/prometheus/client_golang/prometheus"

var (
    httpRequestsTotal = prometheus.NewCounterVec(
        prometheus.CounterOpts{
            Name: "http_requests_total",
            Help: "Total number of HTTP requests",
        },
        []string{"method", "endpoint", "status"},
    )

    httpRequestDuration = prometheus.NewHistogramVec(
        prometheus.HistogramOpts{
            Name:    "http_request_duration_seconds",
            Help:    "HTTP request duration in seconds",
            Buckets: []float64{0.01, 0.05, 0.1, 0.5, 1.0, 5.0},
        },
        []string{"method", "endpoint"},
    )
)

func init() {
    prometheus.MustRegister(httpRequestsTotal)
    prometheus.MustRegister(httpRequestDuration)
}
```

```

func handleRequest(w http.ResponseWriter, r *http.Request) {
    start := time.Now()

    // ... обработка запроса ...

    duration := time.Since(start).Seconds()
    status := 200

    httpRequestsTotal.WithLabelValues(r.Method, r.URL.Path, fmt.Sprint(status)).Inc()
    httpRequestDuration.WithLabelValues(r.Method, r.URL.Path).Observe(duration)
}

```

Prometheus scraping:

Приложение expose /metrics endpoint:

```

# HELP http_requests_total Total number of HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",endpoint="/api/users",status="200"} 15234
http_requests_total{method="POST",endpoint="/api/users",status="201"} 482
http_requests_total{method="GET",endpoint="/api/users",status="404"} 12

# HELP http_request_duration_seconds HTTP request duration in seconds
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{method="GET",endpoint="/api/users",le="0.01"} 8521
http_request_duration_seconds_bucket{method="GET",endpoint="/api/users",le="0.05"} 14102
http_request_duration_seconds_bucket{method="GET",endpoint="/api/users",le="0.1"} 15000
http_request_duration_seconds_bucket{method="GET",endpoint="/api/users",le="0.5"} 15200
http_request_duration_seconds_bucket{method="GET",endpoint="/api/users",le="+Inf"} 15234
http_request_duration_seconds_sum{method="GET",endpoint="/api/users"} 652.3
http_request_duration_seconds_count{method="GET",endpoint="/api/users"} 15234

```

Prometheus config (prometheus.yml):

```

scrape_configs:
- job_name: 'my-app'
  scrape_interval: 15s
  static_configs:
  - targets: ['localhost:8080']

```

PromQL (Prometheus Query Language) — примеры:

```

# Текущее число активных соединений
active_connections

```

```

# Запросов в секунду за последние 5 минут
rate(http_requests_total[5m])

```

```

# Запросов в секунду по эндпоинтам
sum(rate(http_requests_total[5m])) by (endpoint)

```

```

# P95 latency (95% запросов быстрее этого значения)
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))

```

```

# Процент ошибок (5xx)
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))

```

```

# Alerting: если error rate > 5%
sum(rate(http_requests_total{status=~"5.."}[5m]))
/

```

```
sum(rate(http_requests_total[5m]))
  > 0.05
```

4.3.4. Grafana: визуализация и дашборды

Что это: платформа для визуализации метрик из Prometheus, Loki, Elasticsearch и др.

Пример дашборда для веб-сервиса:

Панели:

1. **QPS (Queries Per Second)** — graph

```
sum(rate(http_requests_total[1m]))
```

2. **Latency (P50, P95, P99)** — graph

```
histogram_quantile(0.50, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

3. **Error Rate** — single stat + graph

```
sum(rate(http_requests_total{status=~"5.."}[5m]))
  / sum(rate(http_requests_total[5m])) * 100
```

4. **Top endpoints by QPS** — table

```
topk(10, sum(rate(http_requests_total[5m])) by (endpoint))
```

5. **Memory usage** — graph

```
process_resident_memory_bytes
```

Alerting в Grafana:

Настройка алерта “High Error Rate”:

```
WHEN avg() OF query(A, 5m, now) IS ABOVE 0.05
SEND TO: [Slack, PagerDuty, Email]
```

4.3.5. Distributed Tracing (Jaeger, OpenTelemetry)

Проблема: запрос проходит через 5 микросервисов, где узкое место?

Решение: трейсинг показывает путь и время каждого шага.

Пример запроса:

```
User → API Gateway → Auth Service → User Service → Database
                        ↘ Cache
```

Trace:

Trace ID: abc-123-xyz
Total duration: 245ms

```
Span 1: API Gateway (245ms total)
├─ Span 2: Auth Service (50ms)
│   └─ Span 3: Cache GET (5ms)
└─ Span 4: User Service (180ms)
    └─ Span 5: Database SELECT (170ms) ← bottleneck!
```

Инструментирование (OpenTelemetry, Go):

```
import "go.opentelemetry.io/otel"

func handleRequest(ctx context.Context, userID string) error {
    ctx, span := otel.Tracer("api-gateway").Start(ctx, "handleRequest")
    defer span.End()
```

```

// Вызов auth service
if err := checkAuth(ctx, userID); err != nil {
    span.RecordError(err)
    return err
}

// Вызов user service
user, err := getUserData(ctx, userID)
if err != nil {
    span.RecordError(err)
    return err
}

span.SetAttributes(attribute.String("user.email", user.Email))
return nil
}

func getUserData(ctx context.Context, userID string) (*User, error) {
    ctx, span := otel.Tracer("user-service").Start(ctx, "getUserData")
    defer span.End()

    // Запрос к БД
    user, err := db.Query(ctx, "SELECT * FROM users WHERE id = ?", userID)
    // ...
    return user, err
}

```

Trace распространяется через HTTP headers:

```

traceparent: 00-abc123xyz-def456-01
| | | |
| | | └─ flags (1 байт)
| | └─ span ID (8 байт = 16 hex символов)
| └─ trace ID (16 байт = 32 hex символа)
└─ version (1 байт = 2 hex символа)

```

Популярные решения:

- **Jaeger** — open-source от Uber
- **Zipkin** — open-source от Twitter
- **OpenTelemetry** — стандарт для инструментирования (заменяет OpenTracing + OpenCensus)
- **Datadog APM, New Relic** — SaaS-решения

4.3.6. Планировщики задач

1. Celery (Python task queue)

Что это: асинхронная очередь задач для Python.

Архитектура:

Producer → Broker (Redis/RabbitMQ) → Worker → Result Backend

Пример:

```

from celery import Celery

app = Celery('tasks', broker='redis://localhost:6379/0')

@app.task
def send_email(user_email, subject, body):
    # Отправка email (медленная операция)
    smtp.send(user_email, subject, body)
    return f"Email sent to {user_email}"

```

Использование

```
# Синхронно (блокирует):
# send_email("alice@example.com", "Hello", "Welcome!")

# Асинхронно (возвращает сразу):
result = send_email.delay("alice@example.com", "Hello", "Welcome!")
# result.id → task ID для отслеживания
```

Use cases:

- Отправка email/SMS после регистрации
- Генерация отчётов (PDF, Excel)
- Обработка загруженных изображений (resize, watermark)
- Очистка старых данных (cron-задачи)

Periodic tasks (cron):

```
from celery.schedules import crontab

app.conf.beat_schedule = {
    'cleanup-old-sessions': {
        'task': 'tasks.cleanup_sessions',
        'schedule': crontab(hour=2, minute=0), # каждый день в 02:00
    },
}
```

2. Airflow (workflow orchestration)

Что это: платформа для создания, планирования и мониторинга **DAG** (Directed Acyclic Graphs) — data pipelines.

Пример DAG:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

dag = DAG(
    'user_analytics_pipeline',
    start_date=datetime(2025, 1, 1),
    schedule_interval='@daily', # запускать каждый день
)

def extract_data():
    # Скачать данные из S3
    pass

def transform_data():
    # Обработать данные (Spark, Pandas)
    pass

def load_to_warehouse():
    # Загрузить в ClickHouse
    pass

extract_task = PythonOperator(task_id='extract', python_callable=extract_data, dag=dag)
transform_task = PythonOperator(task_id='transform', python_callable=transform_data, dag=dag)
load_task = PythonOperator(task_id='load', python_callable=load_to_warehouse, dag=dag)

extract_task >> transform_task >> load_task # зависимости
```

Визуализация:

```
[Extract] → [Transform] → [Load]
  ↓           ↓           ↓
Success    Success    Success
```

Use cases:

- ETL/ELT pipelines (данные из источников → обработка → хранилище)
- ML training pipelines (подготовка данных → обучение → деплой модели)
- Бэкапы и синхронизация данных

3. Dagster (data orchestrator)

Что это: современная альтернатива Airflow с фокусом на **software-defined assets**.

Отличие от Airflow:

- **Airflow:** описываем **задачи** (tasks)
- **Dagster:** описываем **ресурсы** (assets) и их зависимости

Пример:

```
from dagster import asset

@asset
def raw_user_events():
    # Скачать события из Kafka
    return pd.read_csv("s3://bucket/events.csv")

@asset
def cleaned_events(raw_user_events):
    # Зависит от raw_user_events
    return raw_user_events.dropna()

@asset
def user_metrics(cleaned_events):
    # Вычислить метрики
    return cleaned_events.groupby('user_id').agg({'revenue': 'sum'})
```

Преимущества:

- Типизация данных между шагами
- Автоматическое кеширование (не пересчитываем, если данные не изменились)
- Встроенный data lineage (откуда данные?)

Use cases:

- Современные data pipelines
- ML pipelines с версионированием данных
- Real-time + batch обработка

Сравнение планировщиков:

Характеристика	Celery	Airflow	Dagster
Тип	Task queue	Workflow orchestrator	Data orchestrator
Фокус	Асинхронные задачи	ETL/batch pipelines	Data assets + lineage
Сложность	Низкая	Средняя	Средняя
UI	Flower (опционально)	Встроенный	Встроенный (Dagit)
Retry/backoff	Да	Да	Да
Когда использовать	Фоновые задачи в веб-приложениях	Data pipelines, ETL	Современные data pipelines, ML

5. Практические задания

Теперь, когда мы изучили архитектурные паттерны, структуры хранения данных, методы поиска и операционные компоненты, пора применить знания на практике. Модуль содержит пять комплексных практических заданий, каждое из которых охватывает несколько лекций.

Цель практик: научиться принимать архитектурные решения с обоснованием. В реальных проектах нет “правильного ответа” — есть компромиссы. Важно уметь объяснить **почему** вы выбрали PostgreSQL, а не MongoDB, **почему** используете REST, а не gRPC, **какие метрики** подтверждают, что система выдержит нагрузку.

5.1.1. Варианты

1. **Интернет-магазин** (товары, корзина, заказы, пользователи, доставка)
2. **Социальная сеть** (посты, лайки, комментарии, подписки, уведомления)
3. **Сервис бронирования отелей** (отели, комнаты, бронирования, оплата, отзывы)
4. **Банковское приложение** (счета, транзакции, переводы, история, платежи)
5. **Сервис доставки еды** (рестораны, меню, заказы, курьеры, доставка)
6. **Такси/каршеринг** (водители, поездки, тарифы, местоположение, платежи)
7. **Онлайн-кинотеатр** (фильмы, подписки, просмотры, рекомендации, платежи)
8. **Образовательная платформа** (курсы, лекции, студенты, прогресс, сертификаты)
9. **Система управления проектами** (проекты, задачи, пользователи, комментарии, файлы)
10. **Маркетплейс фриланса** (заказчики, исполнители, проекты, предложения, платежи)
11. **Медицинская система** (пациенты, врачи, приемы, диагнозы, рецепты)
12. **Система бронирования билетов** (события, места, билеты, заказы, возвраты)

5.1.2. Практика 1: Проектирование API и взаимодействия (Лекции 1-4: компоненты, слои, SOLID, протоколы)

5.1.3. Предисловие

В лекциях рассматриваются технические плюсы и минусы различных архитектурных подходов: базовые определения и “здравый смысл” в выборе паттернов — когда их стоит применять, а когда нет.

В этой практике принцип простой: сначала проектируем, потом кодируем. Цель — выбрать осмысленный бизнес-воркфлоу по варианту и заранее спроектировать API, слои и use cases.

Почему в практиках мы отталкиваемся от Clean Architecture

Строгая архитектура даёт два ключевых свойства: **единообразный путь данных** и **общий словарь**.
Единообразный путь данных означает, что любой запрос проходит одну и ту же цепочку слоёв. Без этого каждый новый endpoint изобретает собственную структуру — особенно заметно при работе с LLM:

Промпт	Результат
Добавь endpoint для получения пользователя по ID	GET /users/:id handler → UserRepository → ORM
Добавь endpoint для получения списка задач	GET /tasks handler → ORM (напрямую)
Добавь endpoint для получения статистики пользователя	GET /users/:id/stats handler → UserRepository → StatsManager → ORM

Три endpoint’а — три разных решения. При отладке непонятно, где искать логику. С ростом кодовой базы хаос усугубляется.

Общий словарь (entity, use case, adapter, repository) решает смежную проблему: в больших командах одни и те же роли называют service, observer, manager. Единые термины убирают лишний “перевод” между командами и упрощают ревью и онбординг.

При этом важно не превращать терминологию в догму. Паттерны вроде Strategy, Facade, Observer никуда не исчезают — они живут внутри конкретных компонентов. Строгая архитектура фиксирует внешние границы и путь данных, а не запрещает локальные решения.

5.1.4. Задание

1. Выберите продукт и воркфлоу.

В реальной разработке бизнес описывает путь пользователя: последовательность шагов, которые человек проходит для достижения цели. Именно из этих шагов рождаются endpoints, use cases и слои приложения. Опишите сценарий пользователя на 2-3 шага бизнес-логики (не CRUD).

Примеры воркфлоу по вариантам:

- **Интернет-магазин:** пользователь открывает каталог → выбирает категорию → получает список товаров с фильтрами (цена, рейтинг) → добавляет товар в корзину → система проверяет наличие на складе → если товара нет, предлагает аналоги → оформляет заказ → система резервирует товар и создаёт заказ со статусом “ожидает оплаты”
- **Сервис доставки еды:** пользователь указывает адрес → система показывает рестораны, которые доставляют в эту зону → пользователь выбирает ресторан и блюда → система считает сумму с доставкой → пользователь подтверждает → система ищет свободного курьера и назначает его
- **Такси/каршеринг:** пользователь указывает точку А и Б → система рассчитывает маршрут и стоимость по тарифу → пользователь подтверждает → система ищет ближайшего свободного водителя → назначает водителя → отправляет пользователю ETA
- **Образовательная платформа:** студент открывает курс → видит модули и свой прогресс → открывает урок → проходит тест → система проверяет ответы → если пройден порог, открывает следующий модуль → если все модули пройдены, генерирует сертификат

- **Банковское приложение:** пользователь выбирает “перевод” → вводит номер счёта получателя → система проверяет существование счёта → вводит сумму → система проверяет достаточность средств → списывает с одного счёта, зачисляет на другой → создаёт две записи в истории транзакций
- **Медицинская система:** пациент выбирает специальность врача → система показывает врачей с ближайшими свободными слотами → пациент выбирает время → система проверяет, что слот не занят → создаёт запись → отправляет подтверждение
- **Система бронирования билетов:** пользователь выбирает событие → видит схему зала с доступными местами → выбирает места → система временно резервирует их (5 минут) → пользователь подтверждает → система переводит бронь в “оплачен” и генерирует билет
- **Социальная сеть:** пользователь открывает ленту → система собирает посты от подписок, сортирует по времени → пользователь ставит лайк → счётчик обновляется → пишет комментарий → автор поста получает уведомление

В каждом сценарии есть проверки, условия, несколько сущностей, которые меняют состояние вместе. Именно это и нужно спроектировать.

Внешние сервисы. Некоторые шаги воркфлоу требуют знаний, которых пока нет (геопоиск ближайшего водителя, генерация PDF, отправка email, рекомендации). Это нормально — разрешается сказать: “эту операцию выполняет отдельный внешний сервис” и описать его контракт. Например: `NearestDriverService.Find(location, radius) → Driver` или `NotificationService.Send(userId, message)`. В Практике 2 этот контракт реализуется заглушкой (возвращает фиксированный результат), а в дальнейших практиках, возможно, будет реализован по-настоящему. Ограничение: основной воркфлоу должен жить в вашем приложении, наружу можно вынести 1-2 операции.

2. Выберите интерфейс и протокол.

- Варианты: REST (OpenAPI), GraphQL, gRPC, fullstack (HTML endpoints), комбинации.
- Менее стандартные подходы:
 - **Fullstack (сервер отдаёт HTML):** Phoenix, Django, htmx
 - **Внутренние микросервисы:** gRPC-вызов к отдельному сервису, который выполняет тяжёлую работу
- Кратко обоснуйте выбор. Примеры обоснований:
 - **Система бронирования билетов → REST.** Мы продаём API партнёрам (агрегаторы, виджеты на сайтах площадок). Клиенты — внешние, на разных языках и платформах. REST + OpenAPI spec даёт им документацию и кодогенерацию клиента из коробки.
 - **Банковский сервис переводов → gRPC.** Сервис работает внутри периметра компании, его вызывают другие микросервисы (мобильный BFF, процессинг). Важна типобезопасность контрактов (.proto) и скорость бинарной сериализации. Внешних потребителей нет.
 - **Интернет-магазин → fullstack (HTML).** Страницы каталога и карточки товаров должны индексироваться поисковиками (SEO). SSR решает проблему — сервер отдаёт готовый HTML, поисковой бот видит контент сразу.
 - **Мобильное приложение соцсети → GraphQL.** Экраны сильно различаются: лента показывает пост + автор + счётчик лайков, профиль — аватар + подписчики + последние посты. REST потребовал бы десятки endpoint'ов или возвращал бы лишние поля. GraphQL позволяет каждому экрану запросить ровно те данные, которые нужны.
- Если не хочется тратить время на выбор — REST всегда безопасный вариант. В этом и ценность Clean Architecture: бизнес-логика живёт в use cases и не зависит от интерфейса. Если позже окажется, что нужен gRPC или GraphQL — меняется только слой адаптеров, а domain и use cases остаются без изменений.

3. Определитесь со стеком.

- Выберите язык, веб-фреймворк и подход к DI. Код в этой практике писать не нужно — но к Практике 2 вы должны чётко понимать, на чём будете реализовывать.

Популярные экосистемы для ориентира:

Язык	Веб-фреймворк	DI	Сильные стороны
Go	chi, echo, fiber oapi-codegen (OpenAPI yml first) нативный gRPC	wire (кодогенерация) fx (Uber) ручной DI в main.go	gRPC из коробки, OpenAPI yml first, sqlc для типобезопасного SQL
Python	FastAPI (OpenAPI из коробки) Django REST Framework Flask	FastAPI Depends (встроен) dependency-injector	Быстрый старт, автодокументация API в FastAPI
Java / Kotlin	Spring Boot Quarkus, Micronaut Ktor	Spring IoC (встроен) Dagger 2 (кодогенерация) Koin (Kotlin)	Зрелая экосистема, Spring покрывает всё
C#	ASP.NET Core Minimal API ASP.NET Core MVC	встроен в ASP.NET Core Autofac	DI из коробки, сильная типизация
TypeScript	Express, Fastify NestJS (DI встроен) Next.js, bun	NestJS (встроен) tsyringe, InversifyJS	Fullstack на одном языке, Next.js для SSR
Elixir	Phoenix (LiveView / JSON API)	behaviours + конфигурация	Fullstack с real-time из коробки

4. Опишите API-контракты.

- Для каждого endpoint/query/mutation: входные данные, выходные данные, ошибки.
- Формат: OpenAPI yml, .proto, GraphQL schema или структурированная таблица.

5. Опишите архитектуру приложения.

- Слои, компоненты, направление зависимостей, интерфейсы между слоями.
- На схеме **обязательно** явно различайте:
 - реализационные компоненты (класс/модуль/сервис),
 - интерфейсные компоненты (порт/контракт).
- Для зависимостей укажите тип стрелки или подпись связи:
 - depends on (компонент зависит от интерфейса),
 - implements (компонент реализует интерфейс).
- По умолчанию ожидается Clean Architecture. Допустимы другие подходы (MVC, framework-based и т.д.) — при условии, что вы аргументируете выбор и так же подробно расписываете слои, зависимости и контракты между ними.

6. Опишите use cases.

- Пошагово: проверки, обращения к репозиториям/сервисам, ошибки.

Важно: требования по авторизации в этой практике отсутствуют. Фокус — на архитектуре и бизнес-логике.

Что сдаётся:

1. Краткое описание продукта и выбранного воркфлоу.
2. Обоснование выбора протокола.
3. Выбор стека (язык, фреймворк, DI).
4. Спецификация API (endpoint/query/mutation + request/response/errors).
5. Схема слоёв, контрактов и зависимостей с явным различением интерфейсных и реализационных компонентов.
6. Описание use cases.

Теория к сдаче:

Лекция 1 — Принципы построения приложения:

- Что такое компонент? Контракт (интерфейс) компонента. Примеры компонентов разного масштаба (функция, класс, сервис)
- Что такое зависимость между компонентами? Почему неконтролируемые зависимости — проблема?
- Что такое слой? Правила межслойных зависимостей
- Продуктивность слоя: прокси-компоненты, коэффициент проксирования, когда слой не нужен
- Принципы SOLID (уметь объяснить каждый на примере)
- Dependency Injection: зачем нужен, как реализуется
- Clean Architecture: слои (domain, use cases, adapters, infrastructure), правило зависимостей (зависимости направлены внутрь)

Лекция 2 — Альтернативные архитектурные паттерны:

- Framework-based архитектура (Rails/Django): что значит “opinionated framework”, плюсы и минусы
- Модель акторов (Erlang/Akka): изолированные процессы, обмен сообщениями, “Let it Crash”, когда применяется
- Data-Oriented Design: зачем оптимизировать под кеш процессора, где применяется

Лекция 3 — Протоколы: REST / gRPC:

- REST: HTTP методы, коды ответов, JSON, URI endpoints, OpenAPI/Swagger
- gRPC: Protocol Buffers, HTTP/2, кодогенерация, стриминг
- Когда REST, когда gRPC? (публичный API vs внутренние сервисы)

Лекция 4 — Протоколы: GraphQL / WebSocket:

- GraphQL: проблема over-fetching/under-fetching, единственный endpoint, клиент описывает структуру ответа
- WebSocket: проблема polling, постоянное двустороннее соединение, применение (чаты, real-time)

Лекция 5 — Асинхронные взаимодействия: брокеры, очереди, обмен файлами:

- Зачем асинхронность: развязка сервисов, устойчивость, сглаживание пиков нагрузки
- Брокеры сообщений и очереди: Kafka/RabbitMQ, pub/sub vs очереди задач, at-least-once/at-most-once
- Событийное взаимодействие: события домена, eventual consistency, саги
- Надежность: идемпотентность, дедупликация, ретраи, DLQ
- Обмен файлами/шаринг: когда уместно, плюсы и риски, контроль версий и целостности

5.1.5. Практика 2: Реализация спроектированного приложения (Лекции 1-4: реализация, тесты, отчёт)

5.1.6. Предисловие

Во второй части реализуется проект из Практики 1. Цель — проверить качество архитектуры на рабочем коде без отвлечения на настройку реальной БД.

5.1.7. Задание

1. **Реализуйте слои из вашего проекта:** domain, use cases, adapters, infrastructure.
2. **Репозитории только in-memory** (map/slice/dict), без PostgreSQL/MySQL.
3. **Интеграционные тесты:**
 - Для каждого endpoint: успешный сценарий + логическая ошибка.
 - Минимум один тест на полный воркфлоу из нескольких шагов.
 - Сценарии “ошибка авторизации” не требуются.
4. **Подготовьте отчёт:**
 - Таблица слоёв (зависимости, ответственность, LOC).
 - Рефлексия: что изменилось по сравнению с проектом из Практики 1 и почему.

Язык: Go, Java, C#, TypeScript или Elixir/Phoenix.

Структура проекта: можно выбрать свою. Ниже только ориентир, а не жёсткий шаблон:

```
app/  
├─ domain/  
├─ usecases/  
├─ adapters/  
├─ repositories/in_memory/  
├─ infrastructure/  
└─ main.*
```

Что сдаётся:

1. Работающее приложение по проекту из Практики 1.
2. In-memory реализации всех репозиторий.
3. Интеграционные тесты (endpoint’ы + полный воркфлоу).
4. Таблица слоёв с LOC.
5. Краткая рефлексия по отличиям от проекта.

Теория к сдаче:

Вся теория из Практики 1, плюс вопросы по собственному коду:

- Покажите на своём коде: где domain, где use cases, где adapters? Покажите направление зависимостей
- Что такое инверсия зависимостей? Покажите: use case зависит от интерфейса, а не от конкретной реализации репозитория
- Если завтра нужно заменить in-memory хранилище на PostgreSQL — какие файлы меняются, какие нет? Почему?
- Что такое прокси-компонент? Есть ли в вашем приложении непродуктивные слои?
- Почему вы выбрали именно этот протокол? В каком случае выбрали бы другой?

5.1.8. Практика 3: OLTP-система и расчёт нагрузки (Лекции 5-9)

5.1.9. Введение

В реальной разработке необходимо **спроектировать схему данных** и **количественно оценить**, справится ли планируемое железо с ожидаемой нагрузкой. Для оценки производительности используется **масштабирование от эталонных бенчмарков**.

Ключевая идея: мы не можем точно предсказать производительность системы, которая ещё не написана. Но мы можем:

1. **Спроектировать схему данных** для нашей системы
2. **Найти результаты эталонных бенчмарков** для целевого железа
3. **Нормализовать** наш запрос относительно эталона (посчитать, во сколько раз он сложнее)
4. **Оценить sarasity** — понять, хватит ли железа для требуемой нагрузки

Важно: это **приблизительная оценка порядка величин**, а не попытка получить реалистичные числа. Цель — выработать инженерный подход к предварительной оценке нагрузки: понимать порядок величин и осознавать, что каждая дополнительная операция (SELECT/JOIN/UPDATE) увеличивает стоимость транзакции.

5.1.10. Задачи практики

Задача 1. Модель данных в выбранной СУБД

Выберите СУБД (PostgreSQL, MySQL, MongoDB) и спроектируйте модель данных:

- 4-6 таблиц/коллекций в формате SQL DDL или схеме NoSQL
- Расчёт размера одной записи в байтах для основных сущностей
- Оценка общего объёма данных при целевой нагрузке

Задача 2. Описание одного endpoint'a из Практики 2

Выберите один write-heavy endpoint и детально опишите:

- Название и назначение endpoint'a
- Какие таблицы читаются/пишутся
- SQL-код всех запросов внутри транзакции

Задача 3. Поиск baseline производительности

Найдите опубликованные результаты бенчмарков для конфигурации, близкой к вашему целевому железу:

- **Официальные источники:** TPC-C/TPC-E результаты на tpc.org
- **Vendor benchmarks:** документация облачных провайдеров (AWS RDS, Azure, GCP Cloud SQL)
- **Независимые тесты:** Percona, Benchmarksql, публикации на GitHub

Укажите источник, конфигурацию железа и полученный TPS — это ваш baseline.

Задача 4. Расчёт максимального RPS для вашего endpoint'a

Шаг 4.1. Нормализация нагрузки

Подсчитайте DML-операции в вашем endpoint'е и сравните с эталоном:

$$K = \frac{\text{DML в вашем endpoint}}{\text{DML в эталоне (pgbench = 5)}}$$

Шаг 4.2. Расчёт max RPS

$$\text{max RPS} = \frac{\text{baseline TPS}}{K}$$

Шаг 4.3. Проверка через CPU-time

$$\text{CPU-time на транзакцию} = \frac{N_{\text{cores}} \times 1000 \text{ мс}}{\text{TPS}}$$

Проверьте: при расчётном RPS загрузка CPU не должна превышать 100%.

Задача 5. Расчёт DAU

$$DAU = \frac{RPS \times \text{активные секунды}}{\text{действий на пользователя} \times \text{пиковый коэффициент}}$$

Параметры формулы определите самостоятельно исходя из вашего сценария:

- Активные секунды: сколько часов в сутки система под нагрузкой
- Действий на пользователя: сколько запросов генерирует один пользователь
- Пиковый коэффициент: во сколько раз пиковая нагрузка выше средней

5.1.11. Пример выполнения

Сценарий: интернет-магазин (e-commerce)

5.1.12. Задача 1. Модель данных

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,          -- 8 байт  
  email VARCHAR(255) UNIQUE NOT NULL, -- ~30 байт  
  password_hash CHAR(60) NOT NULL,   -- 60 байт (bcrypt)  
  created_at TIMESTAMP               -- 8 байт  
);  
-- Размер записи: ~130 байт (с учётом tuple header)
```

```
CREATE TABLE products (  
  id BIGSERIAL PRIMARY KEY,          -- 8 байт  
  name VARCHAR(255) NOT NULL,        -- ~40 байт  
  description TEXT,                  -- ~200 байт  
  price DECIMAL(10,2) NOT NULL,      -- 8 байт  
  stock INT NOT NULL,                -- 4 байт  
  category_id INT                    -- 4 байт  
);  
-- Размер: ~288 байт
```

```
CREATE TABLE orders (  
  id BIGSERIAL PRIMARY KEY,  
  user_id BIGINT REFERENCES users(id),  
  total DECIMAL(10,2) NOT NULL,  
  status VARCHAR(20),  
  created_at TIMESTAMP  
);  
-- Размер: ~76 байт
```

```
CREATE TABLE order_items (  
  id BIGSERIAL PRIMARY KEY,  
  order_id BIGINT REFERENCES orders(id),  
  product_id BIGINT REFERENCES products(id),  
  quantity INT,  
  price DECIMAL(10,2)  
);  
-- Размер: ~60 байт
```

```
CREATE TABLE cart_items (  
  user_id BIGINT REFERENCES users(id),  
  product_id BIGINT REFERENCES products(id),  
  quantity INT,  
  added_at TIMESTAMP,  
  PRIMARY KEY (user_id, product_id)  
);  
-- Размер: ~52 байт
```

Оценка объёма:

- 1 млн пользователей × 130 байт = 130 МБ
- 100K товаров × 288 байт = 29 МБ
- 10 млн заказов × 76 байт = 760 МБ
- 30 млн позиций × 60 байт = 1.8 ГБ

Итого: 2.7 ГБ (без индексов)

5.1.13. Задача 2. Описание endpoint’a

Endpoint: POST /api/orders — оформление заказа

Бизнес-логика:

1. Проверить наличие товаров на складе
2. Создать заказ
3. Создать позиции заказа
4. Уменьшить stock
5. Очистить корзину

SQL-код транзакции (3 товара в корзине):

```
BEGIN;  
  
SELECT id, stock FROM products WHERE id = $1;  
SELECT id, stock FROM products WHERE id = $2;  
SELECT id, stock FROM products WHERE id = $3;  
  
INSERT INTO orders (user_id, total, status, created_at)  
VALUES ($1, $2, 'pending', CURRENT_TIMESTAMP) RETURNING id;  
  
INSERT INTO order_items (order_id, product_id, quantity, price)  
VALUES ($order_id, $p1, $q1, $price1);  
INSERT INTO order_items (order_id, product_id, quantity, price)  
VALUES ($order_id, $p2, $q2, $price2);  
INSERT INTO order_items (order_id, product_id, quantity, price)  
VALUES ($order_id, $p3, $q3, $price3);  
  
UPDATE products SET stock = stock - $q1 WHERE id = $p1;  
UPDATE products SET stock = stock - $q2 WHERE id = $p2;  
UPDATE products SET stock = stock - $q3 WHERE id = $p3;  
  
DELETE FROM cart_items WHERE user_id = $1;  
  
COMMIT;
```

5.1.14. Задача 3. Baseline производительности

Источник: Percona PostgreSQL Benchmark 2024

Конфигурация: 8 vCPU, 32GB RAM, NVMe SSD (аналог AWS db.m5.2xlarge)

Результат: pgbench TPC-B — **3,100 TPS**

Это baseline для pgbench TPC-B нагрузки (5 DML операций) на данной конфигурации.

5.1.15. Задача 4. Расчёт max RPS

Подсчёт DML-операций:

Операция	Количество
SELECT	3
INSERT INTO orders	1
INSERT INTO order_items	3

Операция	Количество
UPDATE products	3
DELETE FROM cart_items	1
Итого DML	11

Эталон **pgbench TPC-B**: 5 DML (1 SELECT, 3 UPDATE, 1 INSERT)

Коэффициент сложности:

$$K = \frac{11}{5} = 2.2$$

Расчёт max RPS:

$$\max \text{RPS} = \frac{3100}{2.2} \approx 1409 \text{ запросов/сек}$$

Проверка через CPU-time (для 8-core сервера):

$$\text{CPU-time}_{\text{pgbench}} = \frac{8 \times 1000}{3100} \approx 2.6 \text{ мс}$$

$$\text{CPU-time}_{\text{наш}} = 2.6 \times 2.2 \approx 5.7 \text{ мс}$$

$$\text{CPU load} = 1409 \times 5.7 = 8031 \text{ мс/сек}$$

$$\text{CPU budget} = 8 \times 1000 = 8000 \text{ мс/сек}$$

Загрузка 100% — на пределе. Консервативная оценка: **1,300 RPS**.

5.1.16. Задача 5. Расчёт DAU

Параметры (обоснование):

- Активные часы: 12 (10:00–22:00 для e-commerce)
- Действий на пользователя: 20 (просмотры каталога, карточек, редкие покупки)
- Пиковый коэффициент: 2.5 (вечерний пик)

$$\text{DAU} = \frac{1300 \times 43200}{20 \times 2.5} = \frac{56160000}{50} \approx 1120000$$

Результат: 1.1 млн DAU

5.1.17. Проверка на здравый смысл

Факторы, которые могут снизить производительность:

1. JOIN'ы и агрегации (наш расчёт предполагал только точечный доступ по PK)
2. Long transactions и lock contention
3. Hot rows (если много пользователей обновляют одну строку)
4. Отсутствие индексов на часто используемых полях

Факторы, которые могут повысить производительность:

1. Read replicas для SELECT-запросов
2. Кэширование (Redis) для корзин и сессий
3. Connection pooling (PgBouncer)

5.1.18. Справочник

5.1.19. Эталонная нагрузка: pgbench TPC-B

Источник: PostgreSQL Documentation: pgbench

pgbench — официальный инструмент PostgreSQL. Режим TPC-B выполняет:


```

BEGIN;
UPDATE pgbench_accounts SET abalance = abalance + :delta WHERE aid = :aid;
SELECT abalance FROM pgbench_accounts WHERE aid = :aid;
UPDATE pgbench_tellers SET tbalance = tbalance + :delta WHERE tid = :tid;
UPDATE pgbench_branches SET bbalance = bbalance + :delta WHERE bid = :bid;
INSERT INTO pgbench_history (tid, bid, aid, delta, mtime)
VALUES (:tid, :bid, :aid, :delta, CURRENT_TIMESTAMP);
END;

```

Итого: 5 DML операций (1 SELECT, 3 UPDATE, 1 INSERT) — все точечные по PK.

5.1.20. Инструменты для других СУБД

- **MySQL:** sysbench — sysbench oltp_read_write
- **MongoDB:** YCSB — Yahoo! Cloud Serving Benchmark
- **Cassandra/ScyllaDB:** cassandra-stress

5.1.21. Формула CPU-time

$$\text{CPU-time на транзакцию} = \frac{N_{\text{cores}} \times 1000 \text{ мс/сек}}{\text{TPS}}$$

Если ваш endpoint в K раз сложнее эталона:

$$\text{CPU-time}_{\text{ваш}} = \text{CPU-time}_{\text{эталон}} \times K$$

Проверка: $\text{RPS} \times \text{CPU-time}_{\text{ваш}} \leq N_{\text{cores}} \times 1000$

5.1.22. Формула DAU

$$\text{DAU} = \frac{\text{RPS} \times \text{активные секунды}}{\text{действий на пользователя} \times \text{пиковый коэффициент}}$$

5.1.23. Практика 4: Специализированные индексы и поиск (Лекции 10-13: Inverted Index, векторный поиск, геопоиск)

5.1.24. Предисловие

Классические индексы (B-tree, hash) оптимизированы для точного поиска по ключу. Но многие задачи требуют **поиска похожих элементов**: текстовый поиск по словам, рекомендации по семантической близости, геопоиск “рядом со мной”. Для этих задач используются специализированные структуры данных: Inverted Index, векторные индексы (HNSW, LSH), пространственные индексы (R-Tree).

5.1.25. Задание

Для вашего варианта добавьте **три типа поиска**:

1. Полнотекстовый поиск (Inverted Index)

Реализуйте поиск по одному из:

- Интернет-магазин: поиск товаров по названию и описанию
- Социальная сеть: поиск постов по ключевым словам
- Сервис бронирования: поиск отелей по названию и удобствам
- Банк: поиск транзакций по описанию

Требования:

- Опишите структуру inverted index (term → list of documents)
- Покажите пример запроса (AND/OR терминов)
- Выберите решение: PostgreSQL full-text search, Elasticsearch, или custom
- Обоснуйте выбор

2. Векторный поиск (Embeddings + ANN)

Реализуйте семантический поиск:

- Интернет-магазин: “похожие товары” по описанию
- Социальная сеть: рекомендации постов по интересам пользователя
- Сервис бронирования: рекомендации отелей по предпочтениям
- Банк: детекция подозрительных транзакций (аномалии)

Требования:

- Опишите, как создаются эмбединги (модель: BERT, sentence-transformers)
- Выберите метрику близости: cosine similarity, L2, dot product
- Выберите ANN-индекс: HNSW, LSH, IVF
- Оцените размер индекса (число векторов × размерность × 4 байта)

3. Геопространственный поиск (только для магазина/бронирования/такси)

Если ваш сценарий подразумевает геоданные:

- Интернет-магазин: поиск ближайших пунктов выдачи
- Сервис бронирования: поиск отелей в радиусе 5 км

Требования:

- Выберите структуру: R-Tree, QuadTree, KD-Tree
- Покажите пример запроса (range query, kNN)
- Обоснуйте выбор

Формат ответа: документ с описанием каждого типа поиска, примерами запросов и обоснованиями.

5.1.26. Пример

1. Полнотекстовый поиск товаров

Задача: пользователь вводит “беспроводные наушники bluetooth”, нужно найти релевантные товары.

Решение: PostgreSQL Full-Text Search

Создание inverted index:

```
-- Добавляем tsvector колонку для поиска
ALTER TABLE products ADD COLUMN search_vector tsvector;

-- Создаём GIN индекс
CREATE INDEX idx_products_search ON products USING GIN(search_vector);

-- Заполняем search_vector (триггером при INSERT/UPDATE)
UPDATE products SET search_vector =
    to_tsvector('russian', name || ' ' || description);
```

Пример запроса:

```
SELECT id, name, ts_rank(search_vector, query) AS rank
FROM products,
    to_tsquery('russian', 'беспроводные & наушники & bluetooth') AS query
WHERE search_vector @@ query
ORDER BY rank DESC
LIMIT 20;
```

Как работает:

1. `to_tsvector` токенизирует текст → стемминг → вектор терминов
 - “беспроводные наушники” → 'беспровод':1 'наушник':2
2. `to_tsquery` создаёт запрос с операторами AND (&), OR (|)
3. GIN индекс: для каждого термина — список документов (posting list)
4. @@ оператор: пересечение posting lists для AND-запроса
5. `ts_rank`: ранжирование по TF-IDF

Обоснование выбора PostgreSQL:

- Встроенная поддержка (не нужен отдельный сервис)
- Достаточно для 100k товаров
- Поддержка морфологии (русский, английский)
- ☒ Elasticsearch был бы лучше для >1M товаров и сложного ранжирования

2. Векторный поиск похожих товаров

Задача: на странице товара показать “похожие товары” по описанию.

Решение: `sentence-transformers` + `pgvector` (PostgreSQL extension)

Создание эмбеддингов:

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('paraphrase-multilingual-mpnet-base-v2')

# Для каждого товара создаём эмбединг
for product in products:
    text = f"{product.name} {product.description}"
    embedding = model.encode(text) # 768-мерный вектор
    # Сохраняем в БД
```

Схема БД:

```
CREATE EXTENSION vector;
```

```
ALTER TABLE products ADD COLUMN embedding vector(768);
```

```
-- Создаём HNSW индекс для быстрого поиска
CREATE INDEX ON products USING hnsw (embedding vector_cosine_ops);
```

Поиск похожих товаров:

```
SELECT id, name,
    1 - (embedding <=> $1::vector) AS similarity
```

```
FROM products
WHERE id != $2 -- исключаем сам товар
ORDER BY embedding <=> $1::vector -- косинусное расстояние
LIMIT 10;
```

Метрики:

- Размерность: 768
- Метрика: **cosine similarity** (через vector_cosine_ops)
 - Обоснование: длина вектора не важна, важен только смысл описания
- Индекс: **HNSW** (Hierarchical Navigable Small World)
 - Recall 95%, latency 1-5 мс для 100k векторов

Оценка размера:

100,000 товаров × 768 dim × 4 байта = 307 МБ
 HNSW индекс добавляет ~2× overhead = 614 МБ
 Итого: ~1 ГБ (легко помещается в RAM)

Альтернативы:

- **LSH**: хуже recall (80%), но быстрее. Не нужно для 100k векторов.
- **Faiss** (Facebook AI Similarity Search): избыточно, pgvector достаточно

3. Геопоиск пунктов выдачи

Задача: найти ближайшие 5 пунктов выдачи к адресу пользователя.

Решение: PostGIS (R-Tree)

Схема:

```
CREATE EXTENSION postgis;
```

```
CREATE TABLE pickup_points (
  id SERIAL PRIMARY KEY,
  name VARCHAR(255),
  address VARCHAR(255),
  location GEOMETRY(Point, 4326) -- WGS84 координаты
);
```

```
-- Создаём GIST индекс (R-Tree)
```

```
CREATE INDEX idx_pickup_location ON pickup_points USING GIST(location);
```

kNN запрос (5 ближайших):

```
SELECT id, name, address,
       ST_Distance(location, ST_MakePoint(37.6173, 55.7558)::geography) AS distance_meters
FROM pickup_points
ORDER BY location <-> ST_MakePoint(37.6173, 55.7558)::geometry -- kNN оператор
LIMIT 5;
```

Range query (все пункты в радиусе 5 км):

```
SELECT id, name, address
FROM pickup_points
WHERE ST_DWithin(
  location::geography,
  ST_MakePoint(37.6173, 55.7558)::geography,
  5000 -- 5 км в метрах
);
```

Обоснование R-Tree:

- Оптимален для точек (пункты выдачи)
- Поддержка kNN и range queries
- Встроен в PostGIS

Альтернативы:

- ☒ **QuadTree**: хорош для равномерных данных (игры), но пункты выдачи кластеризуются в городах
- ☒ **KD-Tree**: тоже подошёл бы, но R-Tree лучше для неравномерных данных

5.1.27. Практика 5: Операционные компоненты + OLAP (Лекции 14-16: балансировка, авторизация, мониторинг, ETL)

5.1.28. Предисловие

Написать работающий код — только половина задачи. Продакшн-система требует операционных компонентов: защита от перегрузки (rate limiting), распределение нагрузки (load balancing), безопасность (аутентификация/авторизация), наблюдаемость (метрики, логи, трейсы). Также большинство систем нуждаются в аналитике — а это требует отдельного OLAP-хранилища и ETL-пайплайнов.

5.1.29. Задание

Дополните вашу OLTP-систему из Практики 3 **операционными компонентами**:

Часть 1: Rate Limiting и Load Balancing

1. Rate Limiting:

- Выберите алгоритм: Token Bucket, Sliding Window Log, Sliding Window Counter
- Определите лимиты: для авторизованных и неавторизованных пользователей
- Обоснуйте выбор

2. Load Balancing:

- Предположим, у вас 3 инстанса приложения
- Выберите алгоритм: Round Robin, Least Connections, IP Hash, Consistent Hashing
- Обоснуйте выбор

Часть 2: Security

1. Аутентификация:

- Выберите механизм: JWT, Session-based, OAuth 2.0
- Опишите flow (последовательность запросов)

2. Авторизация:

- Выберите модель: RBAC, ABAC
- Определите роли/права для вашего сценария

Часть 3: Observability

1. Метрики (Prometheus):

- Определите 5-7 ключевых метрик для мониторинга
- Укажите тип: Counter, Gauge, Histogram, Summary
- Напишите примеры PromQL-запросов

2. Логирование:

- Определите формат: structured JSON
- Какие поля включать (timestamp, level, user_id, trace_id, etc.)

3. Трассировка:

- Опишите span для типичного запроса (например, создание заказа)

Часть 4: OLAP и аналитика

1. Требования к аналитике:

- Какие отчёты нужны? (например: топ-товары, динамика продаж, RFM-анализ клиентов)

2. ETL Pipeline:

- Как передавать данные из OLTP (PostgreSQL) в OLAP (ClickHouse)?
- Выберите инструмент: Airflow, dbt, custom script
- Опишите schedule (real-time, hourly, daily?)

3. Расчёт объёма:

- Сколько данных в день передаётся в OLAP?
- Формат: Parquet, CSV, JSON?
- Компрессия: gzip, zstd?

Формат ответа: документ с архитектурной схемой и обоснованиями.

5.1.30. Пример

Часть 1: Rate Limiting и Load Balancing

1. Rate Limiting

Алгоритм: Token Bucket

Лимиты:

- Неавторизованные пользователи: 10 req/min (защита от парсеров)
- Авторизованные пользователи: 100 req/min
- API для мобильного приложения: 200 req/min
- Админы: без лимита

Конфигурация (nginx):

```
http {
    # Зона для rate limiting
    limit_req_zone $binary_remote_addr zone=general:10m rate=10r/m;
    limit_req_zone $user_id zone=authenticated:10m rate=100r/m;

    server {
        location /api/ {
            # Неавторизованные
            limit_req zone=general burst=20;

            # Если есть токен – используем зону authenticated
            if ($http_authorization) {
                limit_req zone=authenticated burst=50;
            }
        }
    }
}
```

Обоснование Token Bucket:

- Позволяет burst (всплески): пользователь может сделать 20 быстрых запросов, потом ограничение
- Простая реализация в nginx/Redis
- ❌ Sliding Window Log был бы точнее, но избыточен

2. Load Balancing

Алгоритм: Least Connections

Конфигурация (nginx):

```
upstream backend {
    least_conn; # выбирает сервер с наименьшим числом активных соединений

    server app1.internal:3000;
    server app2.internal:3000;
    server app3.internal:3000;
}

server {
    listen 80;
    location / {
        proxy_pass http://backend;
    }
}
```

Обоснование:

- Запросы неравномерные: создание заказа (долгий) vs просмотр товара (быстрый)

- Least Connections автоматически учитывает загруженность
- ❌ Round Robin не учитывает нагрузку — может перегрузить один сервер
- ❌ IP Hash не нужен (нет sticky sessions)

Часть 2: Security

1. Аутентификация: JWT

Flow:

1. POST /auth/login {email, password}
 - Сервер проверяет пароль
 - Возвращает JWT: {user_id, roles, exp}
2. Клиент сохраняет JWT (localStorage/cookie)
3. Все последующие запросы:


```
GET /api/cart
Authorization: Bearer <JWT>
```
4. Сервер проверяет подпись JWT (без обращения к БД!)

JWT structure:

```
{
  "header": {"alg": "HS256", "typ": "JWT"},
  "payload": {
    "user_id": 12345,
    "email": "alice@example.com",
    "roles": ["customer"],
    "exp": 1704067200 // срок действия: 1 час
  },
  "signature": "..."
```

Обоснование:

- Stateless: не нужно хранить сессии в Redis/БД
- Масштабируемость: любой инстанс может проверить JWT
- ❌ Session-based требует централизованное хранилище (Redis)

2. Авторизация: RBAC

Роли:

- customer: может просматривать товары, управлять корзиной, создавать заказы
- admin: может управлять товарами, просматривать все заказы
- support: может просматривать заказы, менять статусы

Права (permissions):

```
const (
    ReadProducts    = "products:read"
    WriteProducts   = "products:write"
    ReadOrders      = "orders:read"
    WriteOrders     = "orders:write"
    ReadAllOrders   = "orders:read:all" // свои vs все
    UpdateOrderStatus = "orders:update:status"
)

var roles = map[string][]string{
    "customer": {ReadProducts, WriteOrders, ReadOrders},
    "admin":    {ReadProducts, WriteProducts, ReadAllOrders, UpdateOrderStatus},
    "support":  {ReadProducts, ReadAllOrders, UpdateOrderStatus},
}
```


Middleware для проверки:

```
func RequirePermission(perm string) middleware {
    return func(c *Context) {
        userRoles := c.Get("roles") // из JWT
        if !hasPermission(userRoles, perm) {
            c.AbortWithStatus(403) // Forbidden
        }
    }
}

// Использование
router.POST("/products", RequirePermission(WriteProducts), createProduct)
router.GET("/orders", RequirePermission(ReadAllOrders), listAllOrders)
```

Обоснование RBAC:

- Простая модель для типичного e-commerce
- Роли статичны (customer, admin)
- ❌ ABAC избыточен (не нужны динамические правила вроде “только в рабочее время”)

Часть 3: Observability

1. Метрики (Prometheus)

Ключевые метрики:

```
import "github.com/prometheus/client_golang/prometheus"

// 1. Counter: общее количество запросов
var httpRequestsTotal = prometheus.NewCounterVec(
    prometheus.CounterOpts{Name: "http_requests_total"},
    []string{"method", "endpoint", "status"},
)

// 2. Histogram: латентность запросов
var httpDuration = prometheus.NewHistogramVec(
    prometheus.HistogramOpts{
        Name: "http_request_duration_seconds",
        Buckets: []float64{.001, .01, .1, 1, 10}, // 1ms, 10ms, 100ms, 1s, 10s
    },
    []string{"method", "endpoint"},
)

// 3. Gauge: активные соединения
var activeConnections = prometheus.NewGauge(
    prometheus.GaugeOpts{Name: "http_active_connections"},
)

// 4. Counter: заказы
var ordersCreated = prometheus.NewCounter(
    prometheus.CounterOpts{Name: "orders_created_total"},
)

// 5. Counter: ошибки БД
var dbErrors = prometheus.NewCounterVec(
    prometheus.CounterOpts{Name: "db_errors_total"},
    []string{"operation"}, // select, insert, update
)

// 6. Histogram: размер корзины
var cartSize = prometheus.NewHistogram(
    prometheus.HistogramOpts{
        Name: "cart_items_count",
    },
)
```

```

        Buckets: []float64{1, 3, 5, 10, 20},
    },
)

// 7. Gauge: остаток товаров на складе
var productStock = prometheus.NewGaugeVec(
    prometheus.GaugeOpts{Name: "product_stock"},
    []string{"product_id"},
)

```

PromQL запросы:

```

# 1. RPS (запросов в секунду)
rate(http_requests_total[1m])

# 2. Error rate (процент 5xx ошибок)
sum(rate(http_requests_total{status=~"5.."}[1m]))
/
sum(rate(http_requests_total[1m]))

# 3. p95 латентность
histogram_quantile(0.95,
    rate(http_request_duration_seconds_bucket[5m])
)

# 4. Количество заказов за последний час
increase(orders_created_total[1h])

# 5. Топ медленных эндпоинтов
topk(5,
    histogram_quantile(0.99,
        rate(http_request_duration_seconds_bucket[5m])
    ) by (endpoint)
)

```

Grafana дашборд (панели):

- RPS по эндпоинтам (линейный график)
- Error rate (линейный график с алертом > 1%)
- Латентность p50/p95/p99 (линейный график)
- Активные соединения (gauge)
- Количество заказов за день (counter)

2. Логирование

Формат: Structured JSON

```

{
  "timestamp": "2025-01-02T14:30:15.123Z",
  "level": "INFO",
  "service": "ecommerce-api",
  "trace_id": "a1b2c3d4-e5f6-7890",
  "span_id": "f1e2d3c4",
  "user_id": 12345,
  "method": "POST",
  "endpoint": "/api/orders",
  "status": 201,
  "duration_ms": 45,
  "message": "Order created successfully",
  "order_id": 98765
}

```

Пример ошибки:

```
{
  "timestamp": "2025-01-02T14:32:10.456Z",
  "level": "ERROR",
  "service": "ecommerce-api",
  "trace_id": "a1b2c3d4-e5f6-7890",
  "span_id": "f1e2d3c4",
  "user_id": 12345,
  "method": "POST",
  "endpoint": "/api/orders",
  "error": "insufficient stock",
  "product_id": 555,
  "requested": 10,
  "available": 3,
  "message": "Failed to create order"
}
```

Преимущества:

- Легко фильтровать по полям (Elasticsearch, Loki)
- Trace_id связывает логи одного запроса
- Можно парсить и строить метрики

3. Distributed Tracing (OpenTelemetry + Jaeger)

Span для создания заказа:

```
Trace: "POST /api/orders" (trace_id: abc-def)
├─ Span 1: "http.request" (45 mc total)
│   └─ Span 2: "validate_cart" (2 mc)
│       └─ Span 3: "redis.get cart:12345" (0.5 mc)
│           └─ Span 4: "db.transaction.create_order" (40 mc)
│               └─ Span 5: "db.insert orders" (10 mc)
│                   └─ Span 6: "db.insert order_items" (5 mc)
│                       └─ Span 7: "db.update products.stock" (15 mc) ← медленный!
│                           └─ Span 8: "send_email_notification" (3 mc async)
```

Вывод из трейса: обновление stock занимает 15 мс (самая медленная операция). Возможная оптимизация: батчинг UPDATE или использовать Redis для быстрой проверки остатков.

Часть 4: OLAP и аналитика

1. Требования к аналитике

Отчёты для бизнеса:

- **Топ-товары** (самые продаваемые за день/неделю/месяц)
- **Динамика продаж** (revenue по дням, сравнение с прошлым периодом)
- **RFM-анализ клиентов** (Recency, Frequency, Monetary — сегментация)
- **Конверсия корзины** (% пользователей, совершивших заказ)
- **AOV** (Average Order Value — средний чек)

2. ETL Pipeline (Airflow)

Архитектура:

```
PostgreSQL (OLTP)
  ↓ (каждый час)
Airflow DAG: extract → transform → load
  ↓
ClickHouse (OLAP)
```

Airflow DAG:

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime, timedelta
```

```

dag = DAG(
    'oltp_to_olap_etl',
    start_date=datetime(2025, 1, 1),
    schedule_interval='@hourly', # каждый час
)

def extract_orders():
    # Читаем новые заказы из PostgreSQL
    conn = psycopg2.connect("postgresql://...")
    df = pd.read_sql("""
        SELECT o.id, o.user_id, o.created_at, o.total_amount,
               oi.product_id, oi.quantity, oi.price
        FROM orders o
        JOIN order_items oi ON oi.order_id = o.id
        WHERE o.created_at >= NOW() - INTERVAL '1 hour'
    """, conn)
    df.to_parquet('/tmp/orders_hourly.parquet')

def transform_data():
    df = pd.read_parquet('/tmp/orders_hourly.parquet')
    # Добавляем метаданные
    df['extracted_at'] = datetime.now()
    df.to_parquet('/tmp/orders_transformed.parquet')

def load_to_clickhouse():
    df = pd.read_parquet('/tmp/orders_transformed.parquet')
    # Загружаем в ClickHouse
    client = clickhouse_connect.get_client(host='clickhouse.internal')
    client.insert_df('orders_fact', df)

extract_task = PythonOperator(task_id='extract', python_callable=extract_orders, dag=dag)
transform_task = PythonOperator(task_id='transform', python_callable=transform_data, dag=dag)
load_task = PythonOperator(task_id='load', python_callable=load_to_clickhouse, dag=dag)

extract_task >> transform_task >> load_task

```

Schedule:

- **Hourly** для оперативных дашбордов (динамика продаж в реальном времени)
- **Daily** для тяжёлых агрегаций (RFM-анализ)

3. Расчёт объёма

Данные в день:

10,000 заказов/день × 2 товара/заказ = 20,000 строк
 Размер строки: ~100 байт (order_id, user_id, product_id, quantity, price, timestamps)
 20,000 × 100 байт = 2 МБ/день

Формат: **Parquet**

- Колоночный формат (оптимален для ClickHouse)
- Встроенная компрессия (zstd)
- Эффективно для батчинга (hourly loads)

Компрессия:

2 МБ (raw) → ~0.5 МБ (Parquet + zstd)

Объём за год:

0.5 МБ/день × 365 = 182 МБ/год

ClickHouse схема:

```
CREATE TABLE orders_fact (  
    order_id UInt64,  
    user_id UInt64,  
    product_id UInt64,  
    quantity UInt16,  
    price Decimal(10, 2),  
    created_at DateTime,  
    extracted_at DateTime  
)  
ENGINE = MergeTree()  
PARTITION BY toYYYYMM(created_at) -- партиционирование по месяцам  
ORDER BY (user_id, created_at); -- сортировка для быстрых запросов по user_id
```

Пример запроса (топ-товары):

```
SELECT  
    product_id,  
    SUM(quantity) AS total_sold,  
    SUM(quantity * price) AS revenue  
FROM orders_fact  
WHERE created_at >= today() - INTERVAL 7 DAY  
GROUP BY product_id  
ORDER BY revenue DESC  
LIMIT 10;
```

Итого:

- OLTP → OLAP: hourly через Airflow
- Формат: Parquet (колоночный + компрессия)
- Объём: 0.5 МБ/день → 182 МБ/год (легко масштабируется)